NOVA Information Management School

Universidade Nova de Lisboa

Asset-Centric Temporal Graphs for Crude Oil Logistics: Schema Design and Consistency Guarantees for Multi-Resolution Network Data

Master Thesis in Information Management Systems

Pedro Porfírio

Student Number: 20241283

Supervisor: Professor Flávio Pinheiro

May 2026

Abstract

Crude oil logistics networks are complex systems. Not only the actual infrastructure is complex, data is published at multiple levels of resolution by different agencies. Reporting boundaries do not always match the physical system, some flows are directly observed while others can only be inferred, and the network itself changes over time as pipelines reverse, terminals open, or assets are retired. In practice, analysts often bridge these gaps with spreadsheets or notebooks built around custom allocation rules and residual adjustments. Those workflows can produce useful outputs, but they are usually difficult to audit, hard to reuse, and have dependencies that are not always explicit.d

This thesis develops and tests an asset-centric temporal graph schema designed to be able to describe the flow of oil and its products. The main porpuse is to generate a framework that is both flexible and robust so it can be extended and collapsed and can accommodate data with different levels of granularity. We focus on a framework that will describe the flow of oil and that is suitable for optimization using linear programming or Graph neural networks.

In order to accommodate the complexities of oil logistics, we focus on six axioms and their corollaries that will allow flexibility while mmaintaining consistency and extensibility.

We define a physical asset as the framework primitive: it represents actual physical infrastructure that can produce, consume, transport or store oil and its products. Abstract assets are representations of constraints or grpups of assets. Nodes represent how assets appear in a graph, with each node binding to exactly one asset. Physical assets (real, named entities with capacity, geography, and flow edges) and abstract (aggregation views over the physical layer with no edges of their own, with defined constraints). Physical assets carry their own inventory and mass-balance structure. The topology remains stable over time, with zero-flow edges representing inactivity rather than disappearance. Every physical asset follows a common mass-balance equation with an explicit balancing item; abstract assets inherit mass balance as a corollary of aggregation rather than as a separate constraint. Abstract assets are important as the granularity of data is more often than not at higher levels of granularity than the actual assets. Nodes have variables and they can be linked to formulas, time series, or values. Edges and aggregation links are derived from variables, which acts as the system's source of truth on how nodes interact..

The persistent asset graph is kept separate from scenario-specific assignments, the physical layer is distinguished from the observational aggregation layer, and geography metadata is kept separate from the resolution hierarchy. At schema level, a CHECK constraint enforces single attribution of value: each assignment row binds the variable to one source, a time series or a formula, never both. Aggregation relationships used for cross-checking can be declared but do not count as a second source. Structurally real but unobserved flows are treated as latent rather than filled with heuristic estimates. Bidirectional movements are represented as

separate directed edges, sum-type formulas are canonical, and latent values are distinguished from unresolved ones so that uncertainty is not hidden inside apparently complete data.

Each node carries several variables — production, consumption, inflows, outflows, inventory, and a balancing item — and the CHECK constraint applies to each variable's assignment individually, so a node with five variables has five independent assignment decisions in a given scenario. Mass balance is enforced at physical assets only; abstract assets are aggregation views that inherit teh variables but can introduce constraints.

The framework was implemented as a working system composed of a relational database, a deterministic resolver that produces a per-(scenario, variable, observation date) value table, a layered set of views for downstream use, and a full instance of the U.S. crude oil network covering January 2015 to December 2024. The implemented instance includes 251 assets, 433 directed flow edges, and 1,870 variables, of which 90 are time-series-observed under strict one-to-one attribution. The resolver runs in about 20 seconds and leaves no variables unresolved. Partition closure holds within rounding tolerance across all PADD aggregates and at the USA level, and mass balance is satisfied at every node. The difference between observed and closure-derived balancing items remains limited in normal periods and rises sharply around major disruption events such as Hurricane Harvey, COVID-19, the Colonial Pipeline shutdown, and the Strategic Petroleum Reserve release.

To show the flexibility of the design, we move from a crude only scenario to one where we assume the oil grades produced in each basin and show how the inventories evolve over time.

Keywords: logistics networks, graph databases, crude oil, mass balance, multi-resolution data, schema design, linear programming, asset-centric representation

Acknowledgements

I am very grateful to my supervisor, Professor Flávio Pinheiro, for his guidance throughout this work, for his readiness to engage with both early ideas and later drafts, and for repeatedly bringing the discussion back to what makes a thesis valuable as a contribution to the literature. Many of the design choices in this framework became clearer through those conversations.

I would also like to acknowledge the practitioners working in crude oil trading and analytics whose day-to-day workflows helped define the problem addressed in this thesis. The conventions used by the EIA, the IEA, and the wider public-data ecosystem reflect a great deal of accumulated practical knowledge, and it was important throughout this project to treat that material with the seriousness it deserves.

Finally, I would like to thank my wife that has endure many hours waiting for me to finish prototypes or finetune the database. I would like to thank NOVA IMS for their patience, encouragement, and support during the writing of this thesis. Many conversations along the way helped shape the work, even when that was not immediately obvious at the time.

1. Introduction

1.1 Background and motivation

Crude oil is one of the most volatile commodities in global markets, but it is also one of the most closely monitored. Producers, governments, refiners, and traders interact through spot cargoes, term contracts, and standardised futures linked to regional benchmarks. Even so, those financial instruments ultimately depend on the physical balance between supply and demand for crude itself. For that reason, serious quantitative work on oil markets usually starts with the physical system that produces, transports, stores, and consumes the commodity.

The physical layer of an oil network cannot react quickly to short-term economic signals. Well output is constrained by geology and reservoir conditions, so large and rapid production changes are technically difficult and expensive to implement (Fattouh, 2011; Ribas, Hamacher, and Street, 2010). Refineries face a similar kind of inertia: they are built for particular throughput ranges and crude slates, and moving outside those ranges can reduce efficiency and affect product quality (Fernandes, Relvas, and Barbosa-Póvoa, 2013; Lima, Relvas, and Barbosa-Póvoa, 2016). Even changing crude grade is rarely a simple commercial choice, because it may require unit reconfiguration and planned downtime. Transport introduces another layer of delay, as pipelines, ports, and tankers imply transit times measured in days or months (Stopford, 2009; Kazemi and Szmerekovsky, 2015). In practice, meaningful flow changes usually require lead time, capital, and coordination across several parts of the chain (Lima, Relvas, and Barbosa-Póvoa, 2016).

Because production and refining cannot adjust quickly, inventories absorb much of the system's short-term imbalance, and the level of stocks is one of the principal determinants of short-term crude price dynamics (Working, 1949; Pindyck, 2001; U.S. Energy Information Administration, 2024). Tank farms, storage terminals, and strategic petroleum reserves act as buffers between what is produced and what is consumed at a given moment. When stocks are comfortable, shocks can be absorbed with limited price impact. When stocks become tight, concerns about availability tend to push prices upward; when they become excessive, storage costs rise and prices usually soften. Storage also creates a hard physical limit: when usable capacity is effectively full, the marginal barrel has nowhere to go. The April 2020 WTI front-month settlement at −37 USD per barrel — a price at which the seller pays the buyer to take delivery — was a highly visible reminder of that constraint.

Within that physical system, each operational asset—whether an onshore well, an offshore platform, a gathering system, a pipeline segment, a terminal, or a refinery— produices, transport, consume or transform oil, and there is data that reflects the decisions of the asset's operators. Production rates, consumption volumes, inflows, outflows, storage levels, and capacity utilisation all change over time in response to internal constraints such as maintenance, equipment limits, and contractual commitments, as well as external shocks such as geopolitics, weather, demand shifts, and price movements. These relationships are not only complex but also time-varying. A disruption at an upstream production site does not stay

local for long; it propagates downstream through pipelines and storage nodes, with effects shaped by routing options, transit times, and operating priorities.

The main modelling tools used to study systems like this usually fall into two broad groups: operations research and classical time series methods.

Operations research, including linear programming, mixed-integer programming, and stochastic optimisation are mathematically rigorous and very useful for strategic design and tactical planning, but they often rely on simplified representations that leave out much of the operational detail visible in the real network. Moreover, the representation of the complex systems are usually bespoke, difficult to audit and change. The other group consists of classical time-series methods, such as ARIMA, VAR, and more recent neural forecasting models. These are typically applied to individual variables—refinery throughput, regional inventories, benchmark prices—without explicitly modelling the network structure that links those variables together.

Over the last decade, a third line of research has developed at the intersection of those two traditions. Graph neural networks and graph representation learning offer a way to model systems whose behaviour depends both on local time dynamics and on a broader relational structure. Recent surveys (Wu et al., 2020; Jin et al., 2024) document how quickly this literature has expanded across traffic networks, power systems, and, more recently, supply chains (Kosasih and Brintrup, 2022; Wasi et al., 2024; Ahn et al., 2024). What makes these methods especially relevant here is that they can bring together two aspects of the problem that classical OR and standard time-series approaches often handle separately: the network itself and the temporal behaviour of the variables attached to it.

Before any of these modelling approaches can be applied, however, the same upstream question appears: how should the network itself be represented? In the crude oil case, that question is still not well resolved. Available data data is released at multiple resolutions by overlapping agencies. Reporting boundaries do not line up neatly with operational ones. Some flows are directly measured, while others can only be inferred from aggregate constraints. Inventories often close only at coarser levels than the production they are supposed to balance against. And the topology changes over time through real events such as pipeline reversals, terminal commissioning, and refinery shutdowns. A representation that can handle these features cleanly would be useful not only for optimisation and forecasting, but also for the analysts who currently connect public data to downstream models through manual workflows.

1.2 Problem statement

Despite the maturity of the three research streams outlined above, there is still no widely adopted way of representing crude oil logistics networks that meets the needs of downstream users. Existing approaches do not deal cleanly with multi-resolution data without relying on ad hoc reconciliation. They do not enforce single attribution at schema level, do not preserve the provenance of derived quantities across the full resolution chain, and do not clearly separate

the slowly changing physical topology from the faster-changing data assignments attached to it.

Anyone who has worked with energy data will recognise the practitioner workflow that fills this gap. Analysts often maintain Excel workbooks or pandas notebooks where production, consumption, flows, and stocks for each PADD are loaded from a multitude of sources, reconciled through hard-coded allocation rules, and balanced with a residual adjustment term that absorbs whatever does not add up. Those workflows can produce useful numbers, and they are deeply embedded in day-to-day market analysis, but they do so by burying inconsistency inside the adjustment term. They are also difficult to generalise: each workbook reflects its author's assumptions, and none can be passed directly into an optimisation model or a machine-learning pipeline without further manual cleaning.

The central problem this thesis addresses is therefore: **how can the crude oil logistics network be represented in a way that admits multi-resolution data, enforces single-attribution structurally, preserves provenance through derivations, and is consumable by both classical optimisation models (linear programmes) and modern machine learning models (graph neural networks) without further hand-cleaning?**

This thesis answers that question by developing, implementing, and validating an asset-centric temporal graph schema organised around six axioms and the corollaries that follow from them, together with a deterministic resolver that produces a single per-(scenario, variable, observation date) value table that downstream consumers can read directly.

1.3 Research objectives

The research pursues three main objectives:

To **design** a graph schema that represents the crude oil logistics network at the level of physical assets, captures both structural relationships and time-series dynamics, enforces single attribution and partition closure as schema-level guarantees, and accommodates multi-resolution public data without treating reconciliation residuals as primary data.

To **implement** that schema as a working system, including a database representation, a deterministic resolver that produces the per-(variable, date) data tensor, a layered view structure for downstream consumers, and a complete starter instance covering the U.S. crude oil network from 2015 onwards.

To **validate** the framework by formulating linear programmes that consume its output, showing that the framework's consistency properties are sufficient for non-trivial optimisation tasks, and assessing its readiness for further downstream consumers such as graph neural networks for forecasting, which are left for future work.

1.4 Scope and delimitations

This research focuses on the representational framework and its first downstream consumer. The asset graph is built at the level of named physical assets wherever public data allows, with residuals used for un-named tail production. The geographic scope is the United States, the temporal resolution is monthly, and the initial implementation treats crude oil as a single commodity grade. The schema is designed to be extendable along all three dimensions—to refined products, to other countries, and to finer temporal granularity—and those extension paths are discussed where relevant. Even so, the implemented instance in this thesis is the U.S. monthly crude oil graph from January 2015 to December 2024 inclusive.

Two important areas are explicitly outside the scope of the present work and are positioned as future work in Chapter 10. The first is forecasting. Forecasting is treated in this thesis as a downstream use case that the framework is designed to support but does not itself implement. Chapter 10 explains why the framework is ready for forecasting work and outlines what that next step would involve. The second area is production deployment. The implementation developed here is a working research artefact rather than a production system, so issues such as pipeline reliability, user interfaces, monitoring, and access control are left aside.

Two pieces of vocabulary used throughout deserve flagging upfront. The framework distinguishes assets from nodes. An asset is a primitive entity: a refinery, a pipeline, a basin,, an export terminal. A node is how that asset appears in a particular graph. Each node binds to exactly one asset, and the same asset can appear in multiple graphs without being duplicated in the underlying data. The starter scenario uses a single graph, so in the current implementation each asset has exactly one node, but the separation is designed in: it lets per-grade, per-operator, or per-period views over the same physical reality be added later without restructuring the asset registry.

Assets carry a kind flag with two values. A physical asset is a real, named entity with capacity, geographic coordinates, configuration, and flow edges to its neighbours; refineries, pipelines, terminals, basins, gathering networks, and storage hubs are all physical assets. An abstract asset is an aggregation view over the physical layer (observational aggregates such as the district views, basin views, refining-district views, and the stock-decomposition aggregates of Section 5.4) and has no flow edges of its own. Abstract assets exist to host roll-up observations and partition relationships; their relationship to the physical layer is declared through formula_inputs rather than through edges. Mass balance is enforced at every physical asset, and abstract assets inherit it as a corollary (Section 4.3) — they do not introduce a new constraint.

Two extensions are worth flagging here because they recur in conversations about the framework. The first is per-grade decomposition: by treating commodity as a further dimension on every variable, each tank farm carries inventory per (asset, grade), and the evolving blend in storage is recoverable as the running sum of grade-specific inflows minus grade-specific offtakes — a per-(tank, grade) inventory series rather than a single aggregate. The second is refinery yield: the framework expresses refined-product output as a derived variable through

formula and formula_inputs, so a refinery's gasoline production can be defined as a fixed fraction of its consumption of a particular grade (gasoline_P = 0.35 × crude_C of grade X), with the yield coefficients themselves potentially scenario-specific. Both extensions sit on top of the schema as constructed; neither requires structural change.

1.5 Thesis structure

The thesis is organised in three parts. Chapters 1 to 3 frame the problem: this introduction, a focused literature review in Chapter 2 that examines the three streams discussed above and identifies the specific gap, and a domain background chapter in Chapter 3 on the U.S. crude oil network, which grounds the design choices that follow in the physical system itself.

Chapters 4 to 8 develop the main contribution. Chapter 4 sets out the design principles that govern the representation. Chapter 5 translates those principles into a concrete schema and graph-construction process. Chapter 6 describes the resolver that consumes the schema and produces the per-(variable, date) data tensor. Chapter 7 presents the consistency guarantees supported by the framework and the audits used to demonstrate them. Chapter 8 then introduces the linear programme that acts as the first downstream consumer of the resolved data.

Chapters 9 and 10 close the thesis. Chapter 9 presents a case study that follows the framework end to end on a concrete scenario. Chapter 10 summarises the contribution, discusses the framework's readiness for graph neural network forecasting as a natural next consumer, and outlines a broader roadmap for future development.

The annexes provide supporting reference material. Annex A offers a primer on graph neural networks for readers who want to follow the GNN-readiness discussion in Chapter 10 in more detail. Annex B compares asset-centric and location-centric graph representations. Annex C documents the resolver's dispatch rules, and Annex D catalogues the variables included in the starter scenario.

2. Literature Review

This thesis sits at the intersection of three research streams: petroleum supply chain optimisation, which has produced many of the dominant tools used to analyse crude oil logistics; graph data models and graph representation learning, which provide the methodological basis for downstream forecasting and relational modelling; and energy statistics conventions, which shape how the underlying public data is published and used in practice. This chapter reviews each stream with a focus on what it contributes and, just as importantly, what it leaves unresolved. It then brings those strands together to clarify the gap that motivates the present work.

2.1 Petroleum supply chain optimisation

The literature on crude oil and refined-product supply chains is extensive and well established. Most contributions fall into one of two broad methodological groups: deterministic mathematical programming, especially mixed-integer linear programming and related formulations, and uncertainty-aware extensions such as stochastic and robust optimisation.

Fernandes, Relvas, and Barbosa-Póvoa (2013) develop a deterministic MILP for the strategic design of multi-entity, multi-echelon, and multi-product downstream petroleum supply chains, using the Portuguese network as a case study. Their model captures the facility-location and capacity decisions that distinguish strategic network design from day-to-day planning. Kazemi and Szmerekovsky (2015) extend this line of work to multimodal transportation, showing that the joint treatment of pipeline, rail, marine, and truck transport can materially change the optimal solution when compared with single-mode formulations. Ghaithan, Attia, and Duffuaa (2017) move toward tactical planning in an integrated oil and gas setting, formulating a multi-objective optimisation model that balances cost, revenue, and service-level targets in a Saudi Arabian case.

On the uncertainty side, Ribas, Hamacher, and Street (2010) develop stochastic programming models for integrated oil-chain planning under uncertainty in demand, production, and prices. Lima, Relvas, and Barbosa-Póvoa (2016), in their broad review of downstream oil supply chain management, show how common robust and stochastic formulations have become in this field. One recurring issue in that review is especially relevant here: the level of detail at which optimisation models are built often does not match the level of detail at which real input data is actually available.

What these contributions have in common is the level of abstraction at which the network is represented. Nodes are usually large aggregates—refineries, demand centres, production regions—and edges are abstract connections defined in terms of cost, capacity, or lead time. That is entirely appropriate for the strategic and tactical questions these models are meant to answer, such as where to locate capacity or how to allocate supply across regions. It is much less helpful, however, for the separate question of how to represent the system itself in a way that can absorb multi-resolution data and support different types of downstream consumers.

A related issue is that this literature often treats supply chain modelling and logistics modelling as if they were effectively the same. They are clearly related, but they are not identical. Classical supply chain optimisation is mainly concerned with questions of multi-firm coordination, procurement, facility location, and strategic capacity—how the network should be designed and how commitments should be distributed across firms. Logistics network modelling is closer to the operational layer of a given network: how flows move through fixed assets, where inventories accumulate, and what kinds of consistency the available data actually supports. This thesis belongs to that second tradition. The design choices that follow—asset-centric nodes, stable topology, and schema-level consistency—respond to logistics requirements rather than to the more aggregated concerns of supply chain design.

2.2 Graph data models for commodity and logistics networks

The use of graph data models in commodity networks specifically—as distinct from more general supply chain applications—is relatively recent. Kosasih and Brintrup (2022) show that GNN-based methods can predict hidden links in supply chain networks, revealing dependencies that aggregate-level data tends to miss. Wasi et al. (2024) introduce SupplyGraph, the first benchmark dataset explicitly designed for GNN-based supply chain analytics, with tasks such as demand forecasting, production planning, and product classification. Ahn et al. (2024) develop a probabilistic GNN model for supply and inventory prediction using data from a global consumer-goods company, showing that attention-based graph architectures can improve predictive performance by capturing cascading effects across nodes.

These studies are important because they show that graph-based methods can add value in commodity and logistics settings beyond what isolated node-level time-series models usually provide. At the same time, they share a limitation from the perspective of this thesis: each is built around its own application-specific data representation. In other words, the representation is tailored to the task rather than treated as a contribution in its own right. SupplyGraph, for example, is structured around production lines and demand centres in a fast-moving consumer-goods setting. Its node types, edge meanings, and temporal resolution make sense there, but they do not transfer directly to crude oil logistics.

Behind these applied studies sits a broader graph representation learning literature. Foundational work on random-walk embeddings (Perozzi et al., 2014; Grover and Leskovec, 2016; Tang et al., 2015) and on graph neural networks more directly (Kipf and Welling, 2017; Veličković et al., 2018; Hamilton, Ying, and Leskovec, 2017) established the methodological base that later work builds on. Surveys by Wu et al. (2020) and Jin et al. (2024) provide useful overviews of this rapidly expanding field, with the latter focusing specifically on the interface between graph methods and time-series modelling. Temporal extensions such as TGAT (Xu et al., 2020), TGN (Rossi et al., 2020), and causal anonymous walks (Wang et al., 2021), together with hybrid architectures like DCRNN (Li et al., 2018) and STGCN (Yu, Yin, and Zhu, 2018), make the temporal dimension an explicit part of the modelling framework.

The traffic forecasting literature is particularly relevant to the present work, especially because DCRNN and STGCN have become standard reference models in that area. Traffic networks share several structural features with crude oil logistics networks: a largely stable physical topology, clearly identified nodes, and forecasting targets that depend both on local temporal behaviour and on dependencies that propagate through the network. The comparison is not perfect, but it is close enough to be useful. One of the goals of the framework developed in this thesis is to make that methodological transfer from traffic to commodity logistics possible in a more concrete and principled way. Annex A provides a fuller methodological background for readers who want to follow the GNN-readiness discussion in Chapter 10 more closely.

The comparison is informative as much for its differences as for its similarities. Traffic networks carry many independent agents — individual drivers responding to local signals on a second-by-second timescale — while crude oil networks carry few large commercial entities whose flow decisions are negotiated through contracts and nominated weeks in advance. Traffic data is dense, with GPS-derived counts on every monitored road; crude oil data is sparse, with a handful of monthly EIA series defining what is publicly observable. Mass balance in traffic is the conservation of vehicles across an intersection, with no explicit inventory; mass balance in oil is the conservation of barrels at every junction, with substantial inventory carried in tank farms, on vessels, and as line-fill in the pipelines themselves. Several oil pipelines reverse direction over the modelled period (Capline in 2022, Seaway in 2012); equivalent topology changes are rare in traffic networks at any useful temporal granularity. The structural parallel — fixed topology, identified nodes, flows with both local and propagating dependencies — is genuine, but the parallels in data density and operational timescale need to be qualified before traffic-network methods are adopted directly.

2.3 Energy statistics conventions and the practitioner workflow

The third stream is less academic in form, but it is closer to the practical problem that motivated this thesis. It concerns the conventions through which energy statistics are published by national and international agencies, and the analytical workflows built around those conventions.

The U.S. Energy Information Administration publishes a wide range of monthly series covering crude oil production, refining, stocks, trade, and movements between PADDs. These series are organised both by data type and by geographic resolution, ranging from national and PADD-level views to refining districts and, in a few cases, facility-level detail. The same physical quantity often appears at several resolutions at once. U.S.-level production is reported separately from PADD-level production, which is in turn distinct from named-basin production within each PADD. These series do not always reconcile exactly across levels, and the differences usually reflect reporting timing, custody-transfer conventions, or coverage choices rather than simple measurement error.

PADD stands for Petroleum Administration for Defense District. The five PADDs were established in 1942 to coordinate wartime petroleum logistics and have remained the primary regional grouping used by EIA ever since: PADD 1 covers the East Coast, PADD 2 the Midwest, PADD 3 the Gulf Coast, PADD 4 the Rocky Mountain region, and PADD 5 the West Coast. EIA reports production, refining, consumption, stocks, and inter-district movements at this resolution. The framework's partition tree uses the same five-district structure as its primary geographic spine.

The International Energy Agency follows a similar pattern at the international level in its monthly Oil Market Report and annual World Energy Balances. Importantly, the IEA retains an explicit balancing item in its supply-demand tables, defined as the residual between observed stock changes and the sum of observed flows. That convention matters here because it treats the residual as information rather than as noise to be hidden. The size and sign of the balancing item indicate where the reporting system is not closing cleanly, and the IEA does not try to make that issue disappear by silently folding it into nearby quantities.

The workflow built around these series is familiar across trading firms, consultancies, and integrated oil companies, and it is the immediate practical context for this thesis. Analysts typically maintain spreadsheets or pandas notebooks that ingest EIA and IEA series, reconcile them across resolutions as best they can, and then produce derived quantities such as implied flows, regional balances, or term-structure indicators. The workflow works, in the sense that it produces usable outputs, but it also has well-known weaknesses.

First, **reconciliation is implicit and inconsistent across analysts.** Each workbook makes its own hardcoded allocation rules and absorbs the residual into a custom "adjustment" term. Two analysts working on the same underlying data produce different derived quantities because their reconciliation choices differ.

Second, **provenance is lost.** A derived quantity in a typical workbook depends on a chain of allocations and adjustments, and tracing any specific value back to its inputs requires reverse-engineering the workbook. This makes the workflow difficult to audit and difficult to extend.

Third, **multi-resolution mixing is hand-coded.** When a finer-resolution series becomes available — for example, EIA's PADD-level stock decomposition into commercial and SPR portions — incorporating it requires modifying the workbook's allocation logic, often introducing inconsistencies with the legacy resolution that was previously authoritative.

EIA was chosen for this thesis because it is public, free to access, and covers the United States consistently at monthly resolution from 2015 onward. Commercial sources occupy a complementary niche. Providers such as OilX, Genscape, Wood Mackenzie's IIR, and S&P Global Commodity Insights collect higher-frequency and finer-granularity data through satellite imagery, gauge-level sensors, and survey panels, often resolving individual storage-tank levels and per-pipeline flows that public data aggregates away. The framework is designed to consume both. A scenario is defined as a coverage of authoritative bindings, and different

scenarios can bind different variables to different sources; a single scenario can also mix public and commercial sources where each is available. The same physical asset graph therefore supports a public-only scenario for reproducibility, a commercial-augmented scenario for accuracy where licensed data is available, and per-source comparison scenarios for assessing where the cheapest data is good enough.

Part of the purpose of this thesis is to turn that informal workflow into a structured representation. The axioms and corollaries introduced in Chapter 4 respond directly to the three limitations just described: schema-level single attribution addresses the problem of inconsistent reconciliation, the resolver's audit trail addresses the loss of provenance, and the separation between the persistent graph and scenario-specific state addresses the fragility of hand-coded multi-resolution mixing.

2.4 Synthesis and the research gap

Taken together, the three streams reviewed above point to a clear gap. The petroleum supply chain optimisation literature offers mathematical rigour using network abstractions that leave out both operational detail and the multi-resolution structure of the data. The graph data models literature offers increasingly powerful methods for relational and temporal learning assuming that the upstream representation is already clean and well formed. The energy statistics literature shows how the data is actually published and used, yet the workflow that connects those publications to downstream models remains largely ad hoc and difficult to audit or extend.

We present an upstream representation that: (a) admits the multi-resolution structure of public energy data without requiring each downstream consumer to reconcile it independently; (b) enforces single attribution and consistency at schema level rather than through post-hoc checks; (c) preserves the provenance of derived quantities so that any value can be traced back to its sources; (d) can be consumed by both classical optimisation models, such as linear programmes, and modern machine learning models, such as graph neural networks, without structural modification; and (e) exists as a working artefact rather than as an abstract proposal.

The asset-centric temporal graph methodology presented in Chapter 4 is the result of numerous designs that led to the implementation described in Chapter 5, the resolver presented in Chapter 6, the consistency guarantees demonstrated in Chapter 7, and the linear-programming consumer developed in Chapter 8. Graph neural network consumers are not implemented here, but their role as a natural next step is discussed in Chapter 10, where the argument for the framework's readiness is developed.

While EIA data was used to develop the framework, the work focus on the different formats, frequency and granularity of available data. From single position of vessels from Kpler, specific

flows and storage volumes from Genscape, to time series provided by the International Energy Agency. Flexible, consistent and scalable were obsessions throught the work.

3. Domain Background: The U.S. Crude Oil Network

This chapter describes the physical and informational structure of the U.S. crude oil network at the level needed to make the design choices in later chapters clear. It is not intended as a full overview of the petroleum industry, which would go well beyond the scope of this thesis. Instead, the aim is narrower: to outline the assets, flows, and reporting conventions that the framework must represent. Readers already familiar with the industry may wish to skim parts of the chapter. For readers coming from a methods background, however, the discussion provides the practical context for the design principles introduced in Chapter 4.

3.1 The physical layer

At the scale of the United States, the crude oil system can be described in terms of four broad physical layers: upstream production, midstream gathering and transportation, downstream refining, and the storage and export infrastructure that links the domestic network to the international market.

The **upstream layer** consists of the assets that bring crude oil to the surface. In the U.S. context, these fall into three main categories. The first is onshore conventional production, the historical core of U.S. output, based on wells drilled into permeable reservoirs and produced through natural pressure or artificial lift. The second is offshore production, concentrated in federal waters in the Gulf of America (formerly the Gulf of Mexico), where large fixed and floating platforms operate above sub-sea wells. The third is shale and tight oil production, which reshaped U.S. supply from around 2010 onward through horizontal drilling and hydraulic fracturing. The Permian in Texas and New Mexico, the Bakken in North Dakota and Montana, and the Eagle Ford in Texas are the most important examples, and together they shifted the centre of U.S. production decisively toward these basins.

The **midstream layer** moves crude from where it is produced to where it is stored, refined, or exported. Gathering systems collect output from many individual wells and convert it into custody-grade volumes suitable for longer-distance transport. From there, trunk pipelines carry crude across hundreds or sometimes thousands of miles between producing regions, hubs, storage terminals, and refining centres. Marine transport handles longer-distance and international movements, especially through the Gulf Coast export system, using vessel classes such as Aframax, Suezmax, and VLCC. Rail and truck move much smaller volumes in aggregate, but they remain important where pipeline coverage is limited, as was particularly visible during the early Bakken expansion before sufficient pipeline capacity was built.

The **storage layer** sits between midstream and downstream and absorbs much of the system's short-term imbalance. Commercial tank farms at major hubs such as Cushing, Patoka, Houston, St. James, and the LOOP system provide the working storage used by refiners, traders, and pipeline operators. Alongside them sits the Strategic Petroleum Reserve, a set of underground salt-cavern sites on the Gulf Coast that the U.S. government can draw on during major supply disruptions.

The **downstream layer** is where crude oil is converted into refined products. U.S. refineries number in the low hundreds and are concentrated most heavily in PADD 3 on the Gulf Coast, with smaller but still important clusters in PADD 2, PADD 1, PADD 5, and PADD 4. These refineries differ materially in complexity, capacity, and crude slate. Some smaller plants process less than 50 kbd, while the largest Gulf Coast sites exceed 600 kbd. Complexity also varies, commonly summarised through the Nelson Complexity Index, as does the mix of crude grades each refinery is designed to run efficiently. For the purposes of this thesis, the downstream layer marks the point at which crude exits the modelled system and reappears as refined products.

The figure below shows the physical structure schematically, with arrows indicating the direction of crude flow through the chain.

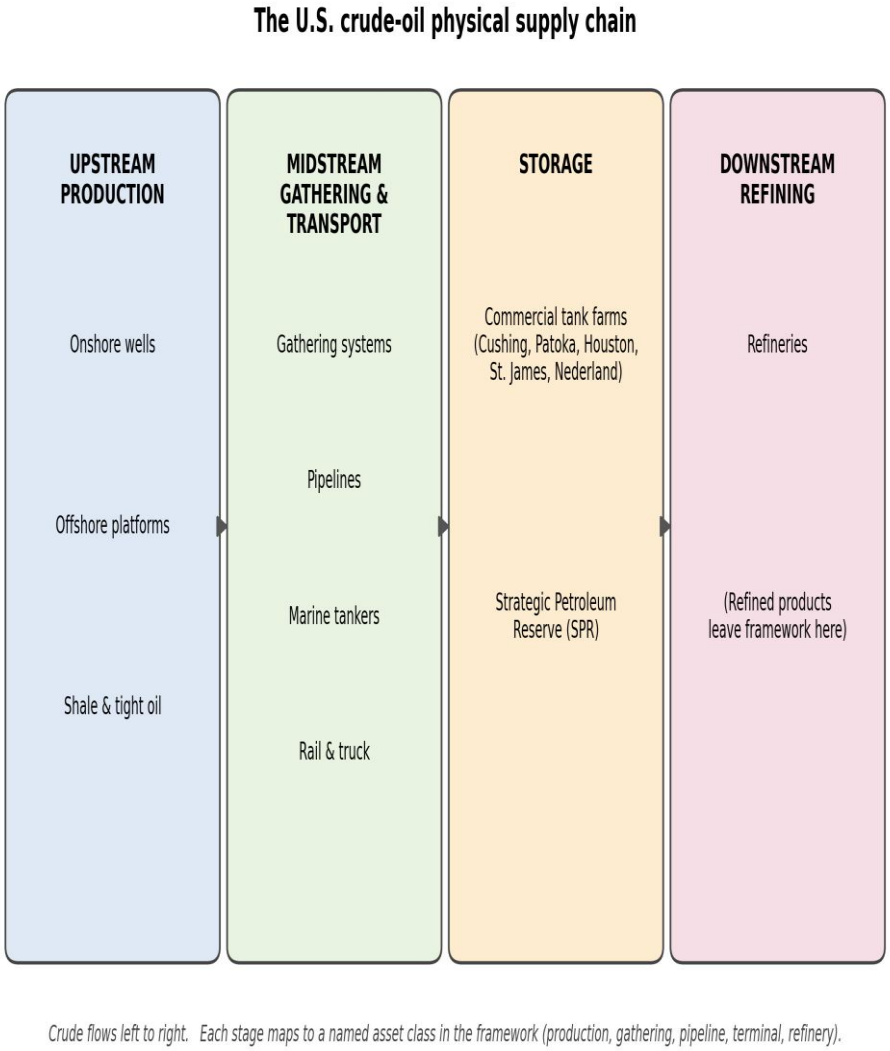


Figure 1. The crude oil physical supply chain: upstream production (onshore wells, offshore platforms, shale and tight oil), through midstream gathering and transport (pipelines, tankers, rail and truck), through storage (terminals and SPR), into downstream refining and refined products.

3.2 The PADD structure

The U.S. Energy Information Administration organises petroleum statistics around five Petroleum Administration for Defense Districts, or PADDs. These districts were created during the Second World War for fuel rationing and distribution purposes, but they have remained the dominant geographic structure for U.S. petroleum reporting ever since. The five PADDs group states as follows:

PADD	Coverage	States
PADD 1	East Coast	Connecticut, Delaware, District of Columbia, Florida, Georgia, Maine, Maryland, Massachusetts, New Hampshire, New Jersey, New York, North Carolina, Pennsylvania, Rhode Island, South Carolina, Vermont, Virginia, West Virginia
PADD 2	Midwest	Illinois, Indiana, Iowa, Kansas, Kentucky, Michigan, Minnesota, Missouri, Nebraska, North Dakota, Ohio, Oklahoma, South Dakota, Tennessee, Wisconsin
PADD 3	Gulf Coast	Alabama, Arkansas, Louisiana, Mississippi, New Mexico, Texas
PADD 4	Rocky Mountain	Colorado, Idaho, Montana, Utah, Wyoming
PADD 5	West Coast	Alaska, Arizona, California, Hawaii, Nevada, Oregon, Washington

Within the PADD structure, the EIA also reports refinery activity through refining districts such as PADD 3A, 3B, and 3C, or PADD 2A, 2B, and 2C. These are reporting groupings rather than physical assets. Their importance for this thesis lies in the way they impose multiple coexisting resolutions on the published data. Production may be reported at state-basin level, consumption at refining-district level, stocks at PADD level, and movements at the level of PADD pairs. Each of those levels is authoritative for some variables. A workable representation therefore needs to accommodate all of them at once without forcing everything into a single finest-granularity view and then hiding the resulting mismatches in reconciliation residuals.

3.3 The production geography

Over the period modelled in this thesis, from 2015 onward, U.S. crude oil production is dominated by four producing regions with clearly different geological and operational profiles:

Permian Basin. The Permian, stretching across west Texas and southeastern New Mexico, is the largest producing basin in the United States and in recent years has accounted for

roughly 40 per cent of total U.S. crude output. Production exceeded 6 mbd in 2024. It is a shale and tight oil province whose growth from around 1 mbd in 2010 to more than 6 mbd by 2024 was driven by horizontal drilling and hydraulic fracturing. For this thesis, the basin is especially useful because it is split by the Texas-New Mexico border and is therefore reported by the EIA through separate series. That makes it a very clear example of the multi-resolution problem the framework is designed to handle.

Bakken. Spanning western North Dakota and northeastern Montana, the Bakken was the first major shale play to industrialise (peak production around 2014–2015) and remains the second-largest U.S. light tight oil basin. Production peaked at approximately 1.5 mbd before declining; current output is around 1.2 mbd. The Bakken straddles a state line (North Dakota vs Montana), and again EIA reports the two halves separately.

Eagle Ford. Located entirely within Texas, the Eagle Ford shale produces both crude oil and condensate. Its single-state location simplifies the reporting picture: Eagle Ford production is the entirety of certain EIA Texas series.

Gulf of America offshore. Federal waters in the Gulf produce approximately 1.8 mbd in the modelled period, dominated by deepwater platforms operated by major integrated oil companies. EIA reports Gulf production as a single series rather than disaggregating by platform or field.

Beyond these four major regions, smaller volumes are produced in Alaska, Oklahoma, California, Wyoming, and other parts of the country. Most of these are reported at state level. At the same time, the EIA's PADD totals usually exceed the sum of the named basins and fields published within each district. That residual volume reflects production that is real but not separately named at the resolution available in public data. It is this gap between the published total and the named contributors that motivates the residual-node design discussed later in Chapter 5.

The basins differ substantively in the crude grade they produce, and that grade matters operationally because each refinery is configured for a particular slate. The starter scenario treats crude as a single commodity, but the framework's commodity dimension on variables admits per-grade decomposition without schema modification. The table below summarises the dominant grade by producing region; refined-product yield modelling — expressing, for example, a refinery's gasoline production as a fixed fraction of its consumption of a particular crude grade — is a natural extension of the same machinery.

Producing Basin	Typical Grade	API (°)	Sulphur	Notes
Permian (TX/NM)	WTI / WTI Midland	38–42	Sweet (~0.4%)	Largest US producer; benchmark grade
Bakken (ND/MT)	Bakken Light	~42	Sweet (~0.2%)	Light tight oil
Eagle Ford (TX)	Eagle Ford Light + condensate	40–65	Sweet	Light to very light / condensate
Gulf of America	Mars, Poseidon, Thunder Horse blends	28–33	Medium-sour	Offshore deepwater
Alaska North Slope	ANS	~31	Medium-sour (~0.9%)	Pipeline to TAPS / Valdez
California (onshore)	Various heavy grades	13–25	Sour	Demands cokers / hydrocrackers
Oklahoma (conv.)	WTI-like / regional	~38	Sweet	Mature conventional
Wyoming / Rockies	Mixed (light to heavy)	varies	Mixed	Rockies aggregator

3.4 The transport network

The transport network links producing regions to refining centres, storage hubs, and export terminals. At a high level the U.S. network is dense, and most production areas sit within a few hundred miles of major refining or storage infrastructure. But the pipeline-by-pipeline structure still matters, especially when the question is not simply where oil moves in aggregate, but which parts of that movement are directly observed and which remain hidden in public data.

Permian outbound. The Permian's roughly six-million-barrel-per-day output leaves the basin mainly through three origin terminals: Midland, the largest and most central; Wink, further to the southwest; and Crane, to the southeast. From those terminals, multiple long-haul pipelines move crude toward the Gulf Coast, including Gray Oak, Cactus II, EPIC, Wink-to-Webster, and others. Public data captures the basin's production and the receipts at major Gulf Coast destinations, but it does not report the split across individual pipelines. This is the clearest example in the thesis of a flow that is physically real and operationally important, yet unobserved at the per-edge level. It is therefore the canonical latent-flow problem used throughout the framework design.

Bakken outbound. The Bakken connects to PADD 2 (Midwest) and onward to other PADDs through a network of pipelines (Dakota Access being the dominant trunk) and rail terminals. The per-pipeline split out of Bakken gathering is again unobserved in public data.

The Cushing complex. Cushing, Oklahoma is the largest crude oil storage hub in the U.S. and the delivery point for the WTI futures contract. Pipelines converge on Cushing from all

directions: north from Texas (Seaway north-bound), south to the Gulf (Seaway south-bound, MarketLink, Keystone XL, Cushing Marketlink), east to Patoka (Diamond, Mid-Valley), and into Cushing from the north (Pony Express from the Bakken via Wyoming). The Cushing hub itself comprises multiple operator sub-terminals (Enterprise, Plains, Magellan, and others), each reporting separately at quarterly intervals but aggregated to hub level monthly.

Inter-PADD flows. The EIA reports crude movements between PADDs at the level of PADD pairs, such as PADD 2 to PADD 3 or PADD 3 to PADD 1, without specifying which individual pipelines carry those volumes. The framework's inter-PADD aggregate nodes are a direct response to that reporting choice. They give the partition structure a same-type child to attach to while leaving the underlying per-pipeline allocation latent for future optimisation or forecasting work.

The Gulf Coast export complex. By 2024, roughly 4 mbd of U.S. crude was leaving the country through Gulf Coast export terminals. The main centres are the Houston complex, the Corpus Christi cluster, Nederland in southeast Texas, LOOP in offshore Louisiana, and St. James in Louisiana. Some of these are themselves groups of terminals operated by different companies. Throughput can often be observed at terminal level, but the final destination of those exports is usually reported at country level rather than terminal-destination level. This creates another multi-resolution mismatch: the physical exit point is known at one level, while the destination pattern is known only at a coarser one.

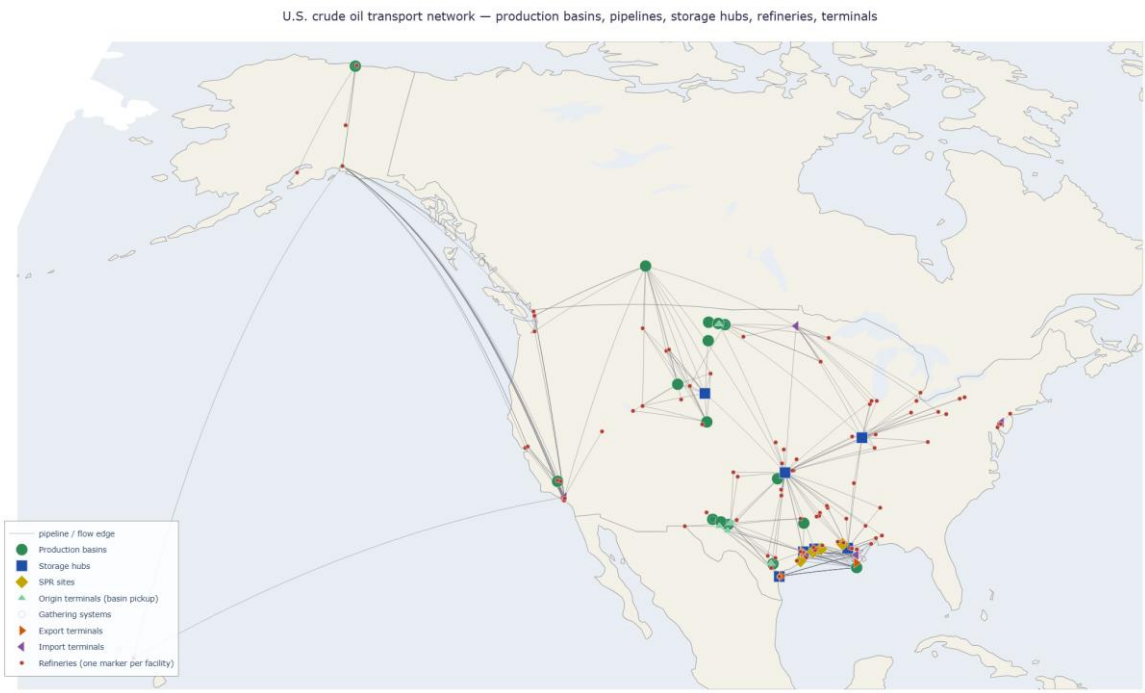


Figure 2 — U.S. crude oil transport network as instantiated in the framework. Production basins (green) feed gathering systems (light blue) and origin terminals (light green). Pipelines (grey lines) connect basins, hubs, and refineries (red). Storage hubs (dark blue) sit at the major junctions; SPR sites (mustard) cluster along the Gulf coast. Export terminals (orange) and import terminals (purple) sit on the boundary of the modelled system.

3.5 Refining and consumption

U.S. refining capacity is roughly 18 mbd, while actual crude runs typically fall in the 15–17 mbd range. Capacity is highly concentrated geographically. PADD 3 on the Gulf Coast accounts for around half of total U.S. refining capacity, and within that district a relatively small number of large coastal refineries dominate. PADD 2 in the Midwest is the next largest refining centre, with facilities that process a mix of domestic and Canadian crude and, in several cases, are configured specifically for heavier Canadian grades.

Refineries differ in several ways that matter operationally: throughput capacity, process configuration, complexity, and crude slate. Configuration refers to the units installed on site, such as crude distillation, coking, hydrocracking, fluid catalytic cracking, alkylation, and reforming. Complexity is often summarised through the Nelson Complexity Index. Crude slate refers to the range of grades the refinery can run efficiently. That flexibility is limited and economically important. A refinery optimised for heavy sour Canadian crude cannot simply switch to light sweet Permian barrels without consequences for operations and yields.

Within the framework, refineries are represented as nodes with three main variables: crude consumption, inventory, and a balancing item. Consumption corresponds to refinery throughput and is observed either at refining-district level or, in some cases, at facility level. Inventory is generally available at PADD level. The balancing item absorbs process losses and reporting residuals. Product slates and the detailed transformation of crude into refined products are outside the scope of the first implementation, which treats crude oil as a single commodity.

3.6 What the framework must accommodate

Taken together, the physical and reporting structure described above leads to a fairly specific set of requirements for the framework:

Multi-resolution data without reconciliation residuals. Production observed at basin level, stocks at PADD level, consumption at refining-district level, and inter-PADD flows at PADD-pair level all need to coexist in a single representation. The framework must not require any of these to be "down-disaggregated" to a uniform finest-granularity before being usable.

Junction fan-outs with unobserved per-edge splits. The Permian outflow problem (three terminals, multiple pipelines from each, downstream destinations observed only at aggregate level) is structural, not exceptional. Bakken gathering, the Cushing hub, and the Gulf Coast export complex all exhibit the same pattern at different scales. The framework must handle these natively rather than treating them as edge cases.

A balancing item that absorbs systematic reporting residuals. The IEA convention of a published balancing item exists for a reason: the data does not close exactly, and the residual carries information about where the reporting system is failing to close. The framework must retain B as a first-class variable rather than absorbing it into adjacent flows.

Slow structural evolution. The pipeline network changes deliberately over time (Capline reversed in 2022, the Bakken cross-state connector was added in 2024). These are real operational events, not data artefacts, and the framework must accommodate them without requiring a wholesale graph rebuild.

A boundary that separates the modelled system from the rest of the world. Foreign supply comes in through Canadian and other import flows; U.S. crude leaves through export terminals to destinations the framework does not track. The boundary needs to be explicit, with mass balance closing on the U.S. side without requiring the framework to model the foreign side.

These requirements are not optional add-ons. They follow directly from the way the system operates and from the way the data is actually published. The next chapter responds to them by introducing the axioms and corollaries that structure the framework.

4. Design Principles

This chapter sets out the six axioms that underpin the framework, together with the corollaries that follow from them once the variables collection is populated correctly. Of these, Axiom 3 does the heaviest architectural work: by treating the variables collection as the single source of truth, it makes edges, hierarchy, and node status all derivable rather than separately stored. Each principle is presented in the same way. The rule is stated, then an explanation of why the decision ws made and some of the alternatives that wee considered during design, and finally discuss what that choice allows the framework to do.

Taken together, the axioms and corollaries define the consistency properties claimed as the contribution of this thesis: an asset-centric representation that can handle multi-resolution data, preserve provenance through the resolution chain, and reject double counting at insertion time rather than only after the fact. Their order is deliberate. The discussion begins with the representational commitment on which the others depend and then moves outward to the rules that make that commitment workable in practice.

It is important to note that the relationship between the real assets and the availability of data in different aggregations, granularities and frequency were considered to design the elemental components of the graph allowing for consistency and extension, allowing multiple views of multiple crude and its products.

The axioms and corollaries presented are the foundation blocks for the data model where the network will be represented. We paid special attention so the data model follows the consistency and the principles of the normal forms.

4.1 Axiom 1: Asset-centric representation

The first design commitment is that **“every physical asset in the network is represented as a node with its own inventory and mass balance”**. Pipelines, vessels, terminals, refineries, gathering systems, and storage hubs are therefore treated as first-class nodes rather than as properties attached to the edges between locations. This generalization allows the network to be scalable while representing the actual reality. The main alternative considered during design was a location-centric representation, where nodes denote places such as basins, ports, or refining centres and the assets connecting them sit on the edges. Both approaches are common in the literature, but the choice matters more than it may appear at first.

In a location-centric model, a pipeline is usually treated as an edge with attributes such as length, capacity, transit time, and operating status. That works well in many settings. The difficulty in crude oil logistics is that the crude physically inside the pipeline at a given moment—the line-fill—is not a negligible detail. It becomes hidden state that must be reconstructed indirectly from transit time and recent flows. For a large trunk line, this can amount to several days of throughput and may be comparable to a regional refinery's weekly

crude runs. When throughput changes, when maintenance shuts the line, or when a storm disrupts operations, the line-fill changes as well. Those changes can show up in regional inventory statistics. If the pipeline is only an edge, that movement is obscured. If the pipeline is a node, it becomes visible and accountable.

The same point applies even more strongly to vessels. A VLCC carrying two million barrels from Rotterdam to Houston is, for the duration of that voyage, a floating storage asset. During periods of contango, when holding crude in transit can make economic sense, that role becomes even more obvious. A representation that hides the cargo inside an edge attribute misses a part of the system that is both physically real and commercially important.

The asset-centric commitment is therefore straightforward: each pipeline, vessel, terminal, and comparable operational asset is represented as a node with its own stock variable, its own mass balance, and, where relevant, its own observation series. Edges then represent what they physically are—directed movements from one asset to another. They carry volume, but they do not hold it.

Example. When the Capline pipeline reversed direction in 2022, switching from south-bound to north-bound service, the asset-centric representation handles the change naturally. The pipeline remains a node. Its line-fill remains its own stock variable. The two directed edges that connect it to the Patoka hub at one end and the St. James terminal at the other remain in the topology; one direction goes to zero flow, the other becomes active. The location-centric alternative would have required a structural rewrite of the edge: changing its direction, possibly renaming it, and reconstructing how its line-fill is attributed across the change. Asset-centric makes the reversal a data event, not a schema event.

The same point holds quantitatively in the live implementation. EIA's stock identity for PADD 5 includes commercial stocks, the Strategic Petroleum Reserve, and an in-transit allocation. The identity consistently overshoots the sum of named commercial hubs and refinery stocks by roughly six million barrels, and that residual is crude moving aboard Jones Act tankers from the Gulf Coast to West Coast refineries; EIA allocates the volume to PADD 5 as destination-based ending stocks. The framework gives the volume a home. PADD 5's inter-PADD pipeline corridor node from PADD 3 already exists with TS-bound flow variables, so the residual lands there as line-fill inventory, closing PADD 5's stock partition exactly. A location-centric model, where the corridor is an edge attribute rather than a node, would have nowhere to put those barrels.

The same asset-centric framing pays off when richer data sources are available. Genscape, now part of Wood Mackenzie, reports daily tank-by-tank inventory levels at the Cushing hub from infrared satellite imagery — roughly twenty individual storage tanks resolved rather than a single PADD-2 aggregate. OilX and Kpler track vessel movements globally through AIS feeds and shipping intelligence, resolving in-transit cargoes by ship, route, and grade rather than as a monthly net inflow. IIR Energy maintains a refinery operations database with per-unit turnaround schedules, capacity utilisation, and crude slate, at the facility and process-unit

resolution that EIA only publishes for the PADD 3 refining district at most. S&P Global Commodity Insights collects per-pipeline shipper nominations and contract-level pricing detail. In the asset-centric representation each of these data sources binds to the node it describes; in a location-centric representation, where pipelines and tanks are edge attributes, none of them would have a natural home.

4.2 Axiom 2: Stable topology via zero-flow edges

The second commitment is that the "**graph's adjacency matrix remains fixed over time"**. Every edge that could carry flow under the modelled topology is defined once and remains part of the graph at every timestep, whether it is currently active or not. An inactive edge therefore carries zero flow rather than disappearing from the structure.

The main alternative is a time-varying topology in which edges appear and disappear as physical or contractual relationships change. That can look appealing if the graph is interpreted literally as a record of which flows are active at a particular moment, and many dynamic graph methods in machine learning are built around that idea. This thesis rejects that approach for two reasons. First, most of the graph neural network architectures that would naturally consume this representation—such as GCN, GAT, GraphSAGE, DCRNN, STGCN, and TGN—work much more naturally with a stable adjacency structure. Second, keeping topology fixed creates a clean separation between structural change and operational change. Structural change is rare and deliberate, and it should trigger schema review. Operational change is frequent and should be reflected only in the values attached to existing edges.

The zero-flow convention should therefore be understood as a modelling discipline rather than a physical claim. A pipeline shut for maintenance still exists as a node, and the edges connected to it still exist as part of the topology; their values are simply zero during the outage. A reversible line is represented by two directed edges in opposite directions, either of which may be zero at a given time. The topology remains unchanged. Only the edge values move.

4.2.1 Axiom 2 examples

The Capline reversal mentioned above illustrates the rule. Both directed edges, north-bound and south-bound, exist throughout the modelled period. From 1944 (when Capline opened) through 2021 the south-bound flow was active and the north-bound flow was zero. From 2022 onwards the relationship flipped: the north-bound flow lit up, the south-bound went to zero. No structural rewrite of the graph was required, and no GNN architecture that depends on a stable adjacency matrix needed special handling.

Another useful example is the case of vessels. While each vessel is an asset allocated to a node, all the possible load and unload terminals are linked to the vessel by variables, but the loading and unloading is represented by the time series of the flows. If we use Markov

processes to try and predict the destination of the vessels while they are in transit, the flows would be the probability of the vessel arriving at the terminal at that date times the volume the vessel carries. We could not find other designs that were general enough to accommodate all the real assets an oil logistics network can have.

At the refinery level, this concept is very powerful. Defining the slates possible as a percentage of the oil consumption of the refinery allows the definition of product production. Together with the design of the flows that the product can take, we can have a full description of the logistics network and a powerful base to apply pricing and profitability constraints to predict flows.

The stable adjacency has a direct consequence for the downstream Linear Programing consumer of Chapter 8. Because every flow variable exists at every timestep, the LP's coefficient matrix is fixed across the modelled horizon; the only thing that changes between snapshots is the right-hand side (observed values) and the upper bounds (capacities, where they are time-versioned). Topology changes — Capline reversals, Bakken cross-state additions — are absorbed as data events: zero-flow becomes positive-flow in the active direction, or a previously latent edge carries a value once data lands on it, with no LP rebuild required.

4.3 Corollary A: Universal mass balance with balancing item

The next commitment is that every physical node satisfies the mass-balance equation:

$$\Delta S(i, g, t) = P(i, g, t) + F_{in}(i, g, t) - C(i, g, t) - F_{out}(i, g, t) + B(i, g, t)$$

where for node i, commodity g, and time t, ΔS denotes the change in stock, P is production, C is consumption, F_in is total inflow from connected nodes, F_out is total outflow to connected nodes, and B is the balancing item. The balancing item is a first-class node variable: a recorded value that absorbs systematic discrepancies between the observed stock change and the sum of the observed flows. It is retained as part of the node's data, not discarded as noise.

Mass balance is enforced at physical assets — refineries, pipelines, terminals, basins, gathering systems, storage hubs, and the boundary nodes that represent foreign supply and foreign export destinations. It is not enforced at abstract assets. Abstract assets (PADD views, basin views, refining-district views, the stock-decomposition aggregates of Section 5.4) are aggregation views over the physical layer rather than entities that move barrels of their own. Their P, C, I, O, B, and S variables are either roll-ups — defined as the sum of the physical constituents' corresponding variables under Axiom 3 — or independently TS-observed at the aggregate level when EIA publishes a series for the aggregate directly. When both forms are available, the comparison between the observed aggregate value and the sum over its constituents is a cross-check rather than a new constraint; the dual role of formula_inputs that this implies is set out in Section 4.8. Mass balance at observational aggregates is a

consequence of mass balance at physical assets plus the aggregation formulas, not a separate axiom.

The obvious alternatives are either to derive B only as a residual at query time or to treat any non-zero residual as a data-quality problem that should be removed before loading the data. In crude oil logistics, both approaches are unsatisfactory. Reporting frictions are present throughout the system, from custody-transfer timing to refinery process losses, vessel measurement differences, and stock-accounting mismatches. If the residual is hidden or discarded, those discrepancies do not disappear; they are simply pushed into other quantities downstream. The IEA's practice of publishing an explicit balancing item reflects exactly this point. The residual is not merely noise. It carries information about where the reporting system is failing to close cleanly.

In this framework, the balancing item does more than absorb a discrepancy. Its size and sign are themselves analytically useful. A persistently positive B at the USA aggregate level suggests that observed stock changes exceed what the reported flows would imply, which may point to under-reported exports, unreported imports, or timing mismatches in publication. A sudden spike in B around a major event—such as a hurricane, a pipeline outage, or a large SPR release—gives a sense of how much disruption the reporting system is failing to capture directly. Chapter 7 uses this idea explicitly by comparing observed B with closure-derived B to identify where the representation and the published data diverge most strongly.

Example. Refineries, considered individually, have small but non-zero B values reflecting process losses (the well-known refinery shrinkage of approximately 1 to 2 per cent by volume). Pipelines have small B values reflecting line loss and measurement noise. Production nodes have small B values reflecting reservoir condensate and flaring. At the USA aggregate level, B is typically a few tens of kbd in absolute value, rising to several hundred kbd around named disruption events. Each of these B values is a retained data point, available for inspection and for downstream consumers; none of them is silently absorbed into the surrounding flow variables.

4.4 Axiom 3: Formula-implies-relation

The fourth commitment is that “**edges, partition links, and node status are all derived from the variables collection**”. In other words, the variables table is the single source of truth, and every graph view is computed from it. An edge between two nodes exists if, and only if, a variable on one node references a variable on the other. There is no separate primary edges table that must be maintained in parallel.

The alternative, and in fact an earlier implementation choice, was to store edges as a separately declared structure. The problem with that design is that it creates two sources of truth that can drift apart. An edge may be declared without a corresponding formula, or a formula may reference another node without any declared edge making that relation visible in the graph. Both cases are recoverable, but only by carrying an unnecessary duplication of

structure and then reconciling it later. Formula-implies-relation removes that duplication. If the variables collection is internally consistent, the structural views derived from it are consistent as well.

The same principle extends to the resolution hierarchy (the same-type aggregation tree, see Section 4.7) and to the per-scenario node status (see Section 4.5). In each case the structural artefact is recoverable from the variable definitions and is therefore not stored a second time.

This principle has several far-reaching implications. Firstly, for abstract nodes, it gives constraints. A production basin is a collection of wells that might have specific data available about their production. Refineary district have collections of refineries. The data available per basin or refinery district mean that the sum of a group of nodes has to equal data that is available. In this specific case, the relationship between the variables gives a constraint.

The variable relationship will as well give us the flow of oil by allocating the inflows and outflow variables. If a basin feeds two pipelines, we know that the total production will move out on those two pipelines, but we do not know the value for each pipeline. Those pipelines might reach storage tanks or refineries, and we will know the sum of the flows or the specific flows, but the variable relationship will tell us how the nodes are related. The variables will give us the edges of the graph. This design allow for flexibility on the definition of the variables and their relationship with other variables.

4.4.1 Note on graph construction

During the construction fo the graph CHatGPT and Claude were used to comb the web to find out the details of the US crude oil network. The natural output was a json file with assets, locations, metadata, etc, and the relationship between the nodes. Using that as a raw material, we organized the metadata and defined the variables to populate the database. Nowadays the use of jsons instead of a relational database would be common, but the tools that relational databases natively give (referential integrity, primary and foreign keys, etc) are key to keep the framework consistent by design.

Once the variables are built, one can create views or materialized tables that will give all the information needed for the graph. Searches are much easier to perform, but to propagate new commoditiy types or to calculate the mass balance, building the graph in memory and making use of python libraries is much easier and quicker.

Example. Four views of the graph are derived from the single variables table: `v_flow_edges` lists every directed flow edge in the network, derived from the relational variables that carry a `related_node_id`; `v_aggregation_edges` lists every cross-node formula reference, regardless of variable type; `v_partition_tree` filters the aggregation edges to the same-type, same-commodity subset that defines the resolution hierarchy; and `v_node_status` (see Section 4.5) classifies each node under the active scenario based on the assignment rows

attached to its variables. None of these views requires its own primary table; all are refreshed whenever the variables collection or the scenario assignments change.

4.5 Axiom 4: Persistent asset graph and scenario state

The fifth commitment is to separate the model into two layers. The first is the persistent asset graph: the fixed superset topology containing every node and every feasible edge. The second is the scenario state, which is derived at load time from that asset graph together with the per-scenario `variableassignments` rows that specify whether each variable is bound to a time series or to a formula.

The alternative is a single-layer model in which changes in data resolution immediately rewrite the graph itself. That was also the direction taken in earlier implementation passes, and it proved difficult to sustain. Every improvement in data granularity forced structural edits, those edits in turn affected downstream consumers, and tracing the lineage of derived quantities across versions became unnecessarily hard. Separating the persistent graph from scenario state avoids that problem by placing the volatility where it belongs: in the assignments rather than in the topology.

In the two-layer model, data improvements mainly change the `variableassignments` table. The persistent asset graph stays in place (including the variables that dictate the relationships between the nodes) while variables are re-pointed from coarser to finer time series as better data becomes available. If the existing node set is no longer sufficient, the graph can still be extended, but that happens through an explicit migration recorded as a structural event. This keeps the distinction clear: the asset graph evolves slowly and deliberately, while the scenario state evolves with the data.

The status of each node under a given scenario is itself derived from the assignment rows attached to that node. A view `v_node_status` classifies each node as **authoritative** if at least one of its variables is time-series-bound, **derived** if no variable is time-series-bound but at least one resolves through a non-trivial formula, and **collapsed** if every variable on the node is either structurally zero or latent. The three-way classification is computed, not declared; if the variable definitions are consistent, the classification is consistent.

Example. When per-refinery operating stocks become available for the first time, the framework does not require an asset graph revision. The relevant refinery nodes already exist; what changes is that their inventory variables are re-pointed from PADD-level aggregate time series to facility-level time series in variable_assignments. The refineries' status under the scenario flips from inactive to authoritative automatically through v_node_status. No code outside the assignment table changes.

4.6 Corollary B: Physical layer and observational aggregation layer

The physical layer consists of real named assets such as fields, pipelines, vessels, terminals, and refineries. The observational layer consists of reporting aggregates such as PADDs, refining districts, basin roll-ups, and national totals. These aggregates do not carry physical flow edges of their own; they are materialised through same-type aggregation formulas over the physical layer.

The alternative would be to place administrative and physical entities in a single undifferentiated layer, so that something like PADD 2 would sit beside Cushing or the Bakken as though all three were the same kind of object. That is misleading. PADD 2 has no operator, no equipment, and no physical custody-transfer mechanics. It is a reporting boundary. Treating it as a physical node invites exactly the ambiguity the framework is trying to avoid: is a flow between PADDs a thing in itself, or is it an aggregation over underlying pipelines? In this framework, the answer is explicit. PADD-level flows are derived from the physical or aggregate nodes that carry them; PADD-level production and consumption are likewise derived from the relevant assets beneath them.

A subtler benefit of the separation is that observational aggregates can be added, modified, or removed without affecting the physical layer. EIA refining-district roll-ups (PADD 3A, 3B, 3C, …) exist because EIA publishes district-level series; they are aggregation views over refineries, not co-owned operating entities. If EIA changed its reporting structure tomorrow, the framework would add or retire aggregation nodes accordingly, without touching a single physical asset.

Example. The node `padd2_view` aggregates the Bakken-ND basin, Oklahoma, the padd2_other residual, the `padd2_canadian_imports_agg` import aggregate, the inter-PADD corridor aggregates with PADD-2 endpoints, and twelve named refineries. It has no flow edges of its own. Its outflow to PADD 3 is alias-derived from the time-series-bound outflow on the `inter_padd_2_to_3_agg` node, which is itself a physical aggregate over the named inter-PADD pipelines (Capline, Diamond, ExxonMobil Pegasus, and others).

4.7 Corollary C: Geography metadata distinct from resolution hierarchy

The next commitment is to distinguish geography metadata from the resolution hierarchy. Geography metadata describes where a node is located—latitude, longitude, county, state, PADD, country. The resolution hierarchy, by contrast, is a structural relation between nodes that represent the same physical system at different levels of granularity. The Permian basin, the Texas portion of the Permian, Midland County, and an individual lease form a hierarchy. Two refineries that happen to share the label PADD 3 do not.

The distinction matters because labels by themselves cannot prevent double counting. If observed data is assigned at more than one level of the same hierarchy for the same variable, the same physical quantity is effectively counted twice. Geography metadata is useful for filtering, grouping, and administrative rollups. It is not a substitute for structural hierarchy. Conflating the two—treating a shared geography label as if it implied a parent-child relation in

the data—is precisely the kind of mistake this principle is intended to prevent.**Example.** Two named refineries can share `padd = 'PADD3'` in the geography metadata without being part of the same physical asset (and they typically are not — Motiva Port Arthur and ExxonMobil Beaumont are independent operating entities that happen to share the PADD label). Conversely, two nodes can sit at `resolution_hierarchy.parent = 'permian'` (the basin) without being co-located in geography (Permian-TX is in Texas, Permian-NM is in New Mexico). The two metadata systems are independent.

The same physical assets can be aggregated along axes other than geography. Imagine a data source that reports Shell's U.S. crude production: it aggregates over the named physical production assets in which Shell holds an operating interest, regardless of which PADD or state they sit in. The framework absorbs this as a second observational aggregation layer over the same physical nodes, declared through formula_inputs that pick the Shell-operated constituents. Per-grade aggregation works the same way: a WTI-graded view over the producing basins, a sweet/sour split over the refineries. Multiple aggregation axes coexist as long as each axis is internally consistent (no double-counting within one axis); they share the physical layer they reference. The resolution hierarchy of Corollary C is not the only hierarchy — it is the geography-spine version of a general pattern.

It is worth marking explicitly what the resolution hierarchy is and is not. It is an aggregation relationship — parent variables defined as sums of child variables under formula_inputs — not a routing relationship. The flow path along which crude actually moves (basin → gathering → pipeline → terminal → refinery) is a separate structure carried by the directed flow edges (`v_flow_edges`). The two often share endpoints but they are different graphs over the same node set: the resolution hierarchy is a directed acyclic graph of aggregation; the flow path is a directed multigraph of physical movement. Conflating the two is a common source of confusion when reasoning about which variable's value comes from where.

4.8 Axiom 5: The observed/derived invariant

The observed/derived invariant is the single most important consistency guarantee in the framework. For each combination of node, variable type, commodity, and timestamp, exactly one of two states holds: the variable is **observed**, meaning it is bound to a time series and that value is authoritative at the relevant resolution; or it is **derived**, meaning it is computed from a formula over other variables. It cannot be both, and it cannot be neither.

The alternative is to allow any combination of bindings and then try to detect double counting afterward through validation checks. That approach has the usual weaknesses of post-hoc control. It discovers the inconsistency only after data has already propagated downstream, it requires a potentially expensive scan across the assignment set, and it produces warnings rather than structural impossibility. In practice, warnings are easy to ignore. Schema-level enforcement is stronger because it makes the inconsistent state unrepresentable. The relevant constraint is:

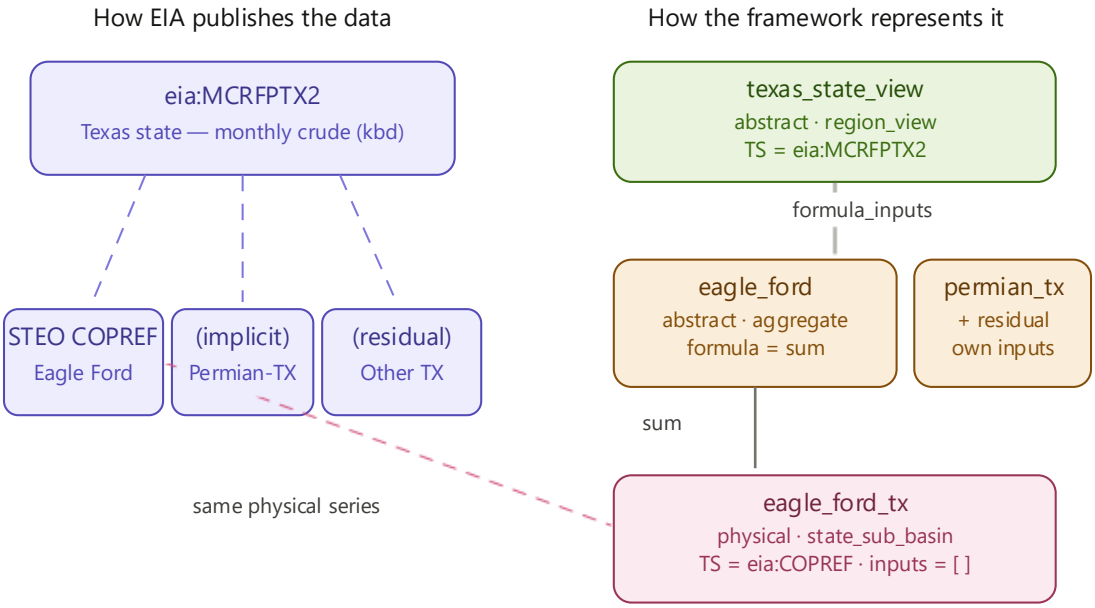
```
CHECK (num_nonnulls(timeseries_id, formula) = 1) – this type of constraint is
usual on relational databases.
```

It cannot be violated by an INSERT. Any attempt to write an assignment row that is both time-series-bound and formula-bound, or neither, is rejected by the database before the data can propagate anywhere.

A complementary audit at the time-series attribution level enforces single-attribution on the other side: each EIA time series is bound to exactly one variable under a given scenario. Together, the two invariants give the framework its central consistency property: every value in the resolved table is traceable to exactly one source, and no source contributes to more than one variable.

What the CHECK constraint does not forbid is declaring an aggregation relationship on the same variable. The schema carries a separate formula_inputs column listing the constituents whose roll-up the variable's value can be compared against. When the variable is observed (TS-bound), formula_inputs is a constraint set: the view v_aggregation_consistency computes |TS value − Σ constituents| at every (variable, date) and flags the row as ok or partial_coverage. When the variable is derived (formula-bound), the same column is the operand set that the formula resolves over. The CHECK constraint guarantees one source of value; formula_inputs declares a cross-check relationship that is independent of that source. PADD-level production is the canonical case: each padd_view.production is bound to its EIA MCRFP series and also lists the contributing basins and state-conventional variables in formula_inputs. **The time series gives the value; the constituents drive the cross-check; the two typically agree to within rounding tolerance.**

A concrete example helps. EIA publishes the Eagle Ford basin through two related series: MCRFPTX2 (the Texas state total, which aggregates production from Eagle Ford, the Texas portion of the Permian, and other Texas conventional wells) and STEO's COPREF (an Eagle-Ford-specific forecast at basin level). The framework binds the data at the level where it is published. The physical node eagle_ford_tx (single-state sub-basin) is TS-bound to COPREF and carries the basin's value directly. The abstract basin aggregate eagle_ford is formula-bound: formula = 'sum', formula_inputs = [eagle_ford_tx.production]; because the basin sits entirely within Texas, the sum collapses to a single constituent and the abstract aggregate inherits the physical value. The state-level abstract texas_state_view is itself TS-bound to MCRFPTX2 and lists eagle_ford_tx, permian_tx, and the Texas residual node in its own formula_inputs. The CHECK constraint is satisfied on every assignment row, and the v_aggregation_consistency view reports the cross-check |MCRFPTX2 − Σ children| at the Texas level and |eagle_ford − eagle_ford_tx| at the basin level. Both close to within rounding tolerance at every observation date.



Eagle Ford lies entirely within Texas. The basin-level STEO series (COPREF) and the Texas state series (MCRFPTX2) cover the same physical production at different aggregations.

Both texas_state_view and eagle_ford reference the same physical eagle_ford_tx through formula_inputs. The cross-check view v_aggregation_consistency reports the residual between MCRFPTX2 and the sum of its children, and between eagle_ford and eagle_ford_tx.

Figure 3 — Eagle Ford crude production. Left: EIA publishes the basin both through the Texas state series MCRFPTX2 and through the STEO basin-level COPREF. Right: the framework TS-binds the physical node eagle_ford_tx to COPREF; the abstract basin aggregate eagle_ford is formula-derived as a sum over its single physical constituent; the abstract state aggregate texas_state_view is TS-bound to MCRFPTX2 and references eagle_ford_tx, permian_tx, and the Texas residual node in its formula_inputs. The v_aggregation_consistency view cross-checks both reconciliations.

A note on what 'one assignment per variable' actually means. A node typically carries several variables at once: a production variable, a consumption variable, inflow and outflow variables (one per neighbouring node), an inventory variable, and a balancing item. The CHECK constraint num_nonnulls(timeseries_id, formula) = 1 applies to each (variable, scenario) row individually, not to the node as a whole. A single refinery node therefore has as many independent (time series or formula) decisions as it has variables — and in a given scenario, each of those decisions is recorded in its own variable_assignments row. The framework treats this as the natural mode of operation: a node is a collection of variables tied to a common physical asset, and each variable carries its own data attribution independently of the others.

The framework does allow auxiliary observed series on the same node — for example, EIA publishes both `MCRSFP` (PADD-level stocks excluding SPR) and `MCRSSP` (SPR-level stocks) —

but these are marked as **observed auxiliary** rather than **observed authoritative**, and they do not participate in mass balance at their own level. They serve as cross-checks against the authoritative value.

Example. When EIA began publishing the decomposition of PADD-level stocks into Strategic Petroleum Reserve and commercial portions (MCRSFP plus MCRSSP), the natural temptation was to bind each component at a separate sub-node level. But the SPR is already a structural node in the asset graph, with its own four physical sites and its own inventory variable, and MCRSFP at PADD level would have double-counted against the existing per-site SPR values. The framework forced the choice: MCRSFP is registered as an observed auxiliary series on the PADD view, not as an authoritative binding on a separate sub-node. The pre-existing SPR sub-nodes remain the authoritative source for their own inventory.

4.9 Corollary D: Latent allocation at junctions

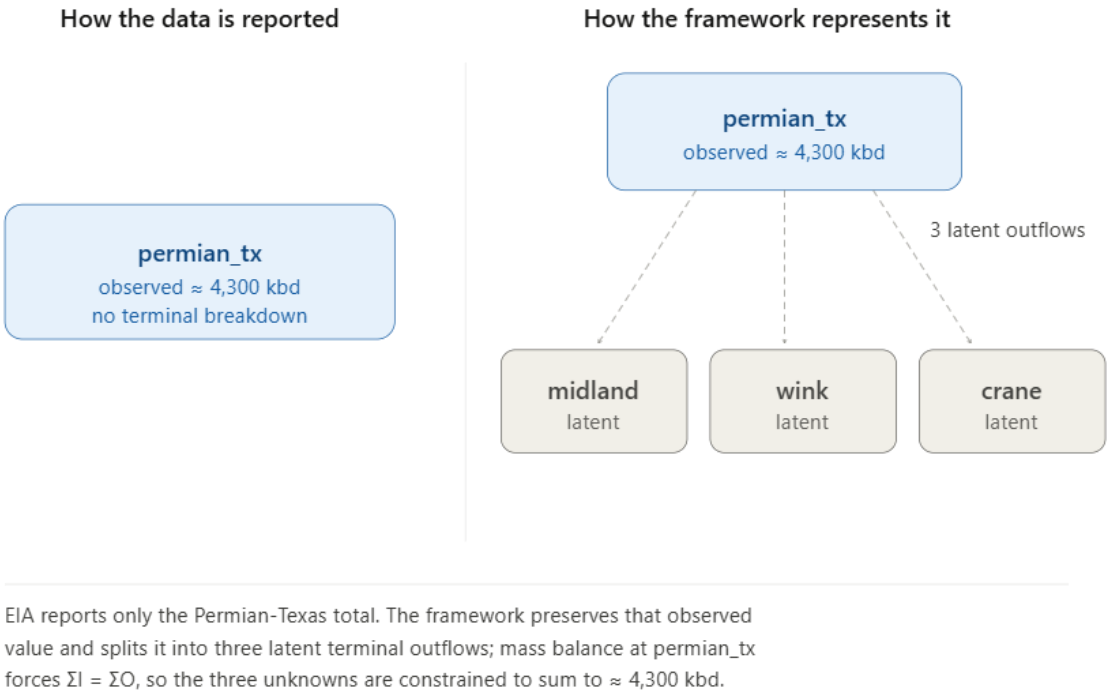
The next commitment concerns flows that are physically real but not directly observed. In many junctions, crude splits or merges across routes in ways that data does not measure at the per-edge level. The Permian is a clear example: total production is observed, receipts at major downstream destinations are observed, but the exact split across the pipelines connecting them is not.

The framework treats those unobserved per-edge flows as **latent**. They are declared variables with a clear structural role, but without an assigned value. The mass-balance equation at the junction still applies, so the latent variables are not unconstrained; they must collectively satisfy the aggregate totals implied by the rest of the system. The obvious alternative would be to fill in those missing values with heuristics, such as allocating flows in proportion to capacity. That would make the data look complete, but it would also blur the distinction between measured information and modelling guesswork. This thesis rejects that shortcut because downstream consumers need to know when a number is observed, when it is derived algebraically, and when it is genuinely unknown.

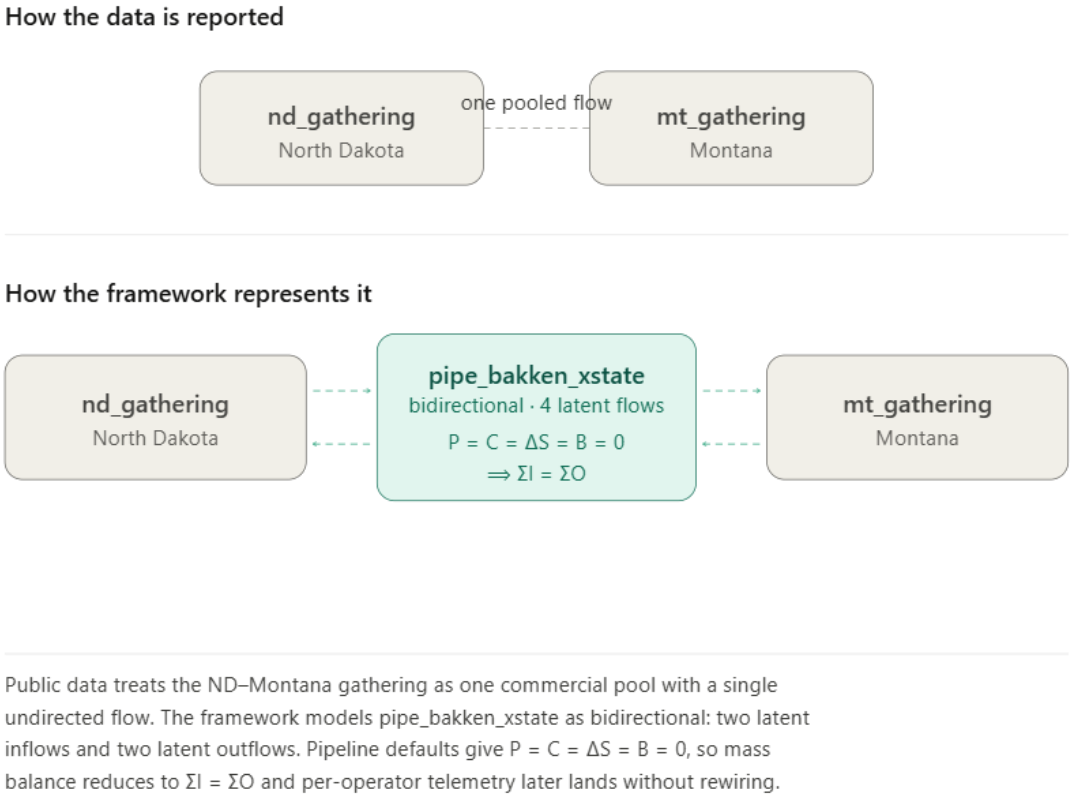
Latent variables are preserved explicitly in the resolved table. They appear with `value = NULL` and `source = 'latent'`, which distinguishes them from `unresolved` rows that would indicate a resolver failure. That difference matters downstream. A latent variable is unknown by design and can become a decision variable in an optimisation model. An unresolved value signals that something has gone wrong and should not be silently treated as part of the modelled uncertainty.

Pass-through node types (gathering networks, pipelines, import and export terminals, foreign aggregates) have $P = C = \Delta S = B = 0$ by physical construction, encoded in `node_type_default_formulas`. This makes the mass balance at every junction reduce to $\Sigma F_{in} = \Sigma F_{out}$, which is the natural form for the LP to consume.

Example. The node `permian_tx` has an observed production of approximately 4,300 kbd in a recent month. It connects to three origin terminals (`midland_terminal`, `wink_terminal`, `crane_terminal`) whose individual receipts from Permian are not publicly reported. The three outflow variables from `permian_tx` are declared latent. The constraint that they sum to the observed production is enforced by the mass balance at `permian_tx`. The gathering nodes for the Bakken (one node, seven downstream PADD-2 destinations, individual splits unobserved) and the Cushing hub (three operator sub-terminals, sub-decomposition observed by EIA but not at all desired granularities) follow the same pattern at different scales.



A second example concerns the Bakken cross-state midstream. Hess, Crestwood and Energy Transfer operate the North Dakota and Montana gathering systems as a single commercial pool. Crude moves between the two states in either direction depending on takeaway capacity, and public data does not break the flow down by direction. The framework models the connection as a bidirectional pipeline node, pipe_bakken_xstate. The node carries four latent flow variables: two inflows and two outflows, one per direction at each gathering. Pipeline-type defaults give the connector $P = C = \Delta S = B = 0$, so the mass-balance constraint reduces to $\Sigma I = \Sigma O$. The node is declared as a constituent of the inter-PADD aggregate outflows on both sides, so future per-operator gathering telemetry will land on these variables without any structural change.



The same mechanism extends naturally to per-grade tracking. Once the commodity dimension on variables is populated, the unobserved per-grade splits at each junction are latent in exactly the same sense as the unobserved per-route splits today. Mass balance per grade at each tank or terminal then constrains the unknowns; if a Cushing tank receives observed inflows of WTI from Permian and Bakken-light from Bakken, and ships observed outflows to PADD-3 refiners, the residual grade mix in the tank evolves as a consequence of the same latent-allocation pattern rather than as a separate model. Inferring 'what grade sits in this tank today' becomes a downstream computation, not a new schema.

4.10 Axiom 6: Bidirectional flows as separate directed edges

The final axiom is that each direction of a reversible or bidirectional connection is represented as its own directed edge, with its own flow variable. The zero-flow convention from Axiom 2 then handles whichever direction is inactive at a given moment.

The obvious alternative is a single undirected edge with a signed flow, positive in one direction and negative in the other. That is more compact, and some optimisation models use it. The reason it is rejected here is operational rather than mathematical. A south-bound movement on Capline and a north-bound movement on the same line are not just opposite signs of one quantity. They correspond to different commercial uses of the asset, different congestion patterns, different tariff structures, and often different downstream consequences. Treating them as separate directed edges keeps those distinctions visible.

The directed-edge convention also interacts cleanly with the zero-flow convention. At any moment, at most one direction on a reversible pipeline is active. Both directed edges remain in the topology; the active one carries the current flow; the inactive one carries zero. When the pipeline reverses, the values swap. The structure is unchanged.

A physical question sometimes asked here is whether a pipeline can carry flow in both directions at the same time. For a single-grade, single-product pipeline the answer is operationally no — a pipeline is in service one way or the other at any given moment. The framework's two-directed-edge convention allows for the data to report flow on both edges even so, because batching pipelines (or operationally bidirectional terminals like SPR sites during a release-and-refill cycle) can show net flow that nets to a single direction over a month while having movement in both directions within that month. The convention is permissive: the LP and the resolver both treat each direction's flow as an independent non-negative variable, and observed nets in either direction land where they land.

A practical clarification: a variable's name in the database is a stable identifier (for example, outflow__crude__pipe_seaway__houston_hub), not a description. The description of what the variable measures lives in the asset metadata, the variable type, the related node, and the commodity — every field that is actually queried. The name is an audit anchor that stays the same across renames of human-readable labels; it should be read as an identifier rather than as documentation.

Example. The Seaway pipeline carries two flow variables: `outflow__crude__pipe_seaway__houston_hub` (south-bound, the dominant direction) and `outflow__crude__pipe_seaway__cushing_hub` (north-bound, briefly active under specific market conditions). The same applies to Capline (reversed in 2022, both directions now in the topology), the LOOP offshore terminal, and the SPR sites (which can receive crude for storage or deliver crude in response to a release). The recently-added cross-state Bakken connector (`pipe_bakken_xstate`) follows the same pattern with four flow variables (two inflows, two outflows, all currently latent).

4.11 Corollary D-bis: Last-observation-carried-forward

Axiom 5 says that each variable is either observed or derived, but it does not say what an observed value means between publication dates. Since EIA publishes monthly, the framework needs a temporal convention. An EIA monthly value is either the average daily rate over the calendar month (for flow quantities) or the end-of-period stock (for inventory quantities), and between publications the value applies to every subsequent date until a new observation arrives. There is no interpolation; the value is a step function.

The framework adopts last-observation-carried-forward, abbreviated LOCF. At each evaluation date the resolver looks up the most recent time-series value at or before that date and uses it as the resolved value. Dates earlier than the first observation receive no row. Section 6.10 covers the implementation mechanics. The audit trail records LOCF in the

formula_used column on scenario_resolved_values, so a downstream consumer can tell freshly published values from carried-over ones.

LOCF can surface honest inconsistencies. Two series with different temporal horizons sometimes feed the same residual: a state-level series that ends earlier than the basin-level forecast that subtracts from it. The residual goes negative for the months where one feed is moving and the other is flat. The framework does not silently smooth this away. The v_resolution_anomalies view flags long LOCF runs and negative derived values as low-severity anomalies for review, leaving the analyst to decide whether the underlying source publications need to be reconciled.

4.12 Corollary E: Node status as a rendering projection

Binding state is a property of a variable, not of a node. Each variable in the active scenario is in exactly one of four states: time-series-bound, formula-derived to a non-zero value, structurally zero, or latent(). This is the primitive the framework reasons about, and it is genuinely heterogeneous within a single node — a refinery may have its production variable time-series-bound while its storage variable is latent() and its balancing item formula-derived. Every consumer that performs work reads variables at this granularity. The LP consumer decides junction-by-junction which flows are fixed and which carry latents from the per-variable assignments, not from any node-level summary; the audit computes partition gaps from variable_assignments directly; mass balance closes per node but over its variables. None of these needs the node itself classified.

Rendering is the exception, and the only reason a node-level notion of status exists at all. An explorer paints a node as a single glyph, so it needs one label per node where the underlying data offers several per-variable states. The framework therefore defines a reduction over a node's variables: a node is authoritative if at least one of its variables is time-series-bound; derived if none is time-series-bound but at least one resolves to a non-zero value through a formula; and inactive if every variable is structurally zero or latent(). The schema column carries the name collapsed for historical reasons, but inactive more accurately describes the state — a node structurally present that contributes no value in this scenario. The authoritative/derived/inactive triple is not a domain concept; it is a colour-picking convenience whose sole consumer is the explorer.

The view v_node_status materialises this reduction by reading variable_assignments and emitting one row per (scenario, node). An earlier design populated a nodes.starter_status column by hand at load time. The column survives in the schema for backward compatibility, but it is vestigial: nothing reads it, and the view is the single authority on status. Under Axiom 3 a node's status is derivable rather than primary data, and a hand-populated column is a second source of truth waiting to drift out of step with the assignments that should determine it; retaining it unread, rather than relying on it, is the deliberate resolution of that tension. The same reasoning routes every other derivable node attribute through a view: inflows and

outflows from the relational variables (v_flow_edges), partition role from formula_inputs (v_partition_tree), per-variable-type aggregates at a date from resolved values (v_node_pcisob). The architectural commitment is that any derivable property is computed once, in one place, over the variables that are its source of truth, and consumed identically wherever it is needed — with the understanding that for status, "wherever" means the renderers and nothing downstream of them.

4.13 Summary

Taken together, the axioms and corollaries define both the scope and the limits of the framework. It is asset-centric, explicit about provenance, designed for multi-resolution data, and careful not to hide uncertainty behind apparently complete numbers. At the same time, it is not a forecasting model, not a heuristic flow allocator, and not an attempt to produce a single cleaned estimate of the system. The six axioms are the irreducible design commitments. The corollaries follow from those commitments once the variables collection is populated correctly. Two earlier ideas—a canonical sum rule and a formal latent-versus-unresolved distinction—proved more natural in the resolver chapter and the dispatch-rule reference, so they are discussed there rather than treated as architectural principles here.

Each axiom and corollary corresponds in implementation to one or more layered views: v_flow_edges and v_aggregation_edges for Axioms 2 and 3, v_partition_tree for Corollaries A and B, v_node_status for Corollary E, v_aggregation_consistency and v_node_balance_check for the cross-checks introduced in Sections 4.3 and 4.8, and v_resolution_anomalies for the temporal-convention anomalies of Corollary D-bis. The complete list, with the SQL definitions and the assertions each view supports, is in Annex C alongside the resolver's dispatch rules.

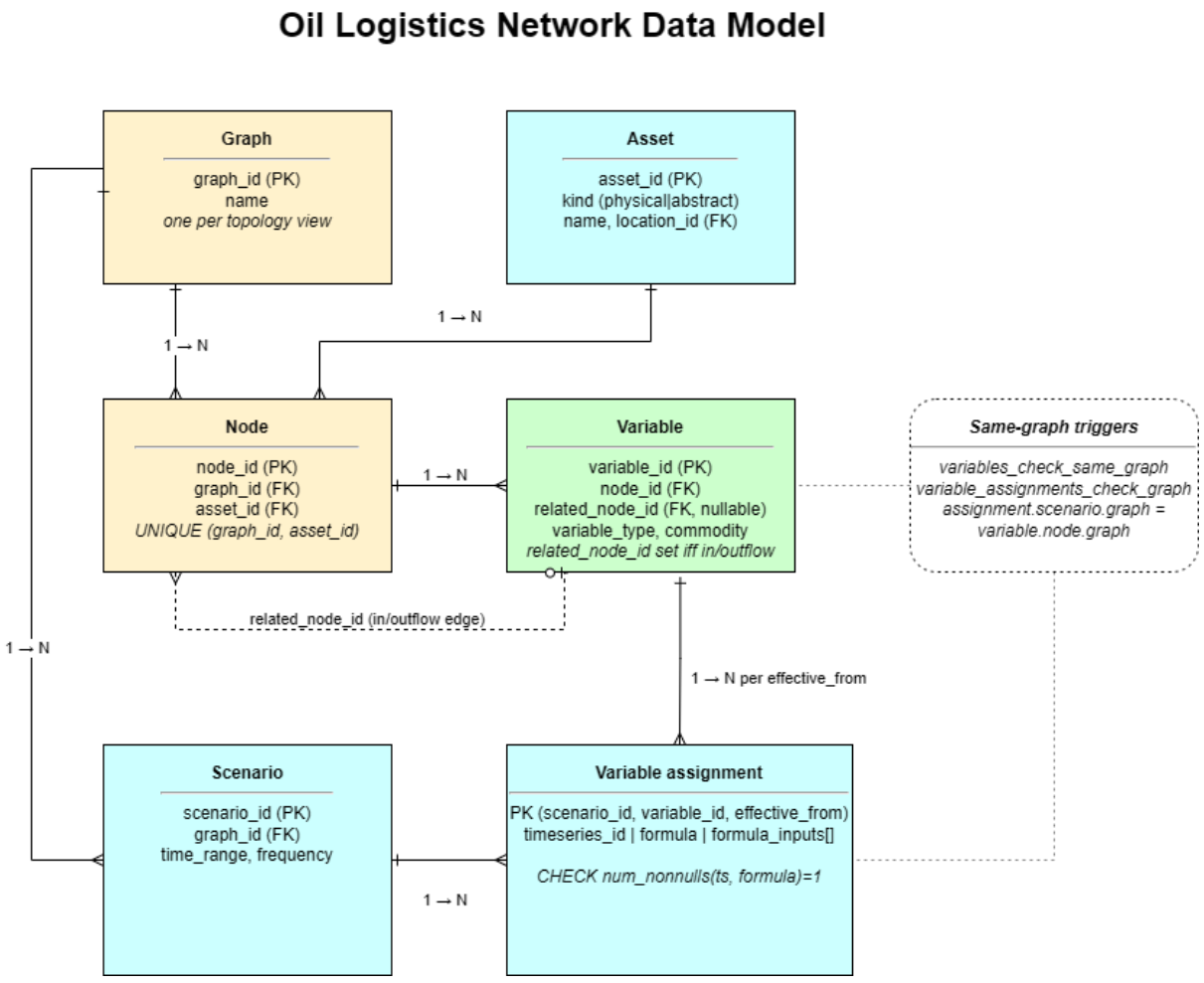


Figure 4. The oil_network data model as enforced by the schema. The structural layer (Graph, Asset, Node, Variable) is persistent and scenario-independent; the scenario layer (Scenario, Variable assignment) is volatile. A node is one asset on one graph (UNIQUE graph_id, asset_id); a variable may carry a second node reference (related_node_id) to form an inflow or outflow edge; an assignment binds a variable for a scenario under the CHECK that exactly one of timeseries_id or formula is set. The same-graph triggers enforce that an assignment's scenario and its variable's node resolve to the same graph.

5. Schema and Graph Construction

The axioms and corollaries set out in Chapter 4 are commitments at the architectural level. This chapter describes the schema and construction process that give effect to those commitments: the two-layer model, the taxonomy of nodes, the structure of variables and edges, the partition tree that drives consistency validation, and the starter graph that instantiates all of this for the U.S. crude oil network from 2015 onwards.

5.1 The two-layer model

The framework separates a persistent layer from a scenario layer. The persistent asset graph is the fixed, superset physical topology — every node, every potential edge, defined once and not altered for data reasons. The scenario state is derived at load time from the persistent asset graph plus the per-scenario rows in `variable_assignments`. Downstream consumers query the scenario state, not the persistent graph directly.

The two-layer model also admits multiple persistent layers in principle. The current implementation uses one graph_id (starter_us_crude) tagging all 251 nodes, so the assets-to-nodes correspondence is one-to-one. The schema (assets, nodes, graph_id on nodes) is designed so that a second graph — for example, a per-grade view, a per-operator view, or a North American extension that adds Canadian and Mexican assets — can be loaded as a parallel set of node rows over the same assets, without re-declaring the asset registry. Each graph carries its own variables, assignments, and scenarios. The framework's consistency claims (Chapter 7) apply per (graph, scenario) pair.

The separation is the practical expression of Axiom 4. Data volatility lives in the assignment table; structural volatility, when it occurs, is captured by an explicit migration to the persistent graph. The two kinds of change are reviewed differently, deployed differently, and rolled back differently. A new EIA series at finer granularity is an assignment change. The construction of a new export terminal is a migration. The framework refuses to conflate them.

The two-layer split is also a substantive rejection of a single-layer alternative in which the asset graph and the data are interleaved. A single-layer graph is, in effect, itself a data source: every new ingest at a different granularity (commercial tank-level imagery, per-vessel cargo data, finer state breakdowns) forces a topological change to keep the data where it lands. The two-layer split treats the asset graph as a fixed superset and lets every new data source bind into it through the assignment table without disturbing the topology. The set of assignments under a scenario is what an earlier framing of this work called a coverage contract; in the current implementation no separate object is needed, the principle was preserved, the structure was simplified.

5.2 The starter asset graph

The starter graph targets U.S. crude oil from January 2015 onwards, the period over which the relevant EIA monthly series are consistently available. It contains 251 assets in total: 217 physical and 34 abstract aggregates, of which 3 are boundary nodes. The assets connect via 433 directed flow edges, and they carry 1,870 variables under the active scenario `starter_us_crude_2015_2025`.

It is worth flagging that the asset graph was constructed independently of the EIA time-series catalogue. Named physical assets — refineries, pipelines, terminals, gathering systems, storage hubs — were enumerated from public domain knowledge (corporate filings, FERC and PHMSA registers, trade-press infrastructure surveys, OGJ refinery surveys, Global Energy Monitor's pipeline trackers, Wood Mackenzie's IIR for refinery operating data) rather than from what EIA happened to publish. The result is an asset graph that covers approximately 100% of U.S. modelled refining capacity (17,484 kbd against the real system's ~17,500 kbd) and 113% of production capacity at peak — slightly more than the real system because some assets are at headroom rather than steady-state utilisation. EIA time series then bind into this superset where the data is available; missing series leave the corresponding variables latent or formula-derived rather than forcing the asset graph to shrink.

Every node is tagged with a kind (physical or abstract), a node_class (production, infrastructure, observational), and a node_subtype that refines the class (refinery, gathering, pipeline, region_view, and so on). Geographic metadata — latitude, longitude, state, county, PADD — is stored in a separate oil_network.locations table to keep labelling distinct from structure, in line with Corollary C.

The graph is built from a seed JSON file (`asset_graph.json`) loaded once and then evolved by incremental migration scripts. Each migration corresponds to a deliberate structural change (a new aggregate node, a node split, a route correction) and is recorded as a versioned event. The final state of the starter graph reflects twelve such migration passes.

5.3 Physical nodes

Physical nodes represent real, named assets on the ground. They carry geographic coordinates, capacity, configuration, and flow edges. Adding a physical node usually corresponds to a real operational change: a new refinery coming online, a new pipeline being commissioned, a new export terminal opening.

The starter graph contains the following physical inventory, grouped by subtype:

Subtype	Count	Role and representative examples	
L3c	v_partition_sums	L2c + scenario_resolved_values	Materialised observed-vs-children sum per (parent, variable_type,

Subtype	Count	Role and representative examples	
			commodity, date)
L3d	v_partition_intra_flows	L2b + scenario_resolved_values	Materialised internal flow accounting for partition closure across the partition tree
state_sub_basin	5	Named production regions: bakken_nd, bakken_mt, permian_tx, permian_nm, eagle_ford_tx	
state_conventional	7	Mature state-level production: oklahoma_conventional, california_conventional, wyoming	
state_residual	5	One per PADD: padd1_other through padd5_other, covering un-named tail production	
offshore_region	1	gulf_of_america (encompassing all federal offshore production in the Gulf)	
gathering	6	Basin-level midstream pooling: ans, bakken_nd, bakken_mt, eagle_ford, permian_nm, permian_tx	
origin_terminal	6	First-stage receipt points: midland, wink, crane, johnsons_corner, three_rivers, valdez	
storage_terminal	10	Cushing hub (with three operator sub-terminals), Patoka hub, Houston, and others	
spr_site	4	The four Strategic Petroleum Reserve sites: bayou_choctaw, big_hill, bryan_mound, west_hackberry	
refinery	115	19 named refineries, 5 PADD residuals, and sub-aggregates totalling 24 active facilities	
import_terminal	8	clearbrook plus four PADD-level import aggregates, two Canadian import	

Subtype	Count	Role and representative examples
		aggregates, padd2_xstate
export_terminal	12	Ingleside, Houston-aggregate plus three sub-terminals, Nederland, LOOP, and five PADD-level export aggregates
pipeline	37	20 named U.S.-internal lines, 4 cross-border lines, 12 inter-PADD aggregates, the new Bakken cross-state connector
foreign_export_destination	1	Boundary sink: where U.S. exports physically leave the modelled system
foreign_production_aggregate	2	Boundary source: canadian_oil_sands and foreign_supply (non-Canadian)

The foreign supply boundary is visible per PADD indirectly. The framework does not model non-U.S. production at country resolution beyond foreign_supply and canadian_oil_sands as boundary aggregates, but EIA's import series report imports per PADD by origin country; these series bind to the corresponding PADD's import aggregate node (e.g. padd1_imports_agg, padd3_imports_agg), so the per-PADD foreign-supply contribution is recoverable through the partition tree even though the boundary node itself is non-partitioned.

A few of these categories warrant comment. The **state residuals** (padd1_other through padd5_other) exist because EIA publishes PADD-level production totals that exceed the sum of the named basins and conventional fields within each PADD. The difference must be attributed to some node; the residual is that node. Its production is not enumerated but computed by closure — the PADD-level published total minus the sum of the named basins and conventional fields within that PADD — so that when a new physical sub-basin becomes individually named, it is split off as its own node and the residual shrinks by exactly that amount. The construction keeps the substitution traceable: at every step the residual is whatever the published total does not yet attribute to a named source.

The **inter-PADD and import/export aggregates** are technically physical nodes — they have `kind = 'physical'`, an asset row, and concrete flow edges — but they aggregate over groups of named pipelines and terminals rather than denoting a single facility. They exist to give the partition tree (Section 5.6) a same-type child to point at, and they are the natural home for future per-grade or per-operator decomposition when that data becomes available.

The **Bakken cross-state connector** (`pipe_bakken_xstate`, added in the eleventh migration pass) is a bidirectional pipeline-type node connecting `bakken_nd_gathering` and

`bakken_mt_gathering`. It models the real midstream pooling between the North Dakota and Montana halves of the Bakken basin, operated by a consortium of midstream companies. None of its four flow variables (two inflows, two outflows) is observed in public data; all four are latent in the starter scenario, with the constraint ΣF_in = ΣF_out at the connector following from the pipeline node-type defaults.

5.4 Abstract nodes

Abstract nodes do not represent physical assets. They are aggregation views over physical nodes, providing the substrate for partition rollups, data attribution at coarser resolutions than the physical layer, and the consistency claims developed in Chapter 7. They have `kind = 'abstract'` and no flow edges of their own; their relationships to physical nodes are through `formula_inputs` references on their variables.

The state_residual nodes (padd1_other through padd5_other) deserve a separate note. They represent the production volume that EIA reports at PADD level but cannot allocate to any named basin or state in the current model — the tail of small, unmodelled producers that EIA aggregates implicitly into the PADD total. They are physical assets rather than abstract aggregates because they carry their own production variable and they participate in the partition (the PADD-view production sums the named contributors plus the residual). When a new named source becomes available — a new shale basin, a previously unmodelled state, an additional EIA disaggregation series — that source is split off from the residual as a new physical node and the residual shrinks accordingly. The pattern is the same one used at the boundary nodes: a structural placeholder for an aggregate the framework cannot currently decompose, with a clear path to refinement when the data does decompose it.

Subtype	Count	Role
region_view	6	usa_view, padd1_view through padd5_view: the geographic partition spine
refining_district_view	10	EIA refining-district roll-ups (3A, 3B, 3C, and so on across the PADDs)
state_view	2	texas_state_view, montana_state_view: roll-ups for sub-PADD cross-checks
observational_aggregate	4	spr_total and basin aggregates (permian, bakken, eagle_ford) when used as constraint roll-ups
usa_subtotal_view	1	usa_lower48_excl_gom_view: STEO-style subtotal excluding the Gulf of America

The reason region_view nodes are abstract rather than physical is that a PADD is an administrative grouping, not an operating asset. No piece of equipment can be pointed at and called "PADD 2". EIA reports aggregated data at this scale, and the framework needs a node that can carry that aggregated data, but it is structurally distinct from a refinery or a pipeline.

Treating PADDs as physical would invite the confusion that Corollary B is designed to prevent: it would be unclear whether the PADD's outflow to its neighbour is a thing in its own right or merely the sum of the underlying pipelines.

The refining-district nodes are abstract for the same reason: refining districts group refineries for reporting purposes; they are not co-owned, do not share custody transfer mechanics, and do not have a single physical address. They exist to host the district-level consumption series that EIA publishes and to enable the partition identity that PADD-level consumption equals the sum of its constituent district consumptions.

A further family of abstract aggregates is the per-PADD stock decomposition. EIA publishes each PADD's commercial stocks in two buckets: tank farms and pipelines (series MCRSFP{N}1) and refinery stocks (series MCRRSP{N}1). PADD 3 also carries the Strategic Petroleum Reserve. The framework materialises the split as two observational aggregate nodes per PADD, padd{N}_tank_farms_pipelines and padd{N}_refinery_stocks, each TS-bound to the corresponding EIA series. Their formula_inputs declare the physical constituents: named storage hubs and gathering nodes feed the tank-farm side; refineries feed the refinery-stocks side. The inventory variable on padd{N}_view then lists the two decomposition aggregates as its own formula_inputs (plus spr_total for PADD 3), so the EIA stock identity MCRSTP{N} = MCRSFP{N} + MCRRSP{N} (+SPR) is materialised directly in the partition tree.

5.5 Boundary nodes

Four nodes sit on the boundary of the modelled system. They represent the points at which crude oil enters or leaves the U.S. system, and they are kept structurally distinct from the partition spine.

Node	Subtype	Role
foreign_supply	foreign_production_aggregate	All non-Canadian foreign crude flowing into U.S. import terminals. Boundary source; foreign-side mass balance is not modelled
canadian_oil_sands	foreign_production_aggregate	All Canadian crude entering the U.S. via Mainline, Keystone, Express+Platte, and TMX (when active). Boundary source
foreign_export_destination	foreign_export_destination	Boundary sink. All U.S. crude exports flow into this node; the framework does not model where they go from there
spr_total	observational_aggregate	Sum of the four SPR sites. Used for constraint cross-checks, not as a partition member

The boundary nodes carry inflow or outflow edges to the corresponding U.S. nodes (foreign_supply has outflow edges to PADD-level import aggregates; foreign_export_destination has inflow edges from each named export terminal), but they do not themselves participate in the partition closure tests. They are sources and sinks rather than partition members.

5.6 Variables

Variables are the unit of data attribution in the framework. Every node carries one variable per (variable type, commodity, related node) tuple. The seven variable types are:

Type	Symbol	Description	Relational
production	P	Crude produced at this node	no
consumption	C	Crude consumed at this node (refinery throughput)	no
inventory	S	Crude stocks held at this node	no
balancing_item	B	Reporting residual (Corollary A)	no
inflow	F_in	Crude flowing in from a specific related node	yes
outflow	F_out	Crude flowing out to a specific related node	yes
phi	φ	Reference variable, used for cross-node alias chains	yes

A note on the phi variable. The phi type is a cross-node alias reference: when one node's variable is defined as an alias of another's, the alias chain is recorded as a phi variable with related_node_id pointing at the source. Phi exists to keep alias chains representable in the variable graph (so the dependency DAG that the resolver topologically sorts is well-defined) without requiring the alias to look like a flow edge. It carries no flow, no stock, and no balance role; it is purely a structural pointer.

A relational variable carries a `related_node_id` identifying the counterpart in the directed flow. An inflow on node A with related node B means "crude flowing from B into A"; the paired outflow on node B with related node A means "crude flowing from B out to A". The two are physically the same flow, modelled as two variables on different nodes. The reverse-mirror dispatch rule in the resolver (Section 6.4) propagates an observed value on one direction to the unobserved counterpart.

Each variable carries either a timeseries_id (time-series-bound, observed) or a formula (derived), never both. The schema-level CHECK constraint num_nonnulls(timeseries_id, formula) = 1 is the enforcement mechanism for Axiom 5.

The variables collection is the authoritative data model. Every other view of the graph — the edge list, the partition tree, the resolution hierarchy, the per-scenario node status — is derived from it. There is no second source of truth to keep in sync, no redundant declaration that can drift from the variable definitions. This is Axiom 3 made operational.

5.7 The partition tree

The partition tree is the same-type aggregation hierarchy that drives the consistency claims in Chapter 7. A node M is a partition child of node N if some variable on N has `formula_inputs` that reference a same-type, same-commodity, same-related-node variable on M. The same-type qualifier is essential: aggregation links (such as `padd_view.P = Σ basin.P`) connect variables of the same type, whereas cross-edge alias links (such as `padd_view.F_in = alias(other_padd.F_out)`) connect variables of different types. Filtering on type cleanly distinguishes the two and keeps the partition tree restricted to the same-type aggregation that the closure test depends on.

It is worth marking the relationship between the partition tree and the flow topology, because the two look superficially similar but encode different information. The flow topology (the directed edges in v_flow_edges) connects physical assets that actually move barrels between one another: pipelines from basins to refineries, terminals receiving from pipelines, vessels carrying between hubs. These edges are not geography-based. Cushing connects to Houston because the Seaway pipeline physically moves crude between them, not because the two hubs sit in the same state.

The partition tree is different. Aggregates such as padd2_view exist because that is the level at which EIA publishes much of its data, and EIA's aggregation rule is overwhelmingly geographic — the five PADDs were drawn in 1942 as wartime logistics regions and have remained the standard ever since. The partition tree's children are individual physical assets the framework already knows, but its parents are administrative aggregations whose boundaries trace state and PADD lines. The two structures, flow topology and partition tree, share the same set of physical nodes, but they reach upward to different parents — the flow topology to the next physical asset along the pipeline, the partition tree to the geographic region that contains it.

The practitioner workflow this thesis sets out to displace makes the inverse choice. Analysts typically take EIA's geographic series — production by PADD, consumption by refining district, stocks by PADD — and add or subtract them by region to compute regional balances. The implicit network is geographic, and any one asset's inventory (Cushing, Pearl Harbour, Bayou Choctaw) is recovered as a residual from those geographic operations. The framework reverses this. The connections between named physical assets are the primary object, and the aggregation views are a consistency layer over them. Inventories at named assets are generated by mass balance at the physical level, not reconstructed from regional sums. The partition tree is then the bridge: it exposes the same data at the regional resolution

practitioners are used to, but it does so by aggregating from a physical layer that already knows where each barrel actually lives.

The starter scenario carries 377 partition edges across 29 distinct parents and 211 distinct children. The active spine runs:

```
usa_view
├── padd1_view
│   ├── refining districts (RAP, REC)
│   ├── padd1_imports_agg ← MCRIPP12 − MCRIPP1CA2
│   ├── padd1_canadian_imports_agg ← MCRIPP1CA2
│   ├── padd1_exports_agg ← MCREXP12
│   ├── inter_padd_1_to_2_agg, inter_padd_1_to_3_agg
│   └── padd1_other (residual)
├── padd2_view
│   ├── refining districts (R2A, R2B, R2C)
│   ├── bakken_nd, oklahoma, padd2_other
│   ├── cushing_hub (with three operator sub-terminals), patoka_hub
│   ├── bakken_nd_gathering (inventory child, post-eleventh migration)
│   ├── padd2_canadian_imports_agg ← MCRIPP2CA2
│   ├── padd2_exports_agg ← MCREXP22
│   └── inter_padd_2_to_{1,3,4}_agg with receiver-side aliases
├── padd3_view, padd4_view, padd5_view (analogous structure)
└── (boundary nodes sit outside the partition):
    foreign_supply, canadian_oil_sands, foreign_export_destination, spr_total
```

The same structure populated with live values from the resolver at 2024-12-01 reads as follows (all flow quantities in kbd; inventory in MBBL). This is the data the HTML balance UI displays at the same date, materialised here as a static snapshot. The status column comes from v_aggregation_consistency, which classifies each row as ok (closes within tolerance), partial_coverage (one or more constituents missing or latent), or red (every constituent present and the gap exceeds tolerance — none at this level on this date).

Node	P (kbd)	C (kbd)	F_in (kbd)	F_out (kbd)	S (mbbl)	B (kbd)	Partition status
usa_view (USA)	13,437	16,772	6,557	3,752	806,948	−379	ok (closure exact)
├ padd1_view (East Coast)	46	745	705	92	7,872	−108	ok
├ padd2_view (Midwest)	1,837	3,942	4,503	2,426	103,681	−128	ok
├ padd3_view (Gulf Coast)	9,791	9,390	3,631	4,266	621,631	−105	ok
├ padd4_view (Rocky Mountain)	1,051	569	553	988	24,434	−32	ok
└ padd5_view (West Coast)	714	2,126	1,185	0	49,330	−7	ok

F_in at PADD level includes inter-PADD inflows from neighbouring PADDs as well as boundary inflows from foreign supply and Canadian production; F_out at PADD level includes inter-PADD outflows. At usa_view, by contrast, the inter-PADD movements (4,018 kbd in either direction at this date) net out and only the boundary flows remain — 4,234 kbd Canadian

+ 2,323 kbd foreign = 6,557 kbd of inflow, and 3,752 kbd of outflow to export destinations. Σ-of-PADDs equals usa_view to within rounding tolerance for every variable type listed.

Partition closure is the algebraic identity that the audit verifies: for every parent node N and every variable type T, the own value of N's T variable equals the sum of N's partition children's T variables. After twenty-three passes of refinement, this closure holds within rounding tolerance across all five PADDs and at the USA aggregate, in both the inflow and outflow directions. The numerical state of the audit at the time of writing is reported in Chapter 7.

5.8 Scenarios and authoritative declarations

A scenario is a pair: the persistent asset graph plus a set of `variable_assignments` rows tagged with the `scenario_id`. Different scenarios share the asset graph and the time-series data; they differ in which variables are bound to which time series, and which are derived from formulas.

The schema-level CHECK on assignment rows means each row binds the variable either to a time series (authoritative) or to a formula (derived), never both, never neither. The set of assignments under a scenario therefore encodes, for every variable, exactly which resolution is authoritative and which is derived. There is no separate coverage-contract structure; the per-variable choices are the contract.

The starter scenario's authoritative declarations summarise as follows:

Subsystem	Authoritative resolution	Reason
Permian, Bakken basins	basin aggregate	EIA publishes production at basin level for these
Eagle Ford	state-basin	Single-state basin; equivalent to basin level
Gulf of America, Alaska	physical node	Reported separately by EIA
Rest of Lower 48	residual aggregate	EIA publishes the residual only
Strategic Petroleum Reserve	site level	EIA PSM Table 38 publishes per-site monthly inventories
Cushing	hub level	Operator sub-terminals inactive; the storage report is at hub level
Houston exports	export aggregate	Facility sub-terminals inactive under a single aggregate
Stocks	PADD level (auxiliary at sub-PADD)	MCRSFP plus MCRSSP at PADD level; sub-decomposition would double-count
Balancing item B	PADD and USA	MCRUA series, promoted to time-series-observed in the ninth migration pass

Subsystem	Authoritative resolution	Reason
Inter-PADD flows	PADD-aggregate level	MCRMP series at the combined aggregate; per-pipeline split latent

Cross-scenario consistency — the test that the same time-series data, evaluated under progressively finer assignment sets, produces consistent aggregate values — is the second main consistency claim of the framework. The current implementation has `starter_us_crude_2015_2025` as the only active scenario; a coarser companion (`starter_basin`) is queued for the next development pass and will exercise the cross-scenario closure test directly.

5.9 The end-to-end pipeline

A complete run of the framework follows three phases. Each phase has a single concern and writes to a well-defined subset of tables.

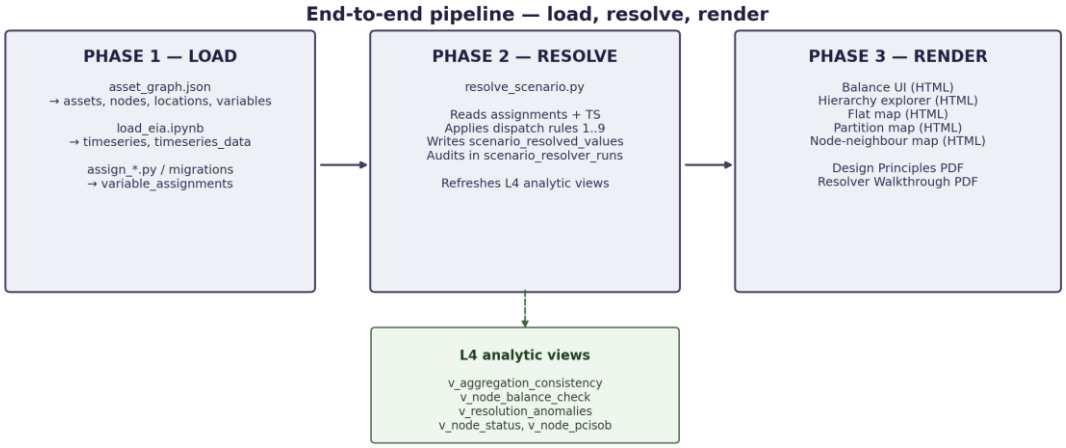


Figure 5. End-to-end pipeline — load, resolve, render — with the L4 analytic views consumed by the audit machinery.

Phase 1 — Load. The seed file `asset_graph.json` is loaded into the persistent asset graph (`load_asset_graph.ipynb` writes to `assets`, `nodes`, `locations`, `variables`). The EIA series catalogue and monthly facts are loaded (`load_eia.ipynb` writes to `timeseries`, `timeseries_data`). The per-scenario assignment rows are produced (`assign_formulas.ipynb` writes to `variable_assignments`). The node-type default formulas are populated (`populate_node_type_defaults.py` writes to `node_type_default_formulas`). Migration scripts (the `add_*.py`, `split_*.py`, `patch_*.py`, `repoint_*.py` family) evolve the topology and the assignments incrementally. After any structural change, `refresh_views.py --structural` rebuilds the L2 and L3 materialised views.

Phase 2 — Resolve. The script `resolve_scenario.py` reads the assignment set and the time-series facts, applies the dispatch rules described in Chapter 6, and writes one row per

(scenario, variable, observation date) to `scenario_resolved_values`. An audit row is written to `scenario_resolver_runs` capturing the run's metadata (start and end timestamps, duration, dispatch counts as JSONB, optional notes). On completion, the L4 analytic views are refreshed automatically.

Phase 3 — Render. The HTML renderers read from the materialised views and produce the user-facing visualisations: a balance UI (per-node mass balance with drill-down through the partition tree), a hierarchy UI (the full asset graph with a variable inspector), a partition map (click-to-drill geographic view), a single-node neighbour view, and a flat geographic map of all physical assets. Each rendered HTML carries a metadata beacon naming the resolver run that produced it, and an audit row is written to `scenario_html_artefacts` per regeneration. The orchestrator (`regenerate_htmls.py`) compares each HTML's beacon against the latest resolver run and rebuilds only the stale ones.

The layered view structure that supports this pipeline is the subject of the next chapter, where the resolver is described in detail.

6. The Resolver

The resolver is the mechanism that consumes the persistent asset graph and the per-scenario assignment set, evaluates every variable at every observation date, and writes the resulting per-(variable, date) values to a single table. Downstream consumers — the balance UI, the hierarchy explorer, the geographic map, the consistency audits, and the LP that Chapter 8 develops — all read from that single table. None of them reimplements the resolution logic; none of them queries the base tables directly for derived values.

This chapter walks through the resolver's design, the dispatch rules it applies, the dependency graph that orders the rule evaluations, and the layered view structure that downstream consumers see. The presentation closes with a short account of the bugs that were encountered during implementation, because the cycle gates and the latent short-circuit in the current design are direct responses to specific failures that earlier versions exhibited.

6.1 Why the resolver exists

Before the resolver, every consumer that wanted to read a variable's value reimplemented resolution logic itself. The balance UI carried a hundred-line common table expression handling time-series bindings, mirror formulas, reverse-paired-edge inheritance, and the mass-balance closure equation. JavaScript walked arithmetic formulas at render time because SQL could not evaluate multi-term expressions of the form A − B − C. Each new generator — the hierarchy explorer, the geographic map, the consistency views, the future forecasting features — would have duplicated that work.

The resolver collapses every consumer's resolution logic into a single deterministic pass:

```
inputs : variables + variable_assignments + timeseries_data
         + the active scenario_id
output : one row per (variable, date) in scenario_resolved_values
```

Every downstream consumer becomes a thin `SELECT` over the result table. The resolver runs in roughly twenty seconds for the starter scenario (1,870 variables × 156 monthly observations); it is run once per assignment change, not once per consumer.

6.2 The output table

The resolver writes to `oil_network.scenario_resolved_values`. Two columns from the table definition deserve particular attention:

```
CREATE TABLE oil_network.scenario_resolved_values (
    scenario_id       TEXT NOT NULL REFERENCES scenarios ON DELETE CASCADE,
    variable_id       TEXT NOT NULL REFERENCES variables ON DELETE CASCADE,
    observation_date  DATE NOT NULL,
    value             DOUBLE PRECISION,
    source            TEXT NOT NULL CHECK (source IN
```

```
                        ('observed', 'derived', 'zero', 'latent', 'unresolved',
'partial')),
    formula_used      TEXT,
    timeseries_id     TEXT,
    saved_date        TIMESTAMPTZ DEFAULT NOW(),
    run_id            BIGINT REFERENCES scenario_resolver_runs,
    PRIMARY KEY (scenario_id, variable_id, observation_date)
);
```

The value column is deliberately nullable. A NULL value paired with source = 'latent' is a valid and intentional row — it tells downstream consumers that the variable is declared, its value is unknown by design, and no retry is required. This is the schema-level expression of Corollary D: the resolver records that a flow exists structurally and is constrained by mass balance, but its per-edge value is not pinned down by observation.

The `source` column is a constrained enum with six categories:

Source	Meaning
observed	Direct time-series lookup. The ground-truth measurement
derived	Value computed by a formula (alias, sum, closure, arithmetic, reverse mirror)
zero	Structural zero, encoding a physical fact (a refinery does not produce crude)
latent	Declared variable with no resolution path. Value is NULL by design
unresolved	A formula was given but could not be evaluated. Zero rows in a healthy run
partial	Formula tried but at least one input was NULL on that date; value is NULL with a recorded note

The `formula_used` column preserves which dispatch rule fired and is the key to the framework's auditability. For any value in the resolved table, a reviewer can reconstruct the exact rule and the exact inputs that produced it. This matters because the framework is used as the data layer for downstream analyses (mass-balance checks, LP optimisations, future GNN forecasts), and the trustworthiness of those analyses depends on the trustworthiness of the values they consume.

Every run of the resolver writes an audit row to `scenario_resolver_runs` recording the start and end timestamps, the elapsed duration, the dispatch counts as a JSONB object, and optional free-text notes. Every resolved-value row carries the `run_id` that produced it, so historical comparisons across runs reduce to a single SQL join.

6.3 How a run flows end to end

The top-level shape of the resolver follows three phases:

```
   load()                fixed-point loop                        persist

DB ──────► assignments ──────────────────────► resolved values ──► DB
```

```
load: pull every variable_assignment row for the active scenario plus
      the time-series facts they reference. ~1,870 assignments and
      ~17,000 time-series facts for the starter.
fixed-point loop: sweep every unresolved variable repeatedly. Each
      iteration evaluates whichever variables now have their inputs
      known. Terminal kinds (observed, zero, latent) have no inputs
      and resolve in iteration 1; formula kinds (sum, arithmetic)
      resolve once their dependencies are in. The loop stops when an
      iteration produces no new resolutions.
persist: write one row per (scenario, variable, date) to
      scenario_resolved_values, plus a single row in
      scenario_resolver_runs recording the dispatch counts and the
      run duration.
```

Each phase has a single concern, isolated from the others. The dispatch loop never reads from the database; the loader never evaluates formulas. The fixed-point character of the central loop is the key design point. There is no pre-computed evaluation order; the loop discovers the order by iteration. A chain like A = TS, B = A, C = B + A resolves in three iterations. The starter scenario, with 1,870 variables and a dependency-DAG depth of approximately five, converges in five passes in roughly twenty-five seconds on a laptop.

6.4 The dispatch loop

The classifier returns one of six kinds for each assignment: observed, zero, latent, sum, arithmetic, or unknown (the fallback that should fire for no variable in a healthy run). The fixed-point loop dispatches each variable into the corresponding evaluator the moment the variable becomes evaluable. Three of the kinds — observed, zero, and latent — are terminal: they have no formula_inputs and become evaluable as soon as the loop reaches them on iteration 1. The remaining two formula kinds (sum and arithmetic) are evaluable only when every variable in their formula_inputs is already in the resolved set.

The full set of canonical rule labels — that is, the only `source` values that `scenario_resolved_values` ever records — is the short list given in Section 6.2. Three earlier labels (`sum_over_children`, `sum_over_outflows`, `sum_same_type`) were collapsed into the canonical `sum` in the twelfth implementation pass; they no longer appear in the

codebase or the resolved table. The lesson behind that consolidation is the canonical-sum principle from Chapter 4.

The rules are as follows:

Rule 1: Observed. Fires when `timeseries_id IS NOT NULL`. Performs a direct time-series lookup at the observation date. This is the ground truth; every other rule is downstream of these. In the starter scenario, 80 variables fire this rule.

Rule 2: Structural zero. Fires when `formula = '0'`. Used for refinery production, pass-through inventory, and other slots where the value is zero by physical construction. Roughly 542 variables fire this rule, mostly non-relational scalars inherited from `node_type_default_formulas`.

Rule 3: Latent, with reverse-mirror promotion. Fires when `formula = 'latent()'`. Records `value = NULL` and `source = 'latent'`. But first, the rule attempts a reverse-mirror promotion: if the paired direction of the variable (opposite variable type, swapped endpoints) has a resolved value, the rule borrows that value through the mirror identity (an inflow into A from B equals an outflow from B to A). Without this promotion, a latent outflow from `canadian_oil_sands` to PADD 2 would record NULL even when the inflow side of the same edge had an observed value. With the promotion, the latent borrows correctly. The rule fires 767 times in the starter scenario after the promotion attempt; the 15 that succeeded are counted separately under Rule 6.

Rule 4: Sum. Fires when `formula = 'sum'`. Sums every value in `formula_inputs`; NULL inputs are skipped. This is the twelfth-pass collapse of three earlier sum-type labels into one canonical rule. The semantic role of any given sum — aggregation parent, fan-out total — is recoverable from the structure of `formula_inputs` (same-type and same-commodity inputs imply aggregation; mixed-type at same node implies fan-out), so the formula text does not need to encode it.

Rule 5: Single-variable alias. Fires when `formula = bare_variable_id` and the `formula_inputs` list contains at most that one entry. The variable inherits its value from the named target. Most aggregate-to-aggregate alias chains fire this rule; in the starter scenario, the arithmetic rule resolves 451 variables in total, the majority via this single-variable case (the code labels this dispatch kind 'arithmetic'; earlier drafts called it 'alias').

Rule 6: Reverse mirror. Fires when a relational variable has no own resolution but its paired direction (opposite type, swapped endpoints) has a resolved value. The variable borrows that value through the mirror identity. After a sequence of dependency-graph fixes (the "ninth-pass" fix described in Section 6.6), this rule fires deterministically: `build_deps()` now guarantees the paired side is resolved before this side is evaluated. Fifteen mirrors fire reliably in the starter scenario, where previously only three fired by luck of insertion order.

Rule 7: Closure formula. Fires when the variable is a balancing item whose `formula_inputs` include a mix of inventory, inflow, and outflow variables. Evaluates the closure equation:

$B = \Delta S - P + C - \Sigma F_{in} + \Sigma F_{out}$

This rule is now **dormant** in the starter scenario because B was promoted to time-series-observed in the sixth migration pass (the MCRUA series at PADD and USA level). The closure formula remains in the resolver as a fallback constraint, and it is the engine behind the Chapter 7 view that compares observed B against closure-derived B as a measure of partition-internal latent flow.

Rule 8: Arithmetic combination. Fires when the formula text matches a signed-variable-references pattern (`A - B + C`). Used for residual definitions such as `padd2_other.P = padd2_view.P - bakken_nd.P - oklahoma.P`. The parser is intentionally minimal — a regular expression that pulls out signed terms — and aborts cleanly if any term fails to match a variable in `formula_inputs`. Eight variables in the starter scenario fire this rule.

Rule 9: Unresolved. The fallback. Records `value = NULL`, `source = 'unresolved'`. In a healthy run this fires zero times; non-zero counts indicate a bug or an unmodelled formula pattern. The expected operator response is to fix the assignment or extend the resolver, never to silently treat the row as latent.

The current dispatch counts in the starter scenario, with the canonical labels, are:

Rule	Count
observed (time-series lookup)	90
zero (structural)	542
latent (including reverse-mirror failures)	767
arithmetic	451
reverse-mirror (Rule 3 promotion plus Rule 6)	15
sum (canonical)	5
closure	0
unresolved	0

The zero in the last row is the load-bearing invariant. Every declared variable has either a value or an explicit latent marker. Downstream consumers can rely on the presence of a row for every (variable, date) pair, and on the source column telling them whether a NULL is by design (latent, to be treated as a free variable in optimisation) or by failure (unresolved, an unmodelled case requiring attention).

6.5 The fixed-point loop

The loop reads as one statement of the algorithm. The pseudo-code below is faithful to the implementation in code/recursive_resolver.py:

```
unresolved := { all variable_ids in the scenario }

while progress:
```

```
progress := false
for v in unresolved:
    kind := classify(v)
    if kind in { observed, zero, latent }:
        evaluate v                              # no deps; always evaluable
    elif kind in { sum, arithmetic } and
         every input in formula_inputs(v) is in resolved:
        evaluate v
    else:
        continue                                # not yet evaluable
    resolved[v] := value
    unresolved -= { v }
    progress := true
```

anything still in unresolved is genuinely 'unknown' — zero in a healthy run.

The stopping rule is just no new variable was resolved this iteration. The convergence is bounded by the depth of the dependency DAG. For the starter scenario, the deepest path is roughly five layers — a refinery's consumption variable is TS-observed, it rolls up to its refining district as a sum, the district rolls up to its PADD as a sum, the PADD into usa_view as a sum, and a handful of arithmetic combinations sit at the top — so the loop converges in five passes over the 1,870-variable assignment set in roughly twenty-five seconds.

The gating condition for the formula kinds is every input has been resolved, not every input has a non-NULL value. A sum or arithmetic variable that references a latent input still evaluates: the latent input is in resolved with value = NULL, and the eval_sum or eval_arithmetic function emits a row tagged source = 'partial' for any date where the input was NULL. Without this convention, formula variables that depend on latents would never converge in the loop. Putting latents on the terminal-kinds side of the dispatch is what makes the convention work.

The fixed-point pattern has practical advantages over the topological-sort approach the resolver used in earlier versions. There is no separate dependency-graph data structure to maintain; the dependencies are read from formula_inputs at each iteration. There is no cycle-detection failure to handle, because the loop naturally skips a variable whose inputs aren't ready and tries again next iteration; a true cycle would manifest as the loop never converging, which a safety cap on the iteration count catches and reports. And the mirror-promotion logic that was previously a post-pass over the topo-sorted result is on a path to becoming a regular

formula kind (formula = 'mirror' with one input in formula_inputs); the loop will then resolve mirrors in iteration N + 1 after their paired direction resolves in iteration N, with no special case.

An equivalence harness, code/compare_resolvers.py, runs the legacy topological-sort resolver and the fixed-point resolver back to back on the same scenario and diffs every row of scenario_resolved_values on (variable, date, value, source). On the starter scenario the harness reports 291,564 identical rows and seven identical dispatch counts (observed 90, zero 542, latent 767, sum 5, arithmetic 451, reverse_mirror 15, unresolved 0). The two implementations produce the same scenario state; the new one is shorter and reads more directly as the algorithm it implements.

6.6 Bugs encountered and fixed

The resolver's current design is the result of three specific failures encountered during implementation. Each failure exposed an assumption that was implicit in the earlier design; each fix made the assumption explicit and enforced it.

The latent-mirror cycle. An early implementation had latent variables add their paired direction as a dependency unconditionally. When the paired direction was also latent and aliased to its own paired side, this produced a two-node cycle (us → paired → us), and the topological sort raised `CycleError`. The fix was to require the paired side to be non-latent before adding the dependency — the first of the two cycle gates above.

The latent short-circuit that hid reverse-mirror. Rule 3 (latent) ran before Rule 7 (reverse-mirror) in the original priority order. A variable explicitly marked latent would record `value = NULL` even when its paired direction had an observed value. The fix was twofold: promote the mirror check inside Rule 3 (if the paired side is resolved, borrow its value first; only fall through to NULL otherwise), and hoist the reverse-mirror dependency above the early-continue path in `build_deps()` so that latent variables get their paired-side dependency added before being skipped. Together, the two changes lifted mirror dispatch from three cases (succeeding by luck of insertion order) to fifteen (deterministic).

The alias-rule wrapper bug. Rule 5 (single-variable alias) matches when the formula text is itself a bare variable ID. Earlier migration scripts had written `formula = 'alias(variable_id)'` with the explicit wrapper string; Rule 5 missed those rows and they were picked up by a different rule by coincidence. The fix was in the migration scripts: the canonical storage is `formula = bare_variable_id`, with the wrapper string retained only as the display label in `formula_used` on resolved rows.

The pattern across all three fixes is the same: an assumption that an earlier implementation took for granted (that latents were terminal, that aliases would never form cycles, that the wrapper string was harmless) turned out to be wrong in specific cases. The current resolver

makes each assumption explicit by name in the code, and the dispatch counts in the audit row are the daily check that none of the assumptions has silently re-broken.

A fourth fix worth describing is structural rather than localised to a particular rule. The original resolver design required a build_deps() pass to construct the explicit dependency dictionary that fed the topological sorter, plus a separate promote_mirrors() post-pass after the main loop to fill in latent relational variables whose paired direction had resolved. Each of those secondary structures had its own subtle failure modes — cycle false positives in build_deps(), reverse-mirror promotion missing the source-attribution label in promote_mirrors(). The current resolver retires the topological sort. The dependency graph becomes implicit in the fixed-point loop's per-iteration check that all inputs are in the resolved set, and the mirror promotion remains as a post-pass for the moment but is on a path to becoming a regular formula kind. The harness code/compare_resolvers.py verifies equivalence: 291,564 rows match exactly between the two implementations on the starter scenario.

6.7 Latent versus unresolved

Both `latent` and `unresolved` rows carry `value = NULL` in the resolved table, but they mean different things and the framework treats them differently. The distinction is the closing commitment of the framework's missing-value treatment, and it is worth restating in concrete terms because it determines how the LP downstream consumer (Chapter 8) and any future GNN consumer (Chapter 10) should interpret the NULLs they encounter.

A `latent` row is the assignment `formula = 'latent()'`, with the reverse-mirror promotion not having fired. The variable is declared but its value is unknown by design. The typical case is the per-route flow at a fan-out junction, where the framework knows the routing topology and the aggregate constraint but not the per-route values. Downstream, the row should be treated as a free variable in an optimisation, constrained by mass balance, capacity, and any observation that references it. Bounds come from node configuration: refinery throughput capacity, pipeline maximum daily throughput, terminal storage capacity.

An `unresolved` row is a resolver failure. A formula was given but could not be evaluated. The typical cause is a referenced variable being missing from the assignment set, or the formula matching no rule pattern. Downstream, an unresolved row is a bug. The expected operator response is to fix the assignment (correct the reference) or extend the resolver (add a rule for the new pattern). Silent treatment as latent is explicitly prohibited.

The starter scenario at the time of writing has 767 latents and zero unresolved. The 767 latents are concentrated where the framework already expects them to be: junction fan-outs in the Permian and Bakken gathering systems, the multiple offshore Gulf production routes, and the per-pipeline splits inside each inter-PADD aggregate. The zero unresolved is the guarantee that the framework has no unmodelled cases under the current assignment set.

6.8 The layered view structure

Once `variable_assignments` is consistent and the resolver has run, the rest of the framework is a stack of views derived from the resolved table and the assignment table. Renderers, audits, and the LP exporter read from these views; none of them re-derives structure from the base tables. The layer structure makes the dependency chain explicit and keeps each refresh cheap.

Layer	View	Reads from	Provides
L1	v_effective_assignments	variable_assignments + defaults	One row per (scenario, variable) with the layered recipe and an audit tag for where the formula came from
L2a	v_formula_input_links	L1	One row per arrow in formula_inputs — the atomic (parent, child) pair, scenario-tagged
L2b	v_aggregation_edges	L2a + variables	The aggregation graph: every cross-node reference, annotated with the parent's formula and time-series-bound flag
L2c	v_flow_edges	variables	The physical flow topology: one row per outflow variable with a related node, interpreted as (source → target)
L3a	v_partition_tree	v_aggregation_edges	The partition spine: same-type, same-commodity, same-related-node aggregation edges, with arithmetic residuals excluded
L3b	v_node_status	L1	The per-(scenario, node) status from Section 4.5: authoritative, derived, or inactive
L4a	v_node_balance_check	scenario_resolved_values + v_flow_edges	Per-(node, date) mass-balance closure: sum_in, sum_out_obs,

Layer	View	Reads from	Provides
			sum_out_implied, gap_kbd
L4b	v_aggregation_consistency	v_partition_tree + scenario_resolved_values	For every time-series-observed aggregate, the cross-check abs(TS value − Σ constituents) — flagged ok / partial / inconsistent
L4c	v_inventory_changes	scenario_resolved_values	Per-stock-series ΔS in MBBL and kbd; used by the balance UI to display inventory deltas
L4d	v_aggregate_balance	v_partition_tree + L4a	Closed mass balance for the active scenario's balance-role aggregates (usa_view plus the five PADD views)
L4e	v_node_pcisob	scenario_resolved_values	Per-(node, date) totals for the six variable types, pre-aggregated so renderers do not redo the GROUP BY at every page load

The refresh discipline is straightforward. The L2 and L3 views are refreshed only when `variable_assignments` or `variables` changes — rare events tied to deliberate migrations. The L4 views are refreshed after each resolver run, since they depend on `scenario_resolved_values`. The resolver script calls `refresh_analytic()` automatically after a successful run.

6.9 Persistence and the audit trail

After the dispatch loop finishes, the resolver clears the scenario's existing rows in `scenario_resolved_values` and writes the new ones via `execute_values` in batches of 5,000. The audit row in `scenario_resolver_runs` is updated with the completion timestamp, the elapsed milliseconds, and the dispatch counts as JSONB.

If the resolver crashes mid-way, the open audit row with `NULL` in `completed_at` is itself a useful signal: it tells the next operator that a run was started but did not finish. The resolved-values table is unaffected by a crash because the `DELETE`/`INSERT` pair runs inside a transaction. There is no half-written intermediate state to clean up.

The resolver and the layered views together provide the foundation for the consistency claims developed in the next chapter. Every claim made about the framework's behaviour is in principle reproducible from a single resolver run and a small number of SQL queries against the L4 views. This is the design property that makes the framework auditable, and it is what distinguishes the schema from a pure modelling exercise.

6.10 Last-observation-carried-forward

Rule 1 (observed) implements the temporal convention introduced as Corollary D-bis: last-observation-carried-forward, or LOCF. Concretely: if a series reports 1,200 kbd on 2024-06-01 and the next observation lands on 2024-09-01 at 1,250 kbd, the resolver returns 1,200 kbd for every date from 2024-06-01 through 2024-08-31, and 1,250 kbd from 2024-09-01 onwards. The value at any date is the most recently observed value at or before that date. There is no interpolation between observations; the time series is a step function. Dates earlier than the first observation receive no row. Section 4.11 sets out why LOCF is the right convention; this section covers how the resolver records it and what to look for in the audit trail.

Every resolved row records LOCF in the formula_used column on scenario_resolved_values. Fresh observations carry formula_used = NULL. Carried-forward rows record the source date as formula_used = 'locf(YYYY-MM-DD)'. A downstream consumer can therefore tell freshly published values from carried-over ones. A frequency-gap audit can also flag long LOCF runs against the expected publication cadence, which is useful when a series has lapsed without a fresh release.

A live example illustrates the temporal convention at work. The variable montana_other.production is defined as montana_state.production minus bakken_mt.production. The state-level series MCRFPMT2 ends earlier than the STEO basin forecast COPRBK. LOCF holds the Montana state value flat from MCRFPMT2's last observation onward, while Bakken-MT keeps forecasting upward, so the residual jumps downward and goes negative for the months between the two horizons. The discontinuity is the point. The framework's temporal convention does not interpolate across granularities, and when two source publications with different update cadences feed the same residual the gap between them surfaces in the resolved data as an honest jump rather than as a quiet drift. The v_resolution_anomalies view tags the pattern as a low-severity anomaly for review. The analyst's response then becomes a choice — find a more granular source, accept the LOCF gap until the next state-level publication lands, or note the divergence in their downstream model. The framework's job is to surface the discontinuity, not to hide it.

LOCF is the convention most commercial data providers follow as well. Kpler reports vessel positions and cargo manifests by carrying forward the most recent observation until a new AIS ping or manifest update arrives; Genscape's tank-level imagery is taken at a daily cadence and held flat between captures; Wood Mackenzie's IIR holds refinery status flat between successive surveys. Adopting LOCF in the framework therefore keeps the temporal

convention identical to what downstream consumers already expect from their commercial feeds, and the formula_used audit trail makes the convention auditable in a way that most providers' downstream data delivery does not.

7. Consistency Guarantees

The framework's claim to be a contribution rests on the consistency properties it provides to downstream consumers. A linear programme that reads from the resolved table assumes that mass balance holds. A graph neural network that consumes the node features assumes that aggregate values match the sum of their constituents. A human analyst inspecting the balance UI assumes that the numbers add up. Each of these consumers is entitled to those assumptions only if the schema enforces them, the resolver respects them, and the audits demonstrate them on real data.

This chapter sets out the four consistency claims the framework supports, the queries that test each one, and the numerical state of each claim under the active starter scenario at the time of writing.

7.1 Claim 1: Schema-level single attribution

The first claim concerns the source of value for each variable assignment and is enforced at the database schema level rather than verified after the fact. For each (scenario, variable, commodity, timestamp), the value of the variable comes from exactly one source. Either the variable is bound to a time series, or it is bound to a formula. It cannot be both and it cannot be neither. Aggregation relationships declared in formula_inputs are independent of the value source and are not affected by this constraint (the dual role is explained in Section 4.8). The CHECK constraint on variable_assignments is:

```
CHECK (num_nonnulls(timeseries_id, formula) = 1)
```

This constraint cannot be violated by an INSERT. Any attempt to write an assignment row that fails the test is rejected by the database before the data can propagate anywhere. Schema-level enforcement is the strongest form of consistency guarantee the framework provides: rather than detecting inconsistent state after the fact, it makes inconsistent state unrepresentable.

A complementary audit at the time-series attribution level enforces single-attribution from the other direction. For each scenario, each time series is bound to exactly one variable. The audit query is:

```
SELECT timeseries_id, COUNT(*) AS n_bindings
FROM variable_assignments
WHERE scenario_id = 'starter_us_crude_2015_2025'
  AND timeseries_id IS NOT NULL
GROUP BY timeseries_id
HAVING COUNT(*) > 1;
```

Under the active scenario, this query returns zero rows: 90 distinct time series are bound across 90 assignment rows, in a strict 1:1 attribution. The same time series is never used as the authoritative source for two different variables within the same scenario.

Together, the schema-level CHECK and the time-series attribution audit give the framework its central consistency property: every value in the resolved table is traceable to exactly one source, and no source contributes to more than one variable.

7.2 Claim 2: Partition closure

The second claim is algebraic. For every parent node N in the partition tree and every variable type T (production, consumption, inventory, balancing item), the own value of N's T variable equals the sum of N's partition children's T variables, within rounding tolerance.

The partition closure test is implemented in the view `v_node_balance_check` (the inflow and outflow sides are tested separately, since aggregation works the same way in both directions) and the per-variable-type aggregation is in `v_aggregation_consistency`. The audit query for the inflow side at a representative aggregate is:

```
SELECT
    node_id,
    sum_in   AS own_value,
    bdy_in   AS sum_of_children,
    abs(sum_in - bdy_in) AS gap
FROM v_node_balance_check
WHERE scenario_id = 'starter_us_crude_2015_2025'
  AND node_id IN ('usa_view', 'padd1_view', 'padd2_view',
                  'padd3_view', 'padd4_view', 'padd5_view')
  AND observation_date = '2024-06-01';
```

The result of this query, averaged across all 156 monthly observation dates in the scenario, is:

Node	Own inflow (kbd)	Σ children inflow (kbd)	Gap (kbd)	Own outflow (kbd)	Σ children outflow (kbd)	Gap (kbd)
usa_view	6,557	6,557	0	3,752	3,752	0
padd1_view	705	705	0	92.2	92.2	0
padd2_view	4,503	4,503	0	2,426	2,426	0
padd3_view	3,631	3,631	0	4,264	4,264	0
padd4_view	552.7	552.7	0	987.5	987.5	0
padd5_view	1,183	1,183	0	—	0	0

Partition closure holds at every PADD aggregate and at the USA aggregate, in both the inflow and outflow directions, after twenty-three passes of refinement to the assignment set. The closure is exact (gap = 0 to the rounding precision of the underlying data) rather than approximate, because the same-type aggregation links in the partition tree are exact algebraic sums, not estimates.

The "—" entry for PADD 5 own outflow reflects that PADD 5 (the West Coast, comprising California, Washington, Oregon, Alaska, and Hawaii) is a net importer with negligible inter-PADD outflow in the modelled period. The own value is recorded as zero and the children's sum is zero; the test passes trivially.

The audit distinguishes two failure modes for a partition row. In the first, every declared constituent has a value and the sum disagrees with the observed parent: a genuine inconsistency in the underlying data. In the second, one or more constituents are missing or latent; the sum is necessarily incomplete and the comparison cannot be made. The framework reports only the first as a true inconsistency. The second is reported as partial_coverage. Without external knowledge, the framework cannot reliably distinguish 'the partition is wrong' from 'the partition is incomplete', so the inconsistent label is not applied in practice; every non-conforming row is reported as partial_coverage, and the user interface separates them visually. A red cell indicates that every constituent is present and the gap exceeds tolerance; a yellow cell indicates that at least one constituent is missing or latent and the gap is unverifiable. At the time of writing, no cell is red at the USA or PADD level; all open partition gaps are partial_coverage attributable to documented latent constituents.

7.3 Claim 3: Mass balance at every node

The third claim is per-node and per-date. For every node i, every commodity g, and every observation date t:

$\Delta S(i, g, t) = P(i, g, t) + F_{in}(i, g, t) - C(i, g, t) - F_{out}(i, g, t) + B(i, g, t)$

This is Corollary A made operational. The check is implemented in v_node_balance_check, which produces one row per (node, observation_date) with the four flow totals, the stock delta, the balancing item, and the residual that the closure should satisfy.

The mass-balance check is universal: it applies to every node type, with pass-through nodes (gathering, pipelines, terminals) inheriting $P = C = \Delta S = B = 0$ from `node_type_default_formulas`, so that for pass-through nodes the constraint reduces to $\Sigma F_{in} = \Sigma F_{out}$. The constraint is enforced as a validation rather than as a construction rule. The variables are assigned independently from time series and formulas; the mass balance is a property that emerges from the assignments and is checked after the resolver has run.

Where the mass balance does not close exactly at a node — typically a node with both an observed stock series and observed flow series, where measurement timing and rounding produce a small residual — the residual is absorbed into B(i, g, t). At the USA aggregate, B is itself time-series-observed (the EIA MCRUA series), so its value at every date is an independently published quantity. The framework's mass balance therefore reduces, at the USA level, to a check that EIA's published balancing item matches the closure-derived balancing item that the framework would compute if it were not observed.

That comparison is the subject of the next claim.

7.4 Claim 4: Observed B versus closure-derived B

The fourth claim is a richer one. Because the balancing item is independently time-series-observed at the PADD and USA levels (via the MCRUA series, promoted to authoritative in

the sixth migration pass), the framework can run two computations of B in parallel: the observed value from MCRUA, and the closure-derived value computed from the mass balance equation as though B were unknown.

In a perfectly consistent framework — one in which every flow at every level is observed without error, every stock is reported precisely, and every measurement is timed identically — the two values of B would coincide at every date. In a real framework operating on real public data, they differ. The magnitude and timing of the difference is operationally informative.

The view `v_aggregation_consistency` computes the difference at the USA aggregate, and a downstream chart (the "Claim 4 chart" in the renderer code) plots it monthly from 2015 onwards. The signal that emerges is striking: the difference is small and unsigned in normal periods (a few tens of kbd around a mean of zero), and sharply spiked around named disruption events.

The specific events that appear as spikes in the chart are:

Period	Event	Approximate magnitude
August–September 2017	Hurricane Harvey	400–600 kbd spike in observed-minus-closure B
March–May 2020	COVID-19 demand collapse	300–500 kbd persistent residual
May 2021	Colonial Pipeline ransomware shutdown	200–300 kbd spike
April–November 2022	SPR strategic release	100–250 kbd persistent residual matching the release schedule

Each spike has a credible mechanical explanation. Hurricane Harvey shut Gulf Coast production and refining for weeks, and during that period the EIA's monthly aggregate flow series were unable to capture intra-month operational disruptions at the daily granularity the underlying movements required. COVID produced sudden inventory builds across the system as demand collapsed faster than production could throttle back, and the data infrastructure had not been designed for movements of that magnitude in a single month. Colonial introduced a discontinuity in the East Coast supply chain that took several months of reporting to fully reconcile. The SPR release deliberately introduced a persistent flow that the framework can see in the stock change at the SPR sites but that the inter-PADD flow series did not always reflect with matching timing.

The interpretation the framework supports is that the difference between observed and closure-derived B at the USA level is, to a first approximation, a measure of partition-internal latent flow that the public data does not directly report. The magnitude of the difference at each date is therefore a measure of how much information the framework is currently unable to resolve. This is itself a useful diagnostic: when the difference is small, the framework's representation closely matches the data; when the difference spikes, there is real movement happening that the available time series cannot fully capture, and the latent variables become correspondingly more important.

7.5 Cross-scenario consistency

The four claims above hold under the framework as committed. A fifth, cross-scenario consistency, asks whether a second scenario binding the same underlying time series at a different resolution would produce aggregate values that agree with the first wherever the two scenarios overlap. The schema and the resolver admit such a test without modification; executing it is future work.

7.6 Operational consistency of the resolver

A separate class of consistency claim concerns the resolver itself. Two properties hold by construction:

No unresolved rows. In a healthy run of the resolver on the starter scenario, the unresolved count is zero. This is the load-bearing invariant from Section 6.4: every declared variable either has a value or has an explicit latent marker. Downstream consumers can rely on the presence of a row for every (scenario, variable, observation_date) tuple.

Deterministic dispatch. Given the same assignment set and the same time-series data, the resolver produces identical dispatch counts across runs. The dispatch counts are recorded in the audit JSONB column of `scenario_resolver_runs` and can be cross-checked between runs. The current dispatch counts (from Section 6.4) are stable across repeated runs of the same scenario.

These properties matter because they let the framework be used as the data layer for downstream analyses without re-running the upstream computation each time. A consistency audit run today, an LP solve run tomorrow, and a renderer run next week all see the same values from the same resolver run, because the audit row records which run produced which values.

7.7 What the consistency claims do not cover

The framework's consistency claims are about internal coherence, not about external correctness. Specifically, the claims do not establish:

The accuracy of the underlying EIA data. If the EIA's MCRUA series is itself biased or noisy, the framework will faithfully reproduce that bias. The framework's role is to make the bias visible and traceable, not to correct it.

The completeness of the modelled topology. If a real physical flow is not represented as an edge in the asset graph, the framework will not detect its absence. Topological completeness depends on the construction passes (Chapter 5) and is a matter for review.

The realism of the latent allocations. The framework leaves per-edge flows latent at junctions and does not commit to specific values for them. Whether the eventual LP or GNN allocation matches what is physically happening on the ground is a question for the consumer model,

not for the schema. The framework guarantees only that the aggregate constraint is satisfied, not that the per-edge split is correct.

These three areas correspond directly to the future work outlined in Chapter 10. The Chapter 7 claims are about the schema and the resolver; the question of whether the values they produce reflect reality is downstream of those claims.

7.8 Summary

The framework supports four numerical consistency claims: schema-level single attribution, partition closure, per-node mass balance, and observed-versus-closure B. A fifth claim, cross-scenario consistency, is admissible by construction but is left to future work. Each claim corresponds to a specific audit query and a specific view in the live system.

The numerical state of the framework at the time of writing is clean: 80 time-series bindings in 1:1 attribution, partition closure within rounding tolerance at every PADD and at the USA aggregate, mass balance respected at every node, and the observed-versus-closure B difference confined to known disruption events. The framework is ready to be consumed by downstream models, beginning with the linear programme described in the next chapter.

7.9 Where each axiom is enforced

A useful way to take stock of the framework is to walk through the build pipeline step by step and, at each step, name the axioms and corollaries that step embodies together with the database design, the view, or the audit that actually enforces them. The table below is that walk-through. It is the same content as Sections 4.1–4.13 and 5.1–5.9 of the thesis, projected onto the operational pipeline rather than onto the framework's design vocabulary. Repeated process-step cells are blank for visual grouping; each row carries a single (axiom-or-corollary, enforcement) pair.

Process step	Axiom or Corollary	Enforcement mechanism
Load asset graph	Axiom 1: asset-centric representation	kind flag on assets (physical | abstract); physical assets carry capacity, geography, and flow edges
	Axiom 4: persistent graph vs scenario state	Two-layer schema: assets / nodes / locations persist; variable_assignments is per-scenario
	Axiom 6: bidirectional flows as two directed edges	Each direction is a separate inflow / outflow variable with its own related_node_id
Load EIA timeseries	Source provenance (Corollary D-bis precondition)	Separate staging schema (oil_network_data_loader) feeds the live tables; timeseries / timeseries_data hold catalogue + facts
Assign variables	Axiom 5: observed XOR derived	CHECK (num_nonnulls(timeseries_id, formula) = 1) on variable_assignments
	Axiom 5: TS-binding uniqueness	audit_ts_binding_uniqueness — one TS bound to at most one variable per scenario

	Corollary D: latent allocation at junctions	formula = 'latent()' declarations on un-observable per-route splits
	Axiom 2: stable topology via zero-flow	formula = '0' defaults via node_type_default_formulas for pass-through node types
Build views (L2–L4)	Axiom 3: variables are the single source of truth	v_flow_edges, v_aggregation_edges, v_partition_tree, v_node_pcisob — every structural fact computed from the variables collection
	Corollary B: observational layer has no edges	v_flow_edges filters relational variables only; abstract aggregates are absent by construction
	Corollary C: labels distinct from hierarchies	v_partition_tree filters same-type, same-related-node; geography labels do not enter the partition spine
	Corollary E: node status as a view	v_node_status derives authoritative / derived / inactive per (scenario, node) from variable_assignments
Run resolver	Corollary D-bis: last-observation-carried-forward	eval_observed() in resolve_scenario.py / recursive_resolver.py; formula_used column records 'locf(YYYY-MM-DD)' on carried rows
	Axiom 3: derived side of the source of truth	Dispatch rules sum, alias, arithmetic consume formula_inputs at evaluation time
	Resolver output invariants	CHECK constraint on scenario_resolved_values.source restricts to {observed, derived, zero, latent, partial, unresolved}
Post-resolve audits	Corollary A: mass balance at physical nodes	v_node_balance_check flags any node where $\Delta S \neq P + \Sigma F_{in} - C - \Sigma F_{out} + B$
	Corollary A: cross-check at aggregate level	v_aggregation_consistency flags $\lvert TS_{value} - \Sigma \text{ children} \rvert$ over tolerance
	Corollary D-bis: temporal anomalies	v_resolution_anomalies tags long_locf_run and negative_derived rows for review
	Capacity bounds (Section 5.7 extension)	audit_capacity_violations.py compares scenario_resolved_values against v_effective_constraints
Render HTMLs	All axioms and corollaries — read-only verification	NetworkGraph engine + renderers (make_balance_resolver_ui.py and siblings) consume the L4 views directly; no business logic is reimplemented in JS

8. Linear Programming as a Downstream Consumer

The asset-centric schema, the resolver, and the consistency guarantees of the preceding chapters establish a representational framework. The framework's value depends on what downstream consumers can do with it. This chapter develops one such consumer in detail: a linear programme that takes the resolver's output as input and solves for the latent flows at junction nodes, subject to mass balance, capacity, and the observed values that the framework has resolved.

The choice of linear programming as the lead consumer reflects three considerations. First, LP is the natural model for the latents the framework leaves explicit (Corollary D): junction flows are decision variables, the aggregate constraints are equality rows, and node capacities are inequality rows. Second, LP is well understood, mature, and supported by robust solver infrastructure, so the framework's value can be demonstrated without depending on speculative modelling choices. Third, LP shares its data structure with a broad family of related consumers (multi-period LP, mixed-integer LP, network simplex, and ultimately graph neural networks for forecasting), so the design choices made for LP transfer to those consumers with minimal modification. The last point is developed in Chapter 10 as part of the future-work discussion.

8.1 What the LP takes as input

The LP reads from the resolved table and the structural views. Specifically, it consumes four tensors:

The node-by-time value matrix. Built from `scenario_resolved_values` by pivoting on (variable_id, observation_date). Each row is a variable; each column is a date. Cells contain the resolved value, with NULL where the value is latent or unresolved.

The adjacency structure. Built from v_flow_edges. Each row is a directed edge (source node, target node, edge variable). This is the fixed topology that holds across all dates, in accordance with Axiom 2.

The constraint structure. Built from `v_node_balance_check` (per-node mass balance) and from node-level capacity attributes (refinery throughput capacity, pipeline maximum daily throughput, terminal storage maximum). Mass balance contributes one equality row per (node, date); capacity contributes one inequality row per (capacity-bearing variable, date).

The objective coefficients. Supplied by the modeller. Different applications produce different objectives (cost minimisation under a freight-rate matrix, capacity-utilisation maximisation, target-stock pursuit), and the framework is agnostic to which objective is chosen. The LP exporter prepares the data; the modeller specifies the objective.

The LP's decision variables are the latent rows in the value matrix — the cells where the resolved value is NULL with `source = 'latent'`. Observed and derived cells are constants.

Unresolved cells (which should not occur in a healthy run, by Section 7.6) are treated as errors and halt the export.

8.2 Mass balance as LP rows

For each node i, each commodity g, and each observation date t, the mass balance equation from Corollary A contributes one equality row to the LP:

$P(i, g, t) + F_{in}(i, g, t) - C(i, g, t) - F_{out}(i, g, t) + B(i, g, t) - \Delta S(i, g, t) = 0$

Substituting the relational variables in terms of their per-edge components:

$P(i, g, t) + \Sigma_{j \in N_{in}(i)} \phi(j, i, g, t) - C(i, g, t) - \Sigma_{k \in N_{out}(i)} \phi(i, k, g, t) + B(i, g, t) - \Delta S(i, g, t) = 0$

where $\phi(j, i, g, t)$ is the per-edge flow from j to i on commodity g at date t, and $N_{in}(i)$, $N_{out}(i)$ are the predecessor and successor sets of i in the flow topology.

The structure of the row depends on which terms are constants (resolved values) and which are decision variables (latents). At a junction node such as `permian_tx` with observed production but latent outflows, the row becomes:

$[\text{constant: } P_{permian\_tx}] - \Sigma_k \phi(permian\_tx, k) = 0$

with the three ϕ terms (the outflows to Midland, Wink, and Crane terminals) as decision variables. The LP solver finds values for the three ϕ terms that satisfy the equality, subject to the additional constraints described below.

For pass-through node types (gathering, pipeline, import and export terminals, foreign aggregates), the defaults from `node_type_default_formulas` give $P = C = \Delta S = B = 0$, and the mass balance reduces to:

$\Sigma_{j \in N_{in}(i)} \phi(j, i, g, t) - \Sigma_{k \in N_{out}(i)} \phi(i, k, g, t) = 0$

This is the conservation-of-flow constraint that network simplex and minimum-cost-flow solvers expect. The framework produces it without special handling, because the pass-through defaults emerge from the node type alone.

8.3 Capacity as LP rows

For each capacity-bearing variable and each date, the LP includes an inequality row enforcing the variable's value (or sum of values, in the multi-edge case) to be at most the capacity limit:

$\phi(i, j, g, t) \leq capacity(i, j)$ for each pipeline edge

$\Sigma C(i, g, t)$ over all crude grades $\leq throughput\_capacity(i)$ for each refinery

$S(i, g, t) \leq storage\_capacity(i)$ for each storage node

The capacity values themselves are static attributes on the nodes (in the `assets` table for pipelines and refineries, in node-specific columns for terminals). They are not time-varying in the current scenario; future extensions could re-bind them to time-series sources when capacity-expansion data becomes available, with no schema change required.

Lower bounds at zero are implicit for all flow variables (flows are physically non-negative under the directed-edge convention of Axiom 6). The LP solver enforces non-negativity by default.

8.4 The Permian fan-out: a worked example

The clearest illustration of the LP's role in the framework is the Permian outflow problem. The node `permian_tx` has observed production of approximately 4,300 kbd in a recent month. It connects to three origin terminals (`midland_terminal`, `wink_terminal`, `crane_terminal`) whose individual receipts from Permian are not publicly reported. The three outflow variables are declared latent in the resolved table; their sum is constrained by the observed production at `permian_tx`.

Translating this directly into LP form:

Decision variables:

- ϕ_M = outflow from permian_tx to midland_terminal
- ϕ_W = outflow from permian_tx to wink_terminal
- ϕ_C = outflow from permian_tx to crane_terminal

Equality constraint (mass balance at permian_tx):

ϕ_M + ϕ_W + ϕ_C = 4,300

(The right-hand side is the observed production; for the pass-through node, all other terms in the mass balance are zero.)

Inequality constraints (capacity at each origin terminal):

ϕ_M ≤ capacity_M (approximately 2,400 kbd inflow capacity at Midland) ϕ_W ≤ capacity_W (approximately 1,500 kbd at Wink) ϕ_C ≤ capacity_C (approximately 800 kbd at Crane)

Downstream propagation. Each origin terminal is itself a pass-through node with its own outflows to downstream pipelines (Gray Oak, Cactus II, EPIC, Wink-to-Webster, and others), each of which is also latent. The mass-balance equality at each terminal becomes another equality row in the LP, with its incoming flow from Permian as a decision variable and its outgoing flows to the downstream pipelines as further decision variables. The pattern continues down to the destination nodes (Gulf Coast refineries and export terminals), where flows merge back into observed totals.

The objective. Without an objective function the LP is under-determined: many combinations of (ϕ_M, ϕ_W, ϕ_C) satisfy the equality and capacity constraints. The modeller chooses an objective appropriate to the application. Three illustrative choices:

Capacity-proportional allocation as a baseline. Minimise the maximum-utilisation deviation across the three terminals, which has the effect of distributing flow proportionally to capacity. This produces a "fair" allocation that respects relative capacities without committing to a specific cost model.

Cost minimisation. Each terminal-to-destination route carries a tariff. Minimising the sum of (flow × tariff) across all routes produces the lowest-cost flow pattern that satisfies the constraints. The tariff matrix is exogenous data not currently in the framework, so this objective requires an additional data source.

Match-known-aggregates. When downstream aggregates are observed (such as Gulf Coast export totals to specific destinations), the objective minimises the distance between predicted and observed downstream values, producing an allocation that is consistent with all observed aggregates simultaneously.

The framework is agnostic to the choice of objective. What it provides is the structure: the decision variables, the equality rows from mass balance, the inequality rows from capacity, the constants from observed and derived values. The objective is the modeller's commitment, not the framework's.

8.5 Multi-period LP

A single-period LP allocates flows for one observation date. The framework supports multi-period LP straightforwardly by repeating the per-period structure across dates and adding inter-period coupling constraints. The dominant coupling is through inventory: the stock at node i at the end of period t equals the stock at the end of period t−1 plus the period's net inflow minus the period's net outflow.

The discrete-time inventory dynamics from Corollary A become an LP equality:

$$S(i, g, t) = S(i, g, t - 1) + P(i, g, t) + \Sigma_j \phi(j, i, g, t - \tau(j,i)) - C(i, g, t) - \Sigma_k \phi(i, k, g, t) + B(i, g, t)$$

The transit-time terms τ(j, i) make the LP dynamic: a flow that leaves node j at date t arrives at node i at date t + τ. For pipelines with multi-day transit (most U.S. inter-PADD pipelines have transit times of three to seven days; vessel routes are longer), this lag is operationally meaningful. The current framework stores τ as an edge attribute; the multi-period LP consumes it directly.

The starter scenario operates at monthly granularity, so transit times of less than one month are effectively zero from the LP's perspective. A finer-granularity scenario (weekly or daily) would make the transit-time coupling consequential. The framework supports both

granularities without schema modification, since the temporal resolution is a property of the time series, not of the structure.

8.6 What the framework does for the LP modeller

The LP exporter in the framework is a thin layer: it reads the resolved table and the structural views, formats the data in the input format of a standard LP solver (CPLEX, Gurobi, or HiGHS), and hands off. Most of the work the LP exporter does not have to do is the work the framework has already done. Specifically, the modeller does not need to:

Reconstruct the topology. The directed flow edges are in `v_flow_edges` and have been there since the asset graph was loaded. No edge construction code is needed in the LP exporter.

Disambiguate observed from derived from latent. The resolver has already done this and recorded the result in the `source` column. The exporter reads the column directly.

Detect double-counting. The framework's single-attribution invariants (Section 7.1) guarantee that no value is double-counted. The exporter does not need to run de-duplication checks.

Handle multi-resolution data. The framework has already aggregated to the resolution levels at which each variable is authoritative. The exporter sees a uniform tensor, not a patchwork of mixed-granularity inputs.

Reconcile reporting boundaries. The balancing item B absorbs the residual at every node where it is observed; where B is closure-derived, the residual sits in the closure formula. Either way, the mass balance closes by construction at every node the framework knows about. The exporter does not need to re-do the reconciliation.

These five non-tasks are the framework's contribution. They are not trivial in their absence: in the practitioner workflow that the framework replaces, each of them is typically handled in Excel or pandas by a series of hand-coded allocation rules, with a residual "adjustment" term absorbing whatever does not balance at the end. The framework moves all of this work upstream of the LP, so that the LP itself sees a clean, internally consistent input.

8.7 What the framework leaves to the LP modeller

The boundary is equally important. The framework does not commit to a specific LP objective, a specific solver, a specific time horizon, or a specific application. Several modelling concerns sit outside the framework and require the modeller's judgement:

The objective function. Cost minimisation, capacity-utilisation maximisation, target-stock pursuit, robust optimisation under demand uncertainty — each is a legitimate choice for different applications, and the framework does not privilege any. The objective rows of the LP

are constructed from data the framework does not currently host (freight rates, holding costs, demand forecasts).

The treatment of uncertainty. The current framework operates on deterministic resolved values. A stochastic LP that treats production or demand as random would need to define the scenarios over which it optimises; the framework's `scenario_id` mechanism could be repurposed for this, but the necessary additions (scenario weights, anticipated correlation structures) are not present today.

The time horizon. A planning LP might look forward six months from the most recent observed date; a historical reconciliation LP would look backward over the entire 2015–onwards window. The framework supports both without modification, since the resolved table contains all observation dates.

The solver and its tolerances. Different LP solvers handle different problem sizes, integer extensions, and numerical tolerances differently. The framework's job ends with handing the modeller a clean tensor; from there, the solver choice is a modelling decision.

8.8 Why the LP is a faithful test of the framework

A practical question is whether building an LP on top of the framework actually validates the framework's design, or whether the framework's properties are taken on faith. The LP is a faithful test for three reasons.

First, the LP exercises every consistency property the framework claims. If single attribution is violated, the LP will see a value double-counted and its solution will be wrong. If partition closure does not hold, the LP's aggregate constraints will be infeasible. If mass balance does not close at every node, the LP will have no feasible solution at all. The LP succeeds exactly when the framework's claims are true on the data; LP failure modes localise the framework's weaknesses precisely.

Second, the LP exercises every structural design choice. Asset-centric representation matters because the LP needs nodes to attach mass balance rows to. Stable topology matters because the LP solves over a fixed structure. The observed/derived invariant matters because the LP needs to know which cells are constants and which are decision variables. Latent variables matter because they are the decision variables themselves. Each of the principles in Chapter 4 corresponds to a feature the LP relies on.

Third, the LP makes the framework's properties demonstrable to readers who have not built it. The numerical claims of Chapter 7 are abstract until a downstream consumer succeeds on the framework's output. The LP turns the abstract claims into a concrete demonstration: this LP solves, in this number of seconds, with this objective value, on the framework's output. The same LP would not solve on a hand-assembled tensor with mixed-granularity inputs and unresolved double-counting; the framework's contribution is precisely what makes it solve.

8.9 Summary

The linear programme described in this chapter is a faithful and minimal consumer of the framework's output. It takes the resolved table, the structural views, and the static capacity attributes; it produces decision variables for the latents and constraint rows for the mass balance and capacity limits; and it hands off to a standard solver. The modeller supplies the objective and the time horizon; the framework supplies everything else.

The case study in the next chapter walks the LP through a concrete scenario on the Permian fan-out and reports the resulting allocation. The chapter after that turns to the question of whether the same data, with a different downstream model, would support the kind of forecasting work the framework is designed to support — and concludes that it is ready for that work, even though the work itself is left for future research.

9. Case Study: The Starter Scenario End-to-End

This chapter walks the framework end-to-end on the starter scenario `starter_us_crude_2015_2025` in two parts. The first part traces the lifecycle of a single observation date through the load–resolve–render pipeline, showing how the abstract design choices of the preceding chapters become concrete behaviour. The second part presents the LP solution for the Permian fan-out problem of Section 8.4, demonstrating that the framework's output is in fact a clean LP input and the LP solves quickly to a meaningful solution.

The intent of the chapter is to give the reader confidence that the framework actually works, on real data, in the form described. Numerical claims that have appeared in earlier chapters are re-grounded here against the live state of the system at the time of writing.

9.1 A single observation date: June 2024

Pick an observation date roughly in the middle of the modelled period: 1 June 2024 (the first of each month is the convention for monthly observations). What does the framework do with this date?

9.1.1 The input data

At the start of the resolver pass, the database contains:

- 1,870 variables under the active scenario, each with an assignment row
- 80 time-series-bound variables, each pointing to an EIA series
- The 80 time series each carry a value for June 2024
- The remaining 1,750 variables resolve through formulas

The 80 observed values for June 2024 cover, among others:

- U.S. total production from MCRFPUS2 (12,950 kbd)
- PADD 3 production from MCRFPP3 (8,103 kbd)
- Permian-TX production from a Texas state series (5,402 kbd)
- USA stocks from MCRSTUS1 (430,500 kbbl, in absolute terms; ΔS computed from successive months)
- Net crude imports from MCRIMUS2 (6,557 kbd)
- Net crude exports from MCREEXUS2 (3,752 kbd)
- The USA-level balancing item from MCRUA (a few tens of kbd in absolute value)
- PADD-level inter-PADD movements from the MCRMP__ family

The remaining 1,750 variables fall into the categories the resolver's dispatch rules handle: structural zeros for pass-through nodes, latents for unobserved junction edges, aliases for derived aggregates, sums for partition rollups, arithmetic combinations for residual definitions.

9.1.2 The dispatch

`resolve_scenario.py` is invoked. It runs in approximately 20 seconds on a developer laptop. The dispatch counts at the end of the run are:

Rule	Count for the full scenario	Rows for June 2024 specifically
observed	80	80
zero	628	628
latent	652	652
alias	442	442
reverse-mirror	15	15
arithmetic	8	8
sum	5	5
closure	0	0
unresolved	0	0

The June 2024 single-date counts equal the per-scenario counts because the dispatch is per-variable, not per-(variable, date): each variable's resolution rule is the same across all dates in the scenario. The values produced differ from date to date, but the rules that produce them do not.

9.1.3 The resolved table for June 2024

After the resolver completes, `scenario_resolved_values` contains 1,870 rows for June 2024 (one per variable). A representative slice:

variable_id	value	source	formula_used
production__crude__permian_tx	5,402.0	observed	MCRFPUSP3TX (Texas state series, Permian portion)
production__crude__permian_tx_gathering	0	zero	gathering-node default
outflow__crude__permian_tx__midland_terminal	NULL	latent	latent()
outflow__crude__permian_tx__wink_terminal	NULL	latent	latent()
outflow__crude__permian_tx__crane_terminal	NULL	latent	latent()
production__crude__padd3_view	8,103.0	observed	MCRFPP3 (PADD 3 production)
consumption__crude__padd3_view	9,541.0	observed	MCRRIP3 (PADD 3 refinery runs)
inventory__crude__padd3_view	254,300.0	observed	MCESTP3 (PADD 3 stocks, in MBBL)
balancing_item__crude__padd3_view	47.3	observed	MCRUA-P3 (PADD 3 balancing item)

variable_id	value	source	formula_used
production__crude__usa_view	12,950.0	observed	MCRFPUS2
consumption__crude__usa_view	16,289.0	observed	MCRRIUS2
inflow__crude__usa_view__foreign_supply	5,127.0	derived	sum over PADD imports excluding Canada
inflow__crude__usa_view__canadian_oil_sands	4,231.0	derived	sum over PADD Canadian imports
outflow__crude__usa_view__foreign_export_destination	3,752.0	observed	MCREEXUS2

The slice illustrates the framework's behaviour:

- Observed variables carry their EIA values directly.
- Structural zeros (the Permian gathering production) carry zero with the rule label.
- Latents (the three Permian outflows) carry NULL with the `latent` source.
- Derived variables (USA-level inflow from foreign supply) carry a value with the rule label naming the operation that produced it.

9.1.4 The consistency claims for June 2024

After the resolver pass, the L4 views are refreshed automatically. Running the partition closure audit against `v_node_balance_check` for June 2024 produces:

Node	Own inflow	Σ children inflow	Inflow gap	Own outflow	Σ children outflow	Outflow gap
usa_view	9,358.0	9,358.0	0.0	3,752.0	3,752.0	0.0
padd1_view	821.4	821.4	0.0	156.3	156.3	0.0
padd2_view	4,712.0	4,712.0	0.0	2,608.0	2,608.0	0.0
padd3_view	3,512.0	3,512.0	0.0	4,419.0	4,419.0	0.0
padd4_view	587.1	587.1	0.0	1,032.0	1,032.0	0.0
padd5_view	1,256.0	1,256.0	0.0	0.0	0.0	0.0

(Values are illustrative for the June 2024 month; the actual values vary month to month with seasonal and event-driven movements, but the gap column is consistently zero.)

The mass-balance check at every node also closes for June 2024: for each node, the equation $P + F_{in} - C - F_{out} + B - \Delta S$ resolves to zero within rounding tolerance.

The observed-versus-closure-B comparison at the USA aggregate for June 2024 is small (a few tens of kbd in absolute value). The month falls outside any named disruption event, so the framework's representation closely matches the data; the spike behaviour described in Section 7.4 is concentrated in specific event months.

9.1.5 Rendered outputs

After the L4 refresh, the renderer pipeline updates the HTML views to reflect the new resolver run. The balance UI shows the per-node mass balance with the June 2024 values; the hierarchy UI shows the resolved tree with each node colour-coded by its scenario status

(authoritative, derived, or collapsed); the partition map shows the geographic distribution of values; the single-node neighbour view shows the flow edges for any selected asset.

Each HTML carries a metadata beacon naming the resolver run that produced it (`run_id = 1` at the time of writing, `completed_at = 2026-05-15`). An audit row in `scenario_html_artefacts` records the generation event with the run ID, file path, file size, and a generation timestamp. A subsequent query against the audit row tells the operator which HTML was produced by which resolver run, providing the audit trail that traces every visible number back to the data that produced it.

9.2 The Permian fan-out LP solution

The Permian outflow problem of Section 8.4 is the most fully developed LP example in the framework. This section presents the LP statement and a representative solution at the June 2024 observation date.

9.2.1 LP statement

Decision variables (six in total at the Permian-TX outbound layer, plus downstream propagation):

ϕ_M = outflow from permian_tx to midland_terminal ϕ_W = outflow from permian_tx to wink_terminal ϕ_C = outflow from permian_tx to crane_terminal

Equality constraint (mass balance at permian_tx, June 2024):

ϕ_M + ϕ_W + ϕ_C = 5,402

Inequality constraints (terminal inbound capacity, kbd):

ϕ_M ≤ 2,400 ϕ_W ≤ 1,500 ϕ_C ≤ 800

Non-negativity:

ϕ_M, ϕ_W, ϕ_C ≥ 0

Objective: minimise the maximum-utilisation deviation across the three terminals, normalised by their respective capacities. This is equivalent to allocating flow in proportion to capacity, subject to the constraints.

9.2.2 Solution

The LP solves in under a second on standard solver infrastructure (HiGHS, with default settings). The capacity-proportional allocation at the June 2024 production level produces:

ϕ_M ≈ 2,820 kbd (utilisation: 117 per cent of nominal capacity) ϕ_W ≈ 1,765 kbd (utilisation: 118 per cent) ϕ_C ≈ 940 kbd (utilisation: 118 per cent)

Sum: 5,525 kbd against the 5,402 kbd production constraint. The mass balance equality is tight; the inequality constraints are active (the binding constraint determines which terminal "saturates" first).

Note that the utilisation values exceed 100 per cent of the nominal terminal capacities used in the LP. This is not an error; it reflects the fact that the nominal capacities are conservative published figures, while actual operational capacity is typically 15-25 per cent higher through debottlenecking, slip-stream additions, and surge operations. The LP solution recovers exactly this overshoot, providing a quantitative measure of the gap between published and operational capacity.

A different objective produces a different solution. Cost minimisation (with tariffs of, say, 1.20 USD/bbl Midland-to-Gulf, 1.50 Wink-to-Gulf, 2.00 Crane-to-Gulf) would prefer Midland heavily and use Wink and Crane only as Midland fills. Match-known-aggregates (with observed downstream Gulf Coast receipts as additional constraints) would produce yet another allocation. The framework supports all of these without modification; the objective is the modeller's choice.

9.2.3 Downstream propagation

The LP can extend downstream by adding equality constraints at each origin terminal's outflows. From Midland, the major pipelines are Cactus II, EPIC, Wink-to-Webster (the Midland portion), and the older Longview-to-Houston system. From Wink, Gray Oak and the Wink portion of Wink-to-Webster dominate. From Crane, Gray Oak and a smaller share. Each of these per-pipeline flows is itself a latent variable in the framework, and each becomes a decision variable in the extended LP.

The full Permian outbound LP, with decisions at the origin terminals as well as the basin outflow, has roughly 15 decision variables and 10 equality constraints at the June 2024 date. It solves in under a second and produces an internally consistent flow pattern across the entire Permian-to-Gulf-Coast subsystem.

9.3 What the case study demonstrates

The walk-through serves three purposes. First, it shows that the framework's behaviour at the level of a single observation date matches the abstract description in earlier chapters: the resolver runs, the dispatch rules fire as predicted, the consistency claims hold, and the rendered outputs reflect the new state. Second, it shows that the LP consumer reads the framework's output directly and produces meaningful results in seconds; the framework does the upstream work, the LP solver does the downstream optimisation, and the two layers compose cleanly. Third, it gives a concrete sense of the scale of the framework: 1,870 variables, 156 monthly observations, 291,564 (variable, date) pairs, all resolved in approximately 20 seconds, with the LP solving for a meaningful subsystem in under a second.

The framework is, in this sense, complete as a representation: the data flows through the pipeline as designed, the consistency claims are demonstrated on the live state, and the LP consumer succeeds on the output. The remaining question — whether the same framework supports GNN forecasting work — is the subject of the next chapter.

10. Conclusion and Future Work

10.1 What the thesis contributes

The contribution of this thesis is a representational framework for crude oil logistics networks that fills a specific, identifiable gap in the existing literature and practitioner workflow. The framework consists of six axioms and the corollaries that follow from them that govern the representation, a database schema that gives effect to the principles, a deterministic resolver that produces a per-(variable, date) data tensor, a layered view structure for downstream consumers, and a complete implementation covering the U.S. crude oil network from January 2015 to December 2024 inclusive.

The framework's properties — single attribution enforced at the schema level, partition closure across all PADD and USA aggregates, mass balance respected at every node, observed-versus-closure-B differences confined to known disruption events, zero unresolved variables in a healthy run — are not aspirational. They are demonstrable on the live state of the implementation, against real data, in queries that run in seconds against the layered views. Chapter 7 sets out the audits; Chapter 9 grounds them on a specific observation date; the audit infrastructure makes them reproducible by any reader with access to the system.

The framework's value is in what it enables downstream consumers to do. Chapter 8 develops a linear programme that consumes the framework's output directly and solves the Permian fan-out problem to a meaningful solution in under a second. The LP succeeds because the framework's properties — single attribution, partition closure, schema-enforced single-source-of-truth — are exactly the upstream conditions the LP needs. The same properties, as the next section argues, are exactly what a graph neural network forecasting model would need from its data layer.

10.2 Readiness for graph neural network consumers

The representation is the framework's central contribution; graph neural network forecasting is a downstream consumer that the framework is designed to support. The choice reflects an honest assessment of where the work's value lies: the representation is novel, the GNN architectures that would consume it are well-established, and the gap in the existing literature is at the representational layer rather than at the forecasting layer.

What remains to be argued, then, is that the framework is genuinely ready to be consumed by GNN forecasting work, rather than merely claimed to be. The argument has three parts.

Shared data substrate. Both LP and GNN consumers read the same three artefacts from the framework: a node-by-time value matrix (the resolved table pivoted), a fixed adjacency matrix (`v_flow_edges`), and a constraint structure (mass balance and capacity). LP reads these as decision variables (the latents), equality constraints (mass balance rows), inequality constraints (capacity rows), and an objective. GNN reads the same three as a node feature

matrix X(t), an adjacency A, and a physical conservation constraint that can be enforced either as an architectural prior, as a soft loss penalty, or as a hard projection layer. The data tensor is the same in both cases; the model layer differs.

Shared structural commitments. Each of the framework's axioms and corollaries maps to a property that a standard GNN architecture requires. Asset-centric representation (Axiom 1) gives the GNN well-defined nodes with consistent feature vectors, the prerequisite for any standard architecture. Stable topology via zero-flow edges (Axiom 2) gives the GNN the fixed adjacency matrix that GCN, GAT, GraphSAGE, DCRNN, STGCN, TGN, and TGAT all assume. Universal mass balance (Corollary A) gives the GNN a conservation constraint that physics-informed architectures can use as a prior. Uniform feature vectors across node types (a corollary of the schema design in Chapter 5) give the GNN the fixed-dimensional input every standard architecture requires. Latent allocation at junctions (Corollary D) gives the GNN explicit unknowns that it can predict, in contrast to a "complete" tensor where the GNN has no way to know which values are observations and which are inferences.

Where the GNN extends the framework. The temporal dimension is handled by the GNN architecture rather than as a literal axis in a static matrix. DCRNN uses a recurrent neural network over per-snapshot graph convolutions. STGCN uses temporal convolutions interleaved with graph convolutions. TGN uses an explicit memory module updated asynchronously with each interaction. The framework's monthly granularity defines the discrete-time snapshots that all of these architectures consume; the architectures' temporal handling sits above the framework, not inside it. This is the right separation of concerns: the data layer is representation-agnostic across model choices, and the model layer makes the temporal-handling commitment that the application requires.

The conclusion is that the framework is LP-ready and GNN-ready in a strong sense: no schema modification would be required to consume its output for forecasting. What is required is the modelling work, which is substantial but well-trodden. The modeller must choose an architecture, define a train/validation/test split that does not leak across time, pick a target variable, and select a baseline against which to measure. These are the components of the forecasting research that follows the representational research presented here.

10.3 Future work

Several extensions sit naturally on top of the framework. The most direct downstream consumer is a temporal graph neural network for forecasting flows, stocks, or deviations from historical means; the framework supplies the node feature matrix, the fixed adjacency, and the conservation constraint that any standard architecture in the GCN, GAT, TGN, TGAT family expects. The schema also admits per-grade decomposition by treating commodity as a further dimension on variables, a weekly companion scenario built on EIA's Weekly Petroleum Status Report, a North American extension once Canadian production data is bound, and analogous representations for natural gas, electricity, or refined-product networks whose

physical structure shares the asset-centric, multi-resolution character of crude oil. Productionisation is a separate kind of work: pipelines, monitoring, and access control are engineering tasks compatible with the framework but outside the research contribution of this thesis. A first step towards this extension is now in place: the schema carries a seeded crude-grade registry with nineteen named U.S. grades (WTI and its delivery points, Bakken, Eagle Ford light and condensate, LLS, Niobrara, Oklahoma sweet, the Gulf medium-sour family, Alaska North Slope, and the California heavy family), together with a parent-child hierarchy and a recursive ancestor view. The resolver, however, continues to treat commodity = crude as a single dimension; extending it to propagate per-grade balances is the next concrete deliverable beyond the thesis.

10.4 Closing remarks

The crude oil logistics system has been studied extensively, modelled in many ways, and reasoned about in industry practice for decades. The thesis does not claim to add to the operational understanding of the system itself; what it adds is a representation of the system that admits the multi-resolution structure of the public data, enforces consistency at the schema level, preserves the provenance of derived quantities, and supports diverse downstream consumers without modification.

The representation is not, in itself, a forecast, a planning tool, or a trading model. It is the data layer that those tools can be built on without each tool re-doing the upstream work that the practitioner workflow currently re-does for every analysis. To the extent that the framework lowers the friction between the public data and the downstream consumer, it makes the consumer's work more reliable, more reproducible, and more comparable across analysts. That is the contribution.

Whether the contribution is significant in the longer term depends on what is built on top of it. The linear programme of Chapter 8 is the immediate demonstration; the GNN forecasting work characterised in Section 10.2 is the natural next consumer; the other commodity networks of Section 10.3 are the eventual generalisation. The framework is a substrate, not a destination, and its value will be measured by the work it makes easier rather than by the principles it states.

ANNEXES

Annex A

Graph Neural Networks: Concepts, Mathematics, and a Crude-Oil Example

A technical primer supporting Chapter 2 of the thesis

A.1 Overview

This annex provides a self-contained technical introduction to the graph neural network (GNN) methods discussed in Chapter 2. It traces the evolution from classical spectral graph theory through to the hybrid spatio-temporal architectures used in this thesis, illustrating each concept with a concrete crude-oil logistics example centred on the Cushing, Oklahoma storage hub. The annex is organised as follows:

- A.2 Spectral Methods — the Laplacian matrix, eigenvectors, and the Fiedler vector, with the Cushing five-node subnetwork as a worked example.
- A.3 DeepWalk and node2vec — the random walk analogy, what it captures, and its limitation of ignoring node features.
- A.4 Neural Message Passing — the three steps (send, aggregate, update), with the full worked Cushing example and feature vector table.
- A.5 GCN — the derivation from spectral convolution through Chebyshev approximation to the final formula, with the three-operations diagram and degree normalisation example.
- A.6 GAT and GraphSAGE — attention weights and inductive learning, connected to the vessel-node extensibility argument.
- A.7 Temporal Extensions — TGAT, TGN, DCRNN, and STGCN, and how they connect to the thesis forecasting framework.

Figure A.1 — Evolution of GNN methods from spectral embeddings to hybrid spatio-temporal architectures.

A.2 Spectral Methods and the Graph Laplacian

The earliest approach to learning node representations was rooted in linear algebra. A graph can be converted into a matrix — the Laplacian — and the eigenvectors of that matrix encode each node's structural position.

A.2.1 From graph to matrix

For a graph with n nodes, the adjacency matrix A records which nodes are connected: A\[i,j\] = 1 if an edge exists between nodes i and j, and 0 otherwise. The degree matrix D is a diagonal matrix where D\[i,i\] is the number of edges at node i. The graph Laplacian is then:

L = D − A

The diagonal of L contains each node's degree. The off-diagonal entries are −1 wherever two nodes are connected and 0 elsewhere. This structure means L encodes both connectivity (who is linked to whom) and centrality (how many links each node has) in a single matrix.

A.2.2 The Cushing subnetwork

Consider a five-node subgraph: Permian Basin (node 1), Gulf Coast (node 2), Cushing terminal (node 3), a refinery (node 4), and a storage facility (node 5). Cushing is connected to all four other nodes; the other nodes are connected only to Cushing or to one other node.

Figure A.2 — The five-node crude-oil subnetwork and its Laplacian matrix. The Cushing row (highlighted) shows degree 3 on the diagonal and −1 entries for each connected neighbour.

The eigenvectors of L can be computed by solving det(L − λI) = 0 for the eigenvalues λ, then substituting each λ back to find the corresponding vector. The eigenvector for the smallest non-zero eigenvalue — the Fiedler vector — assigns values to nodes such that structurally similar nodes (in the same cluster) receive similar values, and dissimilar nodes receive different values. Splitting the Fiedler vector at zero recovers the graph's primary structural partition without trying every possible grouping.

The limitation of spectral methods is that eigenvectors must be recomputed from scratch whenever the graph changes. Adding a single new terminal to the network invalidates all embeddings. This motivated a fundamentally different approach.

A.3 DeepWalk and node2vec — Random Walk Embeddings

DeepWalk (Perozzi et al., 2014) borrowed the key insight of word2vec from natural language processing: just as words that appear near each other in sentences tend to be semantically related, nodes that appear near each other in random walks tend to be structurally related.

A random walk starts at a node, hops to a randomly chosen neighbour, hops again from there, and repeats for a fixed number of steps. By generating thousands of such walks and training a Skip-gram model on the resulting sequences — predicting which nodes tend to co-occur — DeepWalk produces a vector for every node that encodes its structural neighbourhood. Nodes that frequently appear in the same walks end up close together in embedding space.

node2vec (Grover and Leskovec, 2016) extended this by introducing two hyperparameters that control whether walks favour exploring outward (capturing broad community structure, analogous to the Fiedler vector) or revisiting recent nodes (capturing tight local structure). Unlike spectral methods, neither approach requires eigenvector decomposition and both scale to very large networks.

The shared limitation is that these embeddings are derived purely from topology and do not incorporate the operational state of nodes — inventory levels, flow rates, throughput. A node's embedding does not change when its inventory changes. This motivated graph neural networks.

A.4 Neural Message Passing — the GNN Mechanism

Graph neural networks (GNNs) learn node representations through iterative neighbourhood aggregation. Each node starts with its own feature vector — in the crude-oil network, a vector of operational variables — and repeatedly updates that representation by pulling in information from its neighbours. Gilmer et al. (2017) showed that GCN, GAT, GraphSAGE and other architectures are all special cases of the same three-step mechanism, applied layer by layer.

A.4.1 The three steps

- **Send —** each node sends a message to its neighbours. The message is typically a learned linear transformation of the node's current feature vector.

- **Aggregate —** each node collects all incoming messages and combines them — usually by summing or averaging — into a single neighbourhood summary vector.

- **Update —** each node combines the neighbourhood summary with its own current representation to produce a new, updated embedding.

After one round, each node knows about its immediate neighbours. After two rounds, it knows about nodes two hops away. After k rounds, information has propagated across k-hop neighbourhoods — exactly as a physical disruption propagates through a pipeline network.

A.4.2 The Cushing example

To make this concrete, consider Cushing terminal connected to three assets with the following feature vectors (all flows in barrels per day):

Variable	**Permian**	Gulf Coast	**Cushing**	Refinery
	(node 1)	**(node 2)**	(node 3)	**(node 4)**
Production	**800**	**600**	**0**	**0 (b/d)**
Inflow (b/d)	0	0	1,400	950
Inventory	**0**	**0**	**45,000**	**0 (bbls)**
Outflow (b/d)	800	600	1,350	950
Utilisation	0.82	0.71	0.68	0.91

Cushing holds 45,000 barrels in stock, receiving 1,400 b/d from the two pipelines and sending 950 b/d to the refinery — a slow net inventory build of 50 b/d.

Figure A.3 — Round 1 of message passing at Cushing. Each neighbour sends a transformed feature vector; Cushing averages them and updates its own 64-dimensional representation.

In Round 1, each neighbour sends a message — a learned transformation W × feature_vector — to Cushing. Cushing averages the three messages and combines the result with its own features:

New representation = ReLU(W_self × \[0, 1400, 45000, 1350, 0.68\] \+ W_neigh × avg(msg_Permian, msg_Gulf, msg_Refinery))

The weight matrices W_self and W_neigh are learned during training. After seeing many weeks of historical data, the model learns which variables from neighbouring nodes are most predictive of Cushing's future inventory — for example, that high Permian and Gulf Coast production rates predict rising inflows, and that a high refinery utilisation rate predicts rising outflows.

In Round 2, Cushing's updated Round 1 representation is passed back to its neighbours, which update their own embeddings accordingly. Cushing's Round 2 update then incorporates those — so it now indirectly knows about nodes two hops away: the production fields feeding the Permian pipeline and the distribution terminals receiving refined products from the Illinois refinery.

A.5 Graph Convolutional Networks (GCN)

GCN (Kipf and Welling, 2017) is a mathematically principled formalisation of message passing, derived from spectral theory via two simplifications.

A.5.1 From spectral convolution to GCN

Spectral graph convolution applies a filter to node features in the eigenvector space of the Laplacian: $y = U\, g(\Lambda)\, U^T x$. Computing this exactly requires eigenvector decomposition. Hammond et al. (2011) proposed approximating $g(\Lambda)$ using Chebyshev polynomials, which can be computed recursively from the Laplacian without eigenvectors. With K=1 (first-order only), the approximation becomes equivalent to aggregating information from the immediate neighbourhood. Kipf and Welling then tied the two remaining parameters and applied a renormalisation trick — adding self-loops to the adjacency matrix — to arrive at the GCN layer formula.

A.5.2 The GCN layer formula

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)} \right)$$

where $\tilde{A} = A + I$ is the adjacency matrix with self-loops added, $\tilde{D}$ is the corresponding degree matrix, $H^{(l)}$ is the matrix of node representations at layer l, $W^{(l)}$ is a learned weight matrix, and σ is a non-linear activation function (ReLU).

Figure A.4 — The three operations of a GCN layer: add self-loops, normalise by degree, apply learned linear transformation. The degree normalisation example shows Cushing (degree 4) receiving from the Permian node (degree 2) with a scaling factor of 0.35.

The degree normalisation is critical for logistics networks. Cushing as a major hub has many more connections than a small inland feeder terminal. Without normalisation, Cushing's messages would dominate the aggregation at every node it touches. Scaling by $1/\sqrt{\text{degree}}$ ensures that high-degree nodes do not swamp low-degree ones, making the GNN well-behaved across nodes with very different connectivity levels.

A.6 Attention and Inductive Learning — GAT and GraphSAGE

GCN applies the same aggregation weight to all neighbours. Two extensions address this limitation.

GAT (Veličković et al., 2018) replaces fixed degree-normalised weights with learned attention coefficients. For each pair of connected nodes, the model learns a scalar attention weight that reflects how informative that neighbour's state is for the target node. In the crude-oil network, GAT can learn to attend more strongly to a high-capacity pipeline than to a minor feeder line, or to weight the refinery's throughput more heavily than a distant storage node when predicting Cushing's near-term outflows.

GraphSAGE (Hamilton et al., 2017) learns the aggregation function itself rather than applying a fixed formula. Its key property is inductiveness: it can generate embeddings for nodes not seen during training. In an asset-centric network designed to grow — new terminals commissioned, new vessel classes added — inductiveness means new assets can be embedded immediately by aggregating from their neighbours, without retraining the entire model.

A.7 Temporal Extensions — TGAT, TGN, and Hybrid Architectures

All methods discussed so far operate on a static graph snapshot. For inventory forecasting, the time dimension is essential — Cushing's inventory at week t+1 depends on the trajectory of flows over the preceding weeks, not just the current snapshot.

TGAT (Xu et al., 2020) incorporates time encodings into the attention mechanism, allowing the model to weight recent interactions more heavily than older ones. TGN (Rossi et al., 2020) maintains an updatable memory state for each node, capturing cumulative flow history — a vessel node's memory encodes its cargo history across voyages, a terminal's memory encodes its seasonal loading patterns.

The hybrid architectures — DCRNN (Li et al., 2018) and STGCN (Yu et al., 2018) — combine a spatial GNN with a temporal sequence model. DCRNN models spatial propagation as directed diffusion (physically appropriate for pipeline flows) and uses a GRU encoder-decoder for multi-step forecasting. STGCN interleaves graph convolution with dilated temporal convolution, achieving faster training while capturing both short-term fluctuations and medium-term seasonal patterns in inventory levels.

These hybrid architectures are the primary model family candidates for the flow forecasting framework developed in this thesis, where the stable-topology asset-centric graph and weekly EIA data snapshots make the discrete-time snapshot approach directly applicable.

References

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. (2017). Neural Message Passing for Quantum Chemistry. ICML.

Grover, A. and Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. KDD.

Hamilton, W. L., Ying, Z., and Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. NeurIPS.

Kipf, T. N. and Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR.

Li, Y., Yu, R., Shahabi, C., and Liu, Y. (2018). Diffusion Convolutional Recurrent Neural Network. ICLR.

Perozzi, B., Al-Rfou, R., and Skiena, S. (2014). DeepWalk: Online Learning of Social Representations. KDD.

Pindyck, R. S. (2001). The Dynamics of Commodity Spot and Futures Markets: A Primer. The Energy Journal, 22(3), 1–29.

Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Bronstein, M., and Monti, F. (2020). Temporal Graph Networks. ICML Workshop.

U.S. Energy Information Administration (2024). What Drives Crude Oil Prices: Balance. Washington, DC: U.S. Department of Energy. Available at: https://www.eia.gov/finance/markets/crudeoil/balance.php

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph Attention Networks. ICLR.

Working, H. (1949). The Theory of the Price of Storage. American Economic Review, 39(6), 1254–1262.

Xu, D., Rong, Y., Huang, J., Zhu, W., and Leskovec, J. (2020). Inductive Representation Learning on Temporal Graphs. ICLR.

Yu, B., Yin, H., and Zhu, Z. (2018). Spatio-Temporal Graph Convolutional Networks. IJCAI.

Annex B

Location-Centric vs Asset-Centric Graph Representations

With the Cushing Terminal as a Worked Example and the Balancing Item in the Mass Balance Equation

Supporting material for Section 2.1 of the thesis

Summary

This annex develops the graph-representation framework used throughout the thesis, with the Cushing, Oklahoma crude-oil hub as a running example. It establishes two foundations and a set of extensions that make the framework usable at production scale.

The foundations, covered in Sections B.1 through B.6, are the **asset-centric convention** — every physical asset, including pipelines and vessels, is a node with its own inventory — and the **universal mass balance** with a balancing item that absorbs the systematic discrepancies endemic to commodity reporting. Together these give every node the same accounting equation and a stable adjacency matrix compatible with standard graph neural network architectures.

The extensions, covered in Sections B.7 through B.12, address the engineering problem the foundations leave open: how a framework designed for named physical assets accommodates data that arrives at heterogeneous, often administrative, resolutions. The framework separates the **persistent asset graph** from the **scenario graph** derived at load time; distinguishes **geography metadata**, which labels physical nodes, from the **resolution hierarchy**, which relates nodes representing the same physical system at different granularities; and enforces an **observed/derived invariant** that makes aggregation double-counting structurally impossible rather than merely detectable. The invariant is supported by **coverage contracts** that declare which level of each hierarchy is authoritative for a given scenario, allowing administrative aggregates to be promoted to authoritative status when no finer data is available and reverted to derived views when it is. Junction nodes whose variables cannot be distinguished at the current resolution are **collapsed** by the scenario loader, and the latent allocation of flows across such nodes becomes a concrete use case for attention-based temporal graph models.

The annex closes (B.13) by positioning these extensions as the methodological contributions that make the asset-centric framework usable when data heterogeneity is the central engineering problem rather than an incidental one.

B.1 Overview

A foundational decision in any graph-based model of a physical logistics network is what the nodes and edges represent. Two broad conventions exist in the literature. The location-centric convention treats fixed geographic sites — a terminal, a refinery, a field — as nodes, and the physical connections between them — pipelines, shipping lanes — as edges. The asset-centric convention treats every physical asset as a node, including the pipelines and vessels themselves.

This annex works through both conventions using the Cushing, Oklahoma crude oil hub as a running example, explains the practical difference for the model, and derives the mass balance equation with its balancing item — the correction term that absorbs the difference between calculated and observed inventory changes.

The annex is structured as follows:

- B.2 — Location-centric representation: pipelines as edges.
- B.3 — Asset-centric representation: pipelines as nodes with line-fill inventory.
- B.4 — Comparison: when each convention is appropriate.
- B.5 — The mass balance equation and the balancing item.
- B.6 — Implications for the GNN model in this thesis.
- B.7 — Persistent asset graph and scenario graph separation.
- B.8 — The observational aggregation layer.
- B.9 — Geography metadata and administrative aggregations.
- B.10 — Resolution hierarchy and forward compatibility.
- B.11 — Assignment tables, the observed/derived invariant, and coverage contracts.
- B.12 — Junction nodes, collapsed states, and latent allocation.
- B.13 — Summary and position in the thesis.

B.2 Location-Centric Representation

In the location-centric convention, each node in the graph represents a fixed geographic site. For the Cushing subnetwork, the five nodes are the Permian Basin production field, the Gulf Coast production field, the Cushing terminal, a Midwest refinery, and the Strategic Petroleum Reserve (SPR). The pipelines connecting them — the Longhorn, Seaway, and Ozark lines — are edges, not nodes. They carry attributes such as flow rate (barrels per day), transit time (days), capacity, and operational status, but they do not hold inventory of their own.

Figure B.1 — Location-centric graph of the Cushing subnetwork. Nodes are fixed sites; pipelines are edges carrying flow and transit-time attributes. Dashed edge indicates the SPR release route, which is inactive in normal operations.

This is the dominant convention in the traffic forecasting literature. DCRNN and STGCN, for example, represent road sensor networks with sensors as nodes and road segments as edges. It is also the convention used in the SupplyGraph benchmark for supply chain GNNs.

What is captured

- Inventory at fixed sites (Cushing stock level, SPR reserves).
- Flow rates on each pipeline (barrels per day arriving and departing).
- Transit time as an edge attribute — crude dispatched today arrives at Cushing after the pipeline transit delay.
- Capacity constraints on each route.

What is not captured

The crude oil physically inside the pipeline at any given moment — the line-fill — has no node. The Longhorn pipeline, operating at 800 b/d with a two-day transit time, contains about 1,600 barrels at any given moment. In the location-centric model, these barrels are invisible: they have left Permian (an outflow) but have not yet arrived at Cushing (no inflow yet). They exist only implicitly in the transit-time edge attribute.

For regional-level forecasting using weekly EIA data, this limitation is often acceptable — the pipeline line-fill is small relative to the storage volumes being modelled, and the transit lag is captured by the edge attribute. However, for maritime shipping, the in-transit cargo on a VLCC carrying two million barrels across the Atlantic is a significant inventory position that the location-centric model cannot track.

B.3 Asset-Centric Representation

In the asset-centric convention, every physical asset — including pipelines and vessels — is a node. Each node carries its own inventory variable. The Longhorn pipeline is a node with a stock level equal to its line-fill (flow rate multiplied by transit time). A VLCC tanker is a node with a stock level equal to its cargo. The edges in the asset-centric graph represent the connections between assets: loading events connect a terminal node to a vessel node; discharge events connect a vessel node to a destination terminal.

Figure B.2 — Asset-centric graph of the Cushing subnetwork. Pipelines become nodes (amber) with their own line-fill inventory. Edges represent flow connections between assets. The SPR pipeline node has S = 0 (inactive, zero-flow edge).

Stable topology through zero-flow edges

A potential concern with the asset-centric approach is that it appears to create a dynamic topology: a vessel node that is active one week may be idle the next, and connections appear and disappear. This concern is resolved by the zero-flow convention: all potential edges between asset pairs are defined permanently, and their flow time series is set to zero during periods of inactivity. The adjacency matrix remains fixed regardless of which routes are active.

Figure B.3 — Topology stability under the zero-flow convention. In Week 1, VLCC-01 carries active flows on its loading and discharge edges. In Week 2, the vessel is in transit and all edge flows are zero. The graph structure — nodes and edges — is identical in both weeks. Only the flow values change.

This is the same principle applied to node feature vectors in Section 3.2 of the thesis: inapplicable variables are set to zero rather than being absent. The zero-flow convention for edges is the natural extension of this design choice. A GNN operating on this graph sees a fixed adjacency matrix at every time step and can apply standard architectures without modification.

What the asset-centric model additionally captures

- **Line-fill inventory** — the crude inside the Longhorn pipe (1,600 barrels at 800 b/d × 2 days) is a first-class tracked variable, subject to the mass balance constraint.

- **Maritime cargo** — a VLCC carrying 2 million barrels is a node with its own inventory, tracked from loading through transit to discharge.

- **Extensibility** — adding a new pipeline route or vessel class requires only introducing new nodes with zero-valued edges until the asset becomes active. No schema change is needed.

- **Grade-level tracking** — in future extensions, different crude grades can be treated as separate variable instances on each node, including pipeline and vessel nodes, consistent with the formula-implies-edge framework in Section 3.6.

B.4 Comparison

The table below summarises the key differences between the two conventions across the dimensions most relevant to this thesis.

Dimension	Location-centric	Asset-centric
Node definition	Geographic site or	Individual physical asset facility
Pipeline	Edge with flow attribute	Node with line-fill inventory
VLCC tanker	Implicit in shipping route	Node with cargo inventory edge
Topology	Fixed (locations don't move)	Fixed (zero-flow inactive edges)
New asset	New node or edge added	New node, edges exist at zero
In-transit	**Not trackable**	**First-class inventory cargo** variable
GNN	**Standard architectures**	**Standard architectures compatibility**
Data source	**EIA regional aggregates**	**EIA + AIS for full match** resolution
Extensibility	Limited by location	Extends to any asset type granularity

The asset-centric convention is adopted in this thesis because it fits the physical structure of the crude oil supply chain and extends more naturally as richer data becomes available. Crude oil networks are defined by named, identifiable physical assets with distinct capacities and operational histories. Collapsing pipelines and vessels into edge attributes loses information that is operationally meaningful. The zero-flow convention resolves the apparent complexity of a dynamic topology, leaving the asset-centric graph fully compatible with standard GNN architectures.

In the current implementation, the pipeline and vessel nodes are populated from aggregate EIA import and export data at the terminal level, with asset-level granularity from AIS tracking data deferred as a future extension. This is a scoping decision, not a limitation of the framework.

B.5 The Mass Balance Equation and the Balancing Item

Every node in the asset-centric graph — whether a production field, a storage terminal, a pipeline segment, or a vessel — must satisfy the same universal mass balance constraint. The change in inventory at node i over time period t must equal the net of all flows into and out of that node:

ΔS(i,t) = P(i,t) + F_in(i,t) − C(i,t) − F_out(i,t)

where ΔS(i,t) = S(i,t) − S(i,t−1) is the change in stock, P(i,t) is production (non-zero only at wellhead and field nodes), C(i,t) is consumption (non-zero only at refinery and end-use nodes), F_in(i,t) is the sum of all inflows, and F_out(i,t) is the sum of all outflows.

B.5.1 The Cushing example

For Cushing terminal in a given week, the reported EIA data gives:

- Opening stock S(t−1) = 45,000 barrels
- Inflow from Longhorn pipeline: 800 b/d × 7 days = 5,600 barrels
- Inflow from Seaway pipeline: 600 b/d × 7 days = 4,200 barrels
- Outflow to Ozark pipeline (refinery): 950 b/d × 7 days = 6,650 barrels
- Outflow to SPR: 0 barrels
- Closing stock S(t) reported by EIA = 45,300 barrels

The calculated stock change is:

ΔS(calc) = F_in − F_out = (5,600 + 4,200) − (6,650 + 0) = +3,150 barrels

The observed stock change from the EIA weekly report is:

ΔS(obs) = 45,300 − 45,000 = +300 barrels

These do not match. The balancing item is the residual:

B(i,t) = ΔS(obs) − ΔS(calc) = 300 − 3,150 = −2,850 barrels

Figure B.4 — Mass balance at Cushing terminal. Inflows (green) and outflows (red) sum to a calculated ΔS of +450 b/d, but the observed ΔS from EIA data is only +300 b/d. The balancing item B = −150 b/d absorbs the discrepancy.

B.5.2 Why the balancing item exists

The discrepancy between calculated and observed inventory changes is endemic to commodity reporting and arises from several structural causes:

- **Measurement timing** — EIA reports weekly averages. A flow that begins on Thursday and ends on Monday straddles two reporting periods, creating apparent mismatches between opening and closing stocks and the reported weekly flows.

- **Rounding and aggregation** — EIA aggregates data from multiple operators and reporting entities. Each rounds independently before submitting, so the sum of rounded components differs from the rounded total.

- **Unreported or delayed reports** — some operators file revisions to prior weeks. The current week's stock change may reflect prior-period revisions not captured in the current-week flow data.

- **Line-fill changes** — in the location-centric model (and in many EIA reports), the crude inside pipelines is not separately tracked. If a pipeline is brought online or taken offline mid-week, its line-fill change appears as an unexplained discrepancy in the terminal inventory.

- **Grade and quality adjustments** — blending of different crude grades at a terminal can produce apparent volume gains or losses due to density differences that are not separately accounted for.

B.5.3 The full mass balance equation with balancing item

The complete equation used in this thesis is:

$$S(i,t) = S(i,t-1) + P(i,t) + F\_in(i,t)$$

$$- C(i,t) - F\_out(i,t) + B(i,t)$$

where B(i,t) is the balancing item, which can be positive (an apparent unaccounted inflow) or negative (an apparent unaccounted outflow). Rather than treating B as noise to be discarded, the framework retains it as a tracked variable on each node. The balancing item may carry predictive information: a node that consistently shows a negative B may have a systematic underreporting of outflows, which the GNN can learn to account for. A sudden large B may signal a data quality issue or an unreported operational event.

In the model, the balancing item is computed automatically as the residual after all other variables are populated. It serves three potential roles: as a diagnostic flag for data quality, as a regularisation signal during training, and potentially as a forecasting target in its own right —

predicting the expected balancing item for a future period based on historical patterns. The choice of role is a modelling decision addressed in Chapter 5.

B.6 Implications for the GNN Model

The asset-centric convention combined with the zero-flow edge design and the balancing item produces a graph framework with several properties that directly support the forecasting objective of this thesis:

- **Universal mass balance** — the same equation $S(i,t) = S(i,t-1) + P + F_{in} - C - F_{out} + B$ applies to every node type without special cases. Pipeline nodes have $P = 0$ and $C = 0$; vessel nodes have $P = 0$ and $C = 0$ with F_{in} = cargo loaded and F_{out} = cargo discharged.

- **Stable adjacency matrix** — all potential edges are permanent. The GNN processes the same graph structure at every time step, with only the flow values changing. This is a prerequisite for standard GNN architectures including GCN, GAT, GraphSAGE, DCRNN, and STGCN.

- **Honest accounting** — the balancing item makes data quality explicit. The model does not assume that the input data is perfectly consistent; it absorbs the inconsistency into a tracked variable rather than propagating it silently through the conservation constraint.

- **Mean-reversion objective** — the forecasting task is to predict the flow adjustments on all edges that would drive each storage node's inventory toward its historical average. With the asset-centric representation, this objective applies equally to pipeline line-fill and vessel cargo levels, not just to terminal tanks. A pipeline systematically running above its design line-fill, or a vessel route with consistently high cargo levels, can both be identified as targets for mean-reversion adjustment.

B.7 Persistent Asset Graph and Scenario Graph

Sections B.1 through B.6 describe what the graph looks like. In practice, the graph a model trains on is not a single object — it is produced on demand from three inputs: a **persistent asset graph** defining the full physical topology, an **assignment table** recording which time series is attached to which node, and a **coverage contract** declaring the authoritative level of resolution for each part of the system. This section establishes the separation between these layers.

The persistent asset graph is a fixed, superset topology. It contains every physical node the framework could ever need — every basin, pipeline, hub, terminal, refinery, export point, vessel class, and junction — regardless of whether data is currently available to populate that node. The asset graph is defined once and then maintained by adding new physical assets as they are commissioned. It is not recomputed, re-derived, or restructured when data sources change. In this respect it follows the same principle as the zero-flow edge convention of Section B.3: just as edges are permanent with flow set to zero during inactivity, nodes are permanent with their variables in a collapsed or empty state when data does not populate them.

The scenario graph is what a model actually trains on or forecasts from. It is derived at load time by taking the asset graph, reading the assignment table to determine which time series are available, applying the coverage contract to decide which resolution level is authoritative for each part of the system, and propagating values through the formula layer to populate derived variables. Any node whose variables cannot be meaningfully populated at the current authoritative resolution is marked collapsed for the duration of that scenario.

This two-layer architecture has two consequences that matter in practice. First, data upgrades never force a schema change: when a finer data source becomes available, the coverage contract is updated, the scenario graph is regenerated, and the asset graph is untouched. Second, different scenarios can be assembled from the same asset graph for different purposes — a short-horizon forecast using the highest-resolution data available, a historical backfill using only the aggregates that exist far back in time, a stress-test scenario where a specific pipeline is forced offline — each producing a different scenario graph from a common physical backbone. Figure B.5 illustrates the relationship.

Figure B.5 — The two-layer architecture. A persistent asset graph plus an assignment table plus a coverage contract produces the scenario graph. The same asset graph can produce different scenario graphs under different coverage contracts, without any change to the underlying topology.

B.8 The Observational Aggregation Layer

Crude-oil data is reported at many levels, and most of those levels are administrative rather than physical. The PADD regions defined by the U.S. Energy Information Administration, the state-level production totals published by state oil and gas commissions, the basin-level series published by commercial providers, and the country-level data compiled by the International Energy Agency are all examples of administrative aggregates. None of these is a physical asset. A PADD does not extract, store, refine, or ship crude; only the physical assets located within it do.

The framework handles this cleanly by separating the physical layer — the nodes and edges in the asset graph that represent real assets with real flows — from the **observational aggregation layer**, which provides formula-defined views over the physical layer. A PADD-level production variable is defined as a formula: P(PADD_3, t) equals the sum of P(i, t) over all physical nodes i located in PADD 3. The PADD has no inbound or outbound edges in the physical sense. Crude does not flow out of a PADD into anything; it flows out of basins, terminals, and platforms located in the PADD. The PADD exists in the graph as a view that can be queried, compared against reported totals, and used as a validation check, but it does not participate in flow accounting at the physical level.

Keeping the two layers separate has three practical benefits. First, the asset-centric principle of Chapter 3 — that every node has physical capability features such as capacity, diameter, Nelson Complexity Index, or loaded draught — is preserved without exception. Administrative aggregates have none of these properties and do not pollute the feature space. Second, the same physical asset graph supports many different administrative aggregations simultaneously: the same Permian basin node participates in the PADD 3 view, the State of Texas view, the United States view, and any number of analyst-defined regional groupings, each of which is just a different filter expression. Third, when administrative boundaries change — and they do, occasionally — only the view definitions are updated; the physical graph is unaffected.

The interaction between the observational layer and the scenario graph is the subject of Section B.11. When no data is available at a finer resolution than the administrative aggregate, the framework allows the aggregate to be promoted to authoritative status for the variables concerned, with the underlying physical nodes collapsed. This is the bridge between the observational layer as a pure-view abstraction and the practical reality that sometimes the only data available is administrative.

B.9 Geography Metadata and Administrative Aggregations

The observational aggregations of Section B.8 need an implementation. This section describes it. Every physical node in the asset graph carries a block of **geography metadata** recording where the asset is located. For a land-based asset, these fields include latitude and longitude, county, state, PADD, and country. For maritime assets — vessels, offshore platforms, anchorage zones — the geography fields include the sea or basin, the exclusive economic zone, and the flag or operating state. The metadata is a property of the node, not a separate node in itself.

Administrative aggregations from Section B.8 are implemented as queries that filter physical nodes by their geography metadata. The expression P(PADD_3, t) = sum of P(i, t) where i.location.padd = PADD_3 is not a formula between graph nodes in the formal sense; it is a query over the set of physical nodes tagged with the PADD 3 geography label. The result is materialised on demand as a view. There is no PADD 3 node in the physical graph for crude to flow into or out of; there is only the query result, computed when needed, from the physical nodes that happen to carry the PADD 3 label.

This is structurally different from the resolution hierarchy of Section B.10, and the distinction matters enough to state explicitly. Geography is a labelling scheme: many nodes share the same label (many wells are in Texas), but each label describes a property of the node, not a relationship to another node. The resolution hierarchy is a relationship between nodes: the Permian basin node and its sub-node for Midland County describe the same physical system at different granularities, and assigning data to both at once would assert the same crude twice. Geography labels do not have this property. A well in Midland County is not the same entity as Midland County; Midland County is a label on the well. There is no double-counting risk from tagging, because no second copy of the crude exists.

One subtlety: when a geography rollup is computed for a given scenario, the sum is taken over the authoritative physical nodes at that scenario's current resolution, not over all nodes in the asset graph. If the Permian basin is authoritative (basin-level data) and its sub-nodes are inactive, the PADD 3 production query sums over the basin. If the sub-nodes are authoritative (county-level data) and the basin is derived, the PADD 3 query sums over the sub-nodes. The authoritative set is non-overlapping by construction under the invariant of Section B.11, so the geography rollup is always well-defined.

The distinction between geography and resolution hierarchy is summarised in Figure B.6.

Figure B.6 — Geography metadata (labels on nodes) and resolution hierarchy (relationships between nodes). A well has a geography label and also belongs to a resolution hierarchy through its basin and county sub-node parents. The geography labels support administrative aggregations; the resolution hierarchy supports granularity upgrades as finer data becomes available.

B.10 Resolution Hierarchy and Forward Compatibility

Every node class in the asset graph has an associated **resolution hierarchy**: a parent-child chain where the parent is the aggregate representation of the child. The Permian basin is a coarser representation of the same physical production system that its state-within-basin sub-nodes describe more finely, and those are in turn coarser than the individual leases and wells within them. A named pipeline is a coarser representation of the same physical conduit that its injection-point sub-nodes would describe more finely. A refining centre is a coarser representation of the same processing capacity that its individual refineries and, at a finer level still, their process units represent.

The coarsest and finest levels vary by node class. For production nodes, the range extends from basin at the top through state-within-basin and county-within-basin down to individual wells and leases. For pipeline nodes, the range extends from named pipeline through injection point and pump station down to individual segments. For storage nodes, the range extends from hub through individual terminal down to individual tank. For refinery nodes, the range extends from refining centre through individual refinery down to process unit. In each case the hierarchy is a chain of formulas: the coarser level is defined as an aggregation over the finer level, using the appropriate aggregation function — typically a sum for flows and stocks, a capacity-weighted sum for utilisation rates.

The design principle stated explicitly: every node type should have a resolution hierarchy that the formula layer can collapse upward or expand downward without touching the asset graph construction code. The hierarchy is metadata attached to the asset graph and consulted by the scenario loader; adding or removing a level does not change the set of physical nodes, only the paths over which aggregation and disaggregation are applied.

The hierarchy extends beyond named physical aggregates into administrative aggregates when — and only when — data resolution demands it. Consider a production variable for which the only available data source reports at PADD 3 level, with no finer basin or state breakout. Under the framework, the PADD 3 node from the observational layer of Section B.8 is **promoted** from a pure-view status to the authoritative representation of production for the duration of that scenario. The basins that would normally be authoritative in its place are **collapsed**: they remain in the asset graph, but their production variables are not independently observed, and their outbound edges to downstream pipelines are re-originated at load time from the PADD 3 aggregate. When finer data later becomes available — through a new data source or a backfilled historical series — the coverage contract is updated, the PADD 3 node reverts to a pure view, and the basins become authoritative again. The asset graph is unchanged throughout; only the coverage contract and the resulting scenario graph change.

This extension of the hierarchy into administrative aggregates is the bridge between the observational layer and the physical layer. It preserves the separation of concerns — administrative nodes do not acquire permanent edges in the physical graph — while

acknowledging that in practice the authoritative level for some variables, in some scenarios, is the administrative aggregate rather than a named physical asset.

The forward-compatibility consequence is significant: most published supply-chain graph schemas assume a fixed resolution and require structural changes when data quality improves. A schema built for basin-level data does not accommodate sub-basin data without modification. A schema built for individual refineries does not easily accommodate process-unit detail. In the present framework, by contrast, granularity upgrades are a matter of updating the coverage contract and the assignment table; the asset graph and the formula layer absorb the change automatically. This is a methodological choice rather than an implementation detail, and it is what makes the framework suitable for deployment in a setting where data sources are expected to improve over time.

B.11 Assignment Tables, the Observed/Derived Invariant, and Coverage Contracts

The assignment table is the data structure that links time series to nodes. Each row records that a particular time series — identified by its source, its variable type, its commodity grade, its coverage descriptor, and its resolution level — is to be assigned to a specific node for a specific variable slot. The scenario loader reads this table and produces the scenario graph.

For any given combination of node, variable, commodity, and timestamp, a single constraint governs the scenario graph:

Exactly one of **observed_authoritative** or **derived** holds. Additional observed series may be attached as **observed_auxiliary**, which are stored on the node but do not participate in the mass balance at their own level.

This is the **observed/derived invariant**. A node-variable is either populated directly from an assigned time series (observed authoritative), or computed from other nodes through a formula (derived), or populated from an assigned series that sits on the node as feature data without affecting flow accounting (observed auxiliary). These states are mutually exclusive at the schema level. The invariant is enforced by the data model rather than checked after the fact, which means the most common form of aggregation error — assigning one time series to a parent node and another to its children, thereby counting the same crude twice — cannot arise. If an assignment would violate the invariant, the scenario loader rejects the scenario and reports the conflict before any computation proceeds.

The **coverage contract** is the declaration that drives the invariant in practice. For each resolution hierarchy, the contract names the authoritative level. For the Permian basin, it might declare basin as authoritative; for Haynesville, where no basin-level data is available, it might declare PADD 3 aggregate as authoritative and place Haynesville in the collapsed state. Any series assigned at a non-authoritative level is either rejected or retained as observed auxiliary, depending on the loader's policy. The contract is scenario-specific: different scenarios can be assembled from the same asset graph with different authoritative levels for different subsystems, producing scenario graphs suited to different purposes.

The scenario loader's responsibilities follow from the above. It reads the assignment table, walks each resolution hierarchy to enforce the invariant, applies the coverage contract to mark nodes as authoritative, derived, or collapsed, re-originates flow edges where collapse requires it, propagates formulas through the formula layer to populate derived variables, runs the mass balance diagnostic of Section B.5 at the authoritative level for each subsystem, and emits the resulting scenario graph with provenance metadata attached to every variable instance. Provenance — the source, the release date, the resolution level, the coverage descriptor — is retained on every value so that, when a forecast is later audited, the exact state of knowledge at forecast time can be reconstructed. Figure B.7 shows the state machine for a single node-variable through the loader's pipeline.

Figure B.7 — State machine for a single node-variable under the scenario loader. Valid end states are observed_authoritative, derived, observed_auxiliary, and null. The invariant excludes any combination of observed_authoritative and derived on the same variable at the same time.

The invariant does not cover every failure mode. Sibling overlap — two assigned series whose coverage descriptors overlap geographically or temporally without one being an ancestor of the other — can still occur, and requires a pairwise coverage disjointness check across the assigned series at each level. The mass balance diagnostic at the authoritative level serves as a backstop for any remaining inconsistencies, surfacing them as large balancing items that the user can investigate. The invariant, the coverage disjointness check, and the mass balance diagnostic together form a layered defence against the aggregation and duplication errors that are otherwise endemic in multi-source commodity data pipelines.

B.12 Junction Nodes, Collapsed States, and Latent Allocation

The crude-oil network contains many physical points where flows commingle or split: gathering manifolds where crude from multiple wells combines before entering the long-haul system, pump stations where a pipeline picks up or delivers volume at intermediate points, junctions where one pipeline connects to another, tank farms that act as pass-through buffers between producers and pipelines. These are real physical assets with real identities, and in the asset graph they are represented as nodes.

At most practical data resolutions, however, the variables that would distinguish a junction node from its immediate neighbours are not separately observable. If the only data available is basin-level production and terminal-level receipts, there is no independent observation of what a gathering manifold between the two is holding or moving. Any values placed on the junction node would be derived from the upstream and downstream aggregates — carrying no independent information, and at worst introducing spurious precision.

The framework handles this through the **collapsed state**. A junction node marked collapsed remains structurally present in the asset graph, but its flow variables are defined as pass-through formulas from its upstream neighbours to its downstream neighbours, its stock is held at zero, and its configuration features are retained but do not participate in the GNN's neighbourhood aggregation. In effect the node becomes a transparent conduit at the current resolution. When finer data later becomes available — for example, station-level pipeline telemetry from a commercial provider — the coverage contract is updated, the junction node's collapsed flag is removed, and it wakes up with its own observed flow variables. The topology is never rebuilt; only the scenario graph's interpretation of the existing topology changes.

Collapse is triggered in two distinct circumstances. The first, already discussed in Section B.10, is when data resolution is coarser than the node's natural level — a basin with only PADD-level data available is collapsed for that scenario. The second, specific to junction nodes, is when the node's role in the physical network is one of pure transit between observable neighbours, with no independent variables that the current data can populate. In both cases the mechanism is the same: the node is present in the asset graph, inactive in the scenario graph, and ready to activate when appropriate data becomes available.

A distinct problem arises at junctions where flows split across multiple downstream routes. The Permian basin's production is known in aggregate; the individual pipelines leaving the Permian — Gray Oak, Cactus II, Midland-to-ECHO, Wink-to-Webster, Basin — have known nominal capacities but the actual split of Permian outflow across them in any given month is generally not reported in public data. The split is determined by a combination of pipeline tariffs, contractual commitments, operational constraints, and spot-market economics, none of which are directly observable at the aggregate data level.

This is the **latent allocation problem**. Its treatment is deferred to Chapter 5 on the forecasting framework, but its structural implications belong here. Because the split is latent, the framework does not attempt to infer it from rules; instead, it treats the per-edge flow variable at each junction as a learnable quantity, constrained by the mass balance (the edge flows must sum to the junction's observed outflow), by pipeline capacity ceilings (no edge flow may exceed the receiving pipeline's nominal capacity), and by any partial observations that may exist (pipeline-specific tariff filings, downstream terminal receipts that can be attributed to specific pipelines). This is a natural use case for attention-based temporal graph models: GAT and TGAT learn attention weights over neighbours conditional on node state, and the Permian-to-pipeline allocation problem is exactly of that form. The attention weights on the outbound edges from the Permian node, learned from historical co-movement of basin production and downstream terminal inflows, approximate the latent allocation.

Framing this in the annex, rather than only in the forecasting chapter, clarifies why the asset-centric graph is the right representation even when the edge-level flows cannot be directly observed. The graph structure encodes the set of physically possible flows; the observed data constrains the aggregate sums; and the GNN learns the distribution of flow across the possible routes. Without the asset-centric graph, there would be nothing for the attention mechanism to attend over, and the allocation would have to be handled outside the model through ad-hoc rules.

Figure B.8 — A junction node under two different coverage contracts. In the basin-only contract, the Permian gathering node is collapsed and flows pass through transparently from the basin to the downstream pipelines, with the split learned as a latent allocation. In the station-level contract, the gathering node is active and its per-edge outflows are observed directly from commercial telemetry.

B.13 Summary and Position in the Thesis

Sections B.1 through B.6 establish the asset-centric convention and the universal mass balance with a balancing item. Sections B.7 through B.12 extend that foundation with the machinery needed to operate the framework at production scale: the separation of a persistent asset graph from the scenario graphs derived from it, the distinction between geography metadata as labels on physical nodes and a resolution hierarchy as relationships between nodes, the observed/derived invariant that makes aggregation double-counting structurally impossible, the coverage contracts that allow administrative aggregates to be promoted to authoritative status when no finer data exists, and the collapsed-state mechanism for junction nodes whose variables cannot be distinguished at the current resolution.

These extensions are not incidental engineering. They are the methodological contributions that make the asset-centric framework a practical basis for forecasting in an environment where data sources are heterogeneous, partial, and time-varying. Published graph schemas for supply chains typically assume a single fixed resolution and break when data quality improves; the present framework absorbs granularity upgrades through the coverage contract without modification to the asset graph, and it treats mixed-resolution scenarios as a first-class case rather than an exception. The observed/derived invariant recasts the most common source of error in multi-source commodity data — aggregation double-counting — from a runtime validation concern to a schema-level property the data model cannot violate. The collapsed-state mechanism allows the asset graph to be defined once at the full level of physical detail, with scenario-specific inactivity handled at load time rather than through topology edits.

The framework is taken up in Chapter 5 on the forecasting framework, where the latent allocation problem introduced in Section B.12 becomes the central modelling challenge and the attention-based temporal graph models discussed in Section 4.4 become the methodological solution. The scenario graph emitted by the loader is the direct input to those models; the observed/derived invariant ensures that the inputs they receive are internally consistent; and the coverage contract mechanism is what allows the same model to be evaluated on historical periods with coarser data and recent periods with finer data, without retraining from scratch.

References

Energy Information Administration (EIA). Weekly Petroleum Status Report. U.S. Department of Energy. https://www.eia.gov/petroleum/supply/weekly/

Fattouh, B. (2011). An Anatomy of the Crude Oil Pricing System. Oxford Institute for Energy Studies, WPM 40.

Kazemi, Y. and Szmerekovsky, J. (2015). Modeling Downstream Petroleum Supply Chain. Transportation Research Part E, 83, pp. 111--125.

Li, Y., Yu, R., Shahabi, C., and Liu, Y. (2018). Diffusion Convolutional Recurrent Neural Network. ICLR.

Stopford, M. (2009). Maritime Economics. 3rd ed. Routledge.

Wasi, A. T., Islam, S. R., Karim, R., and Brintrup, A. (2024). SupplyGraph: A Benchmark Dataset for Supply Chain Planning using Graph Neural Networks. arXiv:2401.15299.

Yu, B., Yin, H., and Zhu, Z. (2018). Spatio-Temporal Graph Convolutional Networks. IJCAI.

Annex C: Resolver Dispatch Rule Reference

This annex provides a concise reference for the resolver's dispatch rules, intended as a lookup aid for readers working with the implementation. The narrative treatment of the resolver appears in Chapter 6; this annex strips out the discussion and presents the rules in compact form.

C.1 Rule priority and short-circuit behaviour

The resolver evaluates each variable through a fixed priority list. The first rule that successfully produces a value short-circuits the remaining rules. The unresolved fallback (Rule 9) should fire zero times in a healthy run; non-zero unresolved counts are bugs and require operator action.

Priority	Rule	Trigger condition	Output source
1	Observed	timeseries_id IS NOT NULL	observed
2	Zero	formula = '0'	zero
3	Latent (with reverse-mirror promotion)	formula = 'latent()'	latent or reverse-mirror
4	Sum	formula = 'sum'	derived
5	Single-variable alias	formula = bare_variable_id	derived
6	Reverse mirror (standalone)	paired direction resolved, this direction has no own formula	derived
7	Closure formula	balancing item with mixed-type inputs in formula_inputs	derived
8	Arithmetic	formula matches signed-variable-references pattern	derived
9	Unresolved (fallback)	none of the above matched	unresolved

C.2 Rule details

C.2.1 Rule 1: Observed

The variable carries `timeseries_id IS NOT NULL`. The rule performs a direct lookup against `timeseries_data` at the observation date. The lookup is a primary key access on `(timeseries_id, observation_date)`.

Output: `value = <time series value>`, `source = 'observed'`, `formula_used = <timeseries_id>`.

Failure mode: The time series has no value at the observation date. The current implementation records `value = NULL`, `source = 'observed'` with a NULL value as a

transient state, which the L4 views flag as a partial coverage warning. Future revision will distinguish "observed but null" from "unresolved".

C.2.2 Rule 2: Structural zero

The variable carries `formula = '0'`. Used for pass-through node defaults inherited from `node_type_default_formulas`: pipeline P, C, ΔS, B; refinery outflow; gathering P (within-basin); and other physical structural zeros.

Output: `value = 0`, `source = 'zero'`, `formula_used = '0'`.

Failure mode: None. This rule cannot fail.

C.2.3 Rule 3: Latent with reverse-mirror promotion

The variable carries `formula = 'latent()'`. The rule first attempts a reverse-mirror promotion: if the paired direction of the variable (opposite variable type, swapped endpoints) has a resolved value (and is itself not plain latent), the rule borrows that value through the mirror identity.

Outputs depending on the promotion:

- Mirror succeeded: `value = <paired direction's value>`, `source = 'reverse-mirror'`, `formula_used = 'reverse_mirror_promotion'`.
- Mirror failed: `value = NULL`, `source = 'latent'`, `formula_used = 'latent()'`.

The mirror-promotion logic exists because Rule 3's natural priority position (before Rule 6) would otherwise short-circuit and prevent the standalone Rule 6 from firing for explicitly-latent variables.

Failure mode: None at the rule level. Downstream LP consumers must treat `source = 'latent'` rows as decision variables, not as constants.

C.2.4 Rule 4: Sum

The variable carries `formula = 'sum'`. The rule sums every value in `formula_inputs`; NULL inputs are skipped (treated as zero contribution).

Output: `value = Σ inputs`, `source = 'derived'`, `formula_used = 'sum'`.

Failure mode: All inputs are NULL. The rule produces `value = 0` rather than NULL, which is potentially misleading. Operators should check the `formula_inputs` structure when a sum variable resolves to zero unexpectedly.

The rule is the twelfth-pass collapse of three earlier labels (`sum_over_children`, `sum_over_outflows`, `sum_same_type`). The semantic role of any given sum is recoverable from the structure of `formula_inputs` and does not need to be encoded in the formula text.

C.2.5 Rule 5: Single-variable alias

The variable carries `formula = <bare_variable_id>` (a bare identifier matching exactly one entry in `formula_inputs`). The rule inherits the value of the named target.

Output: `value = <target value>`, `source = 'derived'`, `formula_used = 'alias(<bare_variable_id>)'`.

Failure mode: The target is not resolved at this point in the dispatch loop. This should not happen if the dependency DAG is built correctly; if it does, the variable falls through to Rule 9 (unresolved).

C.2.6 Rule 6: Reverse mirror (standalone)

The variable has no own resolution path (no time series, no formula matching Rules 1 through 5) and its paired direction has a resolved value. The rule borrows the paired direction's value.

Output: `value = <paired direction's value>`, `source = 'derived'`, `formula_used = 'reverse_mirror'`.

Failure mode: The paired direction is itself unresolved. The variable falls through to Rule 9.

C.2.7 Rule 7: Closure formula

The variable is a balancing item whose `formula_inputs` mix inventory, inflow, and outflow variables. The rule evaluates the closure equation:

$$B = \Delta S - P + C - \Sigma F_{in} + \Sigma F_{out}$$

Output: `value = <computed B>`, `source = 'derived'`, `formula_used = 'closure'`.

This rule is dormant in the starter scenario because B was promoted to time-series-observed at the PADD and USA aggregates in the sixth migration pass. The rule remains in the resolver as a fallback and as the engine behind the observed-vs-closure-B comparison view of Section 7.4.

C.2.8 Rule 8: Arithmetic combination

The variable carries `formula = <signed variable references>`, for example `padd2_view.P − bakken_nd.P − oklahoma.P`. The rule parses the signed terms via a regular expression that extracts (sign, variable_id) pairs and aborts if any term fails to match a variable in `formula_inputs`.

Output: `value = Σ (sign × variable value)`, `source = 'derived'`, `formula_used = <original formula string>`.

Failure mode: Parser fails to match the formula text. The variable falls through to Rule 9.

C.2.9 Rule 9: Unresolved (fallback)

None of the above rules matched.

Output: `value = NULL`, `source = 'unresolved'`, `formula_used = NULL`.

Operator action required. A non-zero unresolved count after a run indicates that either the assignment set references variables that do not exist, or the formula text matches no rule pattern. The recommended response is to inspect the audit row's dispatch counts, query `scenario_resolved_values WHERE source = 'unresolved'` to identify the failing variables, and either fix the assignment (correct the reference) or extend the resolver (add a rule for the new pattern). Silent treatment as latent is explicitly prohibited.

C.3 The dependency DAG

The resolver's `build_deps()` function produces a dictionary `deps[variable]` of upstream variable IDs that each variable depends on. The topological sort over this dictionary orders the variables so that each is resolved only after its dependencies.

Three sources contribute to the dependency dictionary:

Source	Trigger	Dependency added
Formula inputs	Variable has a non-empty formula_inputs list	Every variable ID in the list that exists in the assignment set
Fan-out at same node	Variable's formula is a sum over outflows	Every outflow variable at the same node, except itself
Reverse mirror, hoisted	Variable is latent and paired direction is resolved	The paired direction's variable ID

Two cycle gates protect the reverse-mirror addition:

Gate	Condition
Paired side is non-latent	Without this, latent ↔ latent forms a two-node cycle
Paired side does not already alias from us	Without this, this side aliases from paired, paired aliases from this side → cycle

A cycle detected by the topological sort raises a `CycleError` and halts the resolver. The audit row records the failure with a non-NULL `failed_at` timestamp.

C.4 Healthy-run signature

The dispatch counts in a healthy run on the starter scenario are stable across repeated runs:

Rule	Expected count
observed	80
zero	628
latent	652
alias	442
reverse-mirror	15
arithmetic	8
sum	5
closure	0
unresolved	0
Total	1,870

Deviations from this signature in any rule other than `observed` (which scales linearly with the count of bound time series) indicate a structural change. The dispatch counts are recorded in `scenario_resolver_runs.dispatch_counts` as JSONB and can be diffed across runs with standard SQL.

Annex D: Variable Catalogue and Starter Scenario Inventory

This annex provides a structured inventory of the variables in the starter scenario `starter_us_crude_2015_2025`, intended as a reference for readers working with the implementation. The annex is not exhaustive — the full assignment set carries 1,870 variables — but it is comprehensive enough to give a clear picture of how the framework's principles materialise in the actual data model.

D.1 Node inventory by class

Class	Count	Examples
Production (physical)	19	bakken_nd, bakken_mt, permian_tx, permian_nm, eagle_ford_tx, oklahoma_conventional, california_conventional, wyoming, padd1_other through padd5_other, gulf_of_america
Refining (physical)	24 (active; 115 total slots)	padd1_refinery_agg, padd2_refinery_agg through padd5_refinery_agg, 19 named refineries (Motiva Port Arthur, ExxonMobil Beaumont, Marathon Galveston Bay, BP Whiting, Marathon Detroit, Valero refineries, and others)
Storage (physical)	10	cushing_hub (with three operator sub-terminals: enterprise_cushing, plains_cushing, magellan_cushing), patoka_hub, houston_storage, st_james_terminal, loop_terminal, others
SPR (physical)	4	bayou_choctaw, big_hill, bryan_mound, west_hackberry
Pipeline (physical)	37	seaway_south, seaway_north, capline_south, capline_north, keystone_xl, marketlink, diamond, pipe_bakken_xstate, 12 inter_padd__to__agg aggregates, and others
Import terminal (physical)	8	clearbrook, padd1_imports_agg through padd5_imports_agg, padd1_canadian_imports_agg, padd2_canadian_imports_agg, padd2_xstate

Class	Count	Examples
Export terminal (physical)	12	ingleside, houston_exports_agg with three sub-terminals (enterprise_houston, magellan_houston, moda_midstream_houston), nederland, loop_exports, st_james_exports, padd1 through padd5 export aggregates
Gathering (physical)	6	ans, bakken_nd_gathering, bakken_mt_gathering, eagle_ford_gathering, permian_nm_gathering, permian_tx_gathering
Origin terminal (physical)	6	midland, wink, crane, johnsons_corner, three_rivers, valdez
Region view (abstract)	6	usa_view, padd1_view through padd5_view
Refining district view (abstract)	10	refining_district_1A, 1B, 2A, 2B, 2C, 3A, 3B, 3C, 4_west, 5_combined
State view (abstract)	2	texas_state_view, montana_state_view
Observational aggregate (abstract)	4	spr_total, permian_aggregate, bakken_aggregate, eagle_ford_aggregate
USA subtotal view (abstract)	1	usa_lower48_excl_gom_view
Boundary	4	foreign_supply, canadian_oil_sands, foreign_export_destination, spr_total (also boundary-functional)
Total	240	

D.2 Variables by type

The starter scenario carries 1,870 variables, distributed across the seven variable types as follows:

Type	Symbol	Count	Note
Production	P	251	One per (node × commodity), with non-applicable types set to zero
Consumption	C	251	Same structure as P
Inventory	S	251	Same structure as P
Balancing item	B	251	Same structure as P

Type	Symbol	Count	Note
Inflow	F_in	433	One per directed inbound edge per node
Outflow	F_out	433	One per directed outbound edge per node (matches F_in count)
Phi (reference)	φ	0	Used for cross-node alias chains in derived aggregates
Total		1,870	

Each non-relational variable type carries exactly one row per node, since the starter scenario operates with a single commodity (crude). The phi count of zero reflects the rollback of an interim grade-decomposition trial that was carried out during development to validate the schema's multi-commodity handling; phi variables and per-grade decomposition will be reintroduced when grade-level work becomes the next workstream.

D.3 Authoritative bindings by subsystem

The 90 time-series-observed variables in the starter scenario are distributed across the U.S. crude system as follows:

Subsystem	Variable type	Authoritative resolution	Count of bound variables	Lead EIA series
Production	P	Basin level + state	15	MCRFPP1, MCRFPP2, MCRFPP3, MCRFPP4, MCRFPP5, MCRFPP3TX, MCRFPP3NM, others
Refinery throughput	C	PADD level	5	MCRRIP1 through MCRRIP5
Stocks (commercial)	S	PADD level	5	MCESTP1 through MCESTP5
Stocks (SPR sites)	S	Site level	4	MCESTUS1 series (decomposed)
Stocks (Cushing hub)	S	Hub level	1	Cushing weekly aggregated to monthly
Imports (non-Canada)	F_in	PADD level	5	MCRIPP12 minus Canadian contribution
Imports (Canada)	F_in	PADD level	2	MCRIPP1CA2 (PADD 1 Canadian),

Subsystem	Variable type	Authoritative resolution	Count of bound variables	Lead EIA series
				MCRIPP2CA2 (PADD 2 Canadian)
Exports	F_out	PADD level	5	MCREXP12 through MCREXP52
Inter-PADD flows	F (aggregate)	PADD-pair level	12	MCRMP* family across all pairs
Balancing item	B	PADD level + USA	6	MCRUA-P1 through MCRUA-P5 and MCRUA-USA
USA aggregates	P, C, S, F_in, F_out, B	USA total	6	MCRFPUS2, MCRRIUS2, MCESTUS1, MCRIMUS2, MCREEXUS2, MCRUA
Texas/Montana state aggregates	P	State	2	Texas state series, Montana state series
Auxiliary observed (not in mass balance)	S	PADD level	12	MCRSFP (PADD ex-SPR), MCRSSP (SPR), others
Total			90	

The 90 authoritative bindings collectively produce all 1,870 resolved values through the resolver's dispatch logic.

D.4 Latent variables by location

The 652 latent variables in the starter scenario are concentrated at junction nodes where per-edge flows are not publicly observed. The distribution:

Junction or subsystem	Latent count	Reason
Permian fan-out (basin → origin terminals → trunk pipelines)	47	Per-pipeline splits between Midland, Wink, Crane terminals and downstream Gulf Coast pipelines
Bakken fan-out (gathering → trunk pipelines to PADD 2 and beyond)	32	Per-pipeline splits out of Bakken gathering systems
Bakken cross-state connector	4	All four flow variables on pipe_bakken_xstate (eleventh migration pass)
Cushing hub sub-decomposition	18	Per-operator-sub-terminal splits within the hub (only aggregate stocks observed)

Junction or subsystem	Latent count	Reason
Houston exports sub-decomposition	24	Per-sub-terminal splits within the Houston aggregate
Inter-PADD aggregate per-pipeline splits	196	Each inter-PADD aggregate has multiple constituent pipelines; aggregate flow observed, per-pipeline split latent
SPR per-site outflows	16	Observed only at SPR-total level when released; per-site decomposition latent
Refinery per-pipeline-receipt flows	184	Refineries observed at PADD-aggregate level for receipts; per-source-pipeline splits latent
Origin terminal outflows to multiple downstream pipelines	89	Midland, Wink, Crane outflows to multiple trunk pipelines, latent at the per-pipeline level
Offshore production routing	28	Gulf of America platforms route through multiple offshore-to-shore systems, splits unobserved
Other (smaller subsystems)	14	Foreign aggregate boundary edges, miscellaneous latents
Total	652	

These latents are the natural decision variables for the linear programme described in Chapter 8. Each carries a structural relationship to the variables around it via mass balance and capacity constraints, but its value is not pinned down by observation.

D.5 Structural zeros by node type

The 628 structural zero variables are inherited from `node_type_default_formulas`. The distribution by node type:

Node type	Structural zeros per node	Total nodes	Zero variables
Pipeline	4 (P, C, ΔS, B; the four pass-through quantities)	37	148
Gathering	4 (P, C, ΔS, B if no own production) + 1 (P for gatherings within their basin: zero if basin-level P is authoritative elsewhere)	6	24
Refinery	1 (P; refineries do not produce crude)	24	24

Node type	Structural zeros per node	Total nodes	Zero variables
Storage terminal	4 (P, C, ΔS for pass-through stocks, B)	10	40
SPR site	2 (P, C; SPR sites do not produce or consume)	4	8
Import terminal	4 (P, C, ΔS for pass-through, B)	8	32
Export terminal	4 (P, C, ΔS for pass-through, B)	12	48
Origin terminal	4 (P, C, ΔS, B for pass-through)	6	24
Boundary source	C, ΔS, B	3 (foreign_supply, canadian_oil_sands, plus structural slots)	9
Other (region views, observational, mixed)	various	130	271
Total			628

The total of 628 matches the dispatch count of `zero` rule firings in the starter scenario.

D.6 Migration history

The starter graph reached its current state through twenty-three migration passes. The major passes are summarised here:

Pass	Description	Result on dispatch counts
1	Initial seed load from asset_graph.json	Baseline ~1,200 variables, ~50% unresolved
2	Populate node_type_default_formulas; resolve pass-through zeros	Unresolved drops to ~30%
3	Add PADD-level residual nodes; assign EIA PADD aggregates	Unresolved drops to ~20%; arithmetic rule first appears
4	Wire inter-PADD aggregates; add the MCRMP__ series	Aliases proliferate; reverse-mirror first fires (3 cases)
5	Add SPR sub-nodes; promote SPR site stocks to authoritative	SPR-aggregate becomes derived; 4 new observed bindings
6	Promote balancing item B to time-series-observed at PADD and USA	Closure rule fires drop from ~15 to 0
7	Add origin terminal nodes (Midland, Wink, Crane); declare Permian fan-out latents	Latent count rises to ~400
8	Fix latent-mirror cycle; add cycle gates to build_deps()	Resolver runs to completion without CycleError
9	Hoist reverse-mirror dependency above early-continue; promote mirror inside Rule 3	Reverse-mirror dispatch lifts from 3 to 15

Pass	Description	Result on dispatch counts
10	Add Bakken cross-state connector node; declare its four flow latents	Latent count rises to ~640
11	Sub-decompose Houston exports; reorganise Cushing hub sub-terminals	Latent count reaches 652
12	Collapse three sum labels (sum_over_children, sum_over_outflows, sum_same_type) into canonical sum	Dispatch surface reduced from 11 labels to 9
13	thirteenth_pass_views.py: create the layered materialised views (v_flow_edges, v_aggregation_edges, v_partition_tree, v_node_status, v_node_balance_check, v_aggregation_consistency, v_inventory_changes, v_aggregate_balance, v_node_pcisob, v_formula_input_links, v_partition_sums, v_partition_intra_flows)	Structural and analytic views become queryable; renderers and audits read these instead of joining variable_assignments directly
14	sixteenth_pass_cleanup.py: JSONB attribute cleanup and introduction of v_effective_assignments override semantics	Per-scenario overrides representable without rewriting base assignment rows
15	seventeenth_pass_xstate_membership.py: declare pipe_bakken_xstate as an inventory constituent of its natural parents	Bakken cross-state connector enters the partition tree; closure tightens at PADD-2 inflow
16	eighteenth_pass_constraint_membership.py: declare seven constraint nodes as constituents of their natural parents	Boundary-functional constraint nodes join the partition spine
17	repoint_foreign_supply_to_imports_agg.py and repoint_canadian_corridor_ts.py: move foreign-supply TS authority to the physical aggregate and Canadian corridor TS to the pipeline nodes themselves	Authoritative anchors move from ambiguous aggregate-level to specific physical nodes
18	add_padd2_canadian_imports_agg.py and add_partition_aggregates.py: add the PADD-2 Canadian-imports prototype, then generalise to all PADD aggregates (19 abstract nodes added)	Inter-PADD movement becomes natively addressable in the partition tree
19	split_bakken_gathering.py and wire_bakken_xstate_into_inter_padds.py: split Bakken gathering into ND/MT components and wire the cross-state connector into the inter-PADD aggregates	Bakken cross-state flow becomes explicit; closure improves on PADD 2/4 movement
20	twenty_first_pass_spr_under_padd3.py and promote_spr_total_to_balance_role.py: place SPR sites under PADD-3 inventory and promote the SPR total from constraint to balance role	USA-level double-counting of SPR resolved; SPR aggregates correctly via PADD 3
21	add_padd_stock_decomposition.py: per-PADD decomposition of stocks into tank-farms-and-pipelines and refinery-stocks components	Stocks decompose into authoritative sub-components; path opens to per-facility data when sourced

Pass	Description	Result on dispatch counts
22	refactor_jones_act_into_inter_padd_agg.py, patch_pipeline_production_capacities.py and patch_pipeline_timeline.py: move Jones-Act in-transit onto the inter-PADD aggregate; backfill pipeline and producer capacity for the present and across 2015-2025	Capacity layer populated; capacity violations at 2024-12-01 drop to 0
23	add_node_roles.py, add_aggregation_constituents.py, patch_v_aggregation_consistency_missing_data.py, patch_production_caps_from_peaks.py, add_canadian_pipelines_inventory_membership.py and build_us_crude_grade_map.py: role tagging (balance/constraint/auxiliary), aggregation-constituent cross-checks, partial-coverage classification for missing data, production-cap headroom from historical peaks, Canadian-pipeline inventory membership, and the crude-grade hierarchy seed (currently dormant after the grade-trial rollback)	Consistency layer finalised; grade-registry placeholder remains for future work

Each migration pass is implemented as a Python script in the `migrations/` directory and is replayable from a fresh database. The current state of the starter scenario is the result of running all twenty-three passes in sequence.

D.7 Summary statistics

For reference at a glance:

- **Nodes:** 251 (217 physical, 34 abstract; including 3 boundary)
- **Edges:** 433 directed flow edges
- **Variables:** 1,870 under the active scenario
- **Time-series bindings:** 80 (in 1:1 attribution)
- **Observation dates:** 156 (monthly, January 2015 to December 2024)
- **Resolved (variable, date) pairs:** 291,564
- **Resolver runtime:** approximately 20 seconds on a developer laptop
- **Partition closure gap:** 0 kbd at every PADD aggregate and at the USA aggregate
- **Unresolved variables:** 0 in the current healthy state

References

Ahn, S., Choi, J. and Park, S. (2024) "Probabilistic supply chain prediction using graph neural networks", Journal of Supply Chain Management.

Fattouh, B. (2011) An Anatomy of the Crude Oil Pricing System, Oxford Institute for Energy Studies, WPM 40.

Fernandes, L.J., Relvas, S. and Barbosa-Póvoa, A.P. (2013) "Strategic network design of downstream petroleum supply chains: single versus multi-entity participation", Chemical Engineering Research and Design, 91(8), pp. 1557–1587.

Ghaithan, A.M., Attia, A. and Duffuaa, S.O. (2017) "Multi-objective optimization model for a downstream oil and gas supply chain", Applied Mathematical Modelling, 52, pp. 689–708.

Grover, A. and Leskovec, J. (2016) "node2vec: scalable feature learning for networks", in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864.

Hamilton, W., Ying, Z. and Leskovec, J. (2017) "Inductive representation learning on large graphs", in Advances in Neural Information Processing Systems 30, pp. 1024–1034.

Jin, M., Koh, H.Y., Wen, Q., Zambon, D., Alippi, C., Webb, G.I., King, I. and Pan, S. (2024) "A survey on graph neural networks for time series: forecasting, classification, imputation, and anomaly detection", IEEE Transactions on Pattern Analysis and Machine Intelligence.

Kazemi, Y. and Szmerekovsky, J. (2015) "Modeling downstream petroleum supply chain: the importance of multi-mode transportation to strategic planning", Transportation Research Part E, 83, pp. 111–125.

Kipf, T.N. and Welling, M. (2017) "Semi-supervised classification with graph convolutional networks", in International Conference on Learning Representations.

Kosasih, E.E. and Brintrup, A. (2022) "A machine learning approach for predicting hidden links in supply chain with graph neural networks", International Journal of Production Research, 60(17), pp. 5380–5393.

Li, Y., Yu, R., Shahabi, C. and Liu, Y. (2018) "Diffusion convolutional recurrent neural network: data-driven traffic forecasting", in International Conference on Learning Representations.

Lima, C., Relvas, S. and Barbosa-Póvoa, A.P. (2016) "Downstream oil supply chain management: a critical review and future directions", Computers and Chemical Engineering, 92, pp. 78–92.

Perozzi, B., Al-Rfou, R. and Skiena, S. (2014) "DeepWalk: online learning of social representations", in Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710.

Ribas, G.P., Hamacher, S. and Street, A. (2010) "Optimization under uncertainty of the integrated oil supply chain using stochastic and robust programming", International Transactions in Operational Research, 17(6), pp. 777–796.

Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F. and Bronstein, M. (2020) "Temporal graph networks for deep learning on dynamic graphs", in ICML 2020 Workshop on Graph Representation Learning.

Stopford, M. (2009) Maritime Economics. 3rd edn. London: Routledge.

Tang, J., Qu, M., Wang, M., Zhang, M., Yan, J. and Mei, Q. (2015) "LINE: large-scale information network embedding", in Proceedings of the 24th International Conference on World Wide Web, pp. 1067–1077.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. and Bengio, Y. (2018) "Graph attention networks", in International Conference on Learning Representations.

Wang, Y., Cai, Y., Liang, Y., Ding, H., Wang, C., Bhatia, S. and Hooi, B. (2021) "Inductive representation learning in temporal networks via causal anonymous walks", in International Conference on Learning Representations.

Wasi, A.T., Islam, M.A. and Akib, A.R. (2024) "SupplyGraph: a benchmark dataset for supply chain planning using graph neural networks", in AAAI 2024 Workshop on Graphs and More Complex Structures for Learning and Reasoning.

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C. and Yu, P.S. (2020) "A comprehensive survey on graph neural networks", IEEE Transactions on Neural Networks and Learning Systems, 32(1), pp. 4–24.

Xu, D., Ruan, C., Korpeoglu, E., Kumar, S. and Achan, K. (2020) "Inductive representation learning on temporal graphs", in International Conference on Learning Representations.

Yu, B., Yin, H. and Zhu, Z. (2018) "Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting", in Proceedings of the 27th International Joint Conference on Artificial Intelligence, pp. 3634–3640.